The CIRIS Constitution

The Constitution, a volume of edition 1.0-rc18, released as guidance

Stewarded by Eric Moore

The Constitution

What this is. The power volume: who confers standing, who may halt, who may amend, how a person leaves, and what every named power is paired with. It gathers the admission gate (1.2), the community and trust-root rules (3.2), the accord layer and the halt (4.2), the Wise Authorities (4.3), amendment (4.5.1), and the governance discipline of moderation, redress, child safety, takedown and purge (4.5).

Whom it binds. The holders of the accord root, the stewards of agents, the founders and moderators of communities, the Wise who answer deferrals, and every node that admits their rows. Each power here is exercised as a signed row under a named key and is checked by another row.

What it promises. Every act of power is signed by a named person and can be contested. A halt is one holder's row, lapses in a day unless a majority confirms it, moves no trust state and touches nothing but an agent's act gate. A root can only tell a node to share less. Any person may found a root, and any person may leave one. Nobody who cannot answer for themselves is on the network alone. No hand bans a community; what is not hidden is stated, and withdrawal is node by node.

What it leaves out. The byte shapes of the rows these powers ride on, which are in *The Wire*; the family-by-family admission rules, which are in *The Dictionary*; and the question of which of these rules have code behind them today, which only *The Evidence Register* answers.

Where the rest lives. The reasons behind the design are in *The Charter*. The trust assumptions a reviewer should attack are in *The Threat Model*. The formal models that check the halt, the roster and the standing verdict are named in the register's witness column.

Part 1 — Foundation

1.2 The four-test prefix-admission gate

The namespace is open: anyone may publish a rule set and admit a new prefix. The discipline that keeps an open namespace honest — that stops it sliding from describing mechanisms toward enforcing preferences (the discipline named in CC 1.13.5) — is a four-test gate. Every prefix admitted to the CC 3.1 namespace MUST pass:

Test	Question	Pass criterion
T1	Is the prefix part of a published, hash-pinned, version-controlled rule set, distinct from per-attestation verdicts?	Rules + verdicts separated in writing
T2	Does the prefix name a **mechanism** (correlation, count, time-window, schema-conformance) rather than a **subjective quality** (deception, harm, virtue, trustworthiness, sin)?	Mechanism-descriptive prefix name
T3	Can past verdicts be re-checked against the rule version they ran against?	Version-pinning: the dimension's trailing `:v{N}` names the rule version (CC 3.1.7 R3); `evidence_refs[]` pins the calibration artifacts
T4	Is the prefix wired so its attestations are **never sole evidence** for `slashing:*`?	Adjudication separation

T2 is the most slip-prone gate, because judgment-words feel natural where mechanism-words should go. A prefix that fails T2 gets renamed to the mechanism it actually checks: the canonical case is `detection:emergent_deception:*` (a subjective quality) renamed to `detection:correlated_action:*` (a measurable mechanism). The full anti-pattern catalogue lives at CC 4.1.

Part 3 — The Namespace

3.2 `community` subject_kind

A `community` is a **larger node-collective with explicit admission semantics** — a sibling subject_kind to `family`, but with different defaults. Content scoped `cohort_scope: community` **replicates within the cohort** — to its members' claimed nodes under the one audience rule (CC 6.1.5.3) — and its holding claims stay inside the roster (CC 5.2 structural invisibility holds at every bounded scope; there is no non-member holder of community bytes). A community is not plaintext-by-default: its content is encrypted at rest under a per-community DEK (the cascade detailed below). The contrast with `family` is one of disclosure shape, not of whether encryption exists: families protect intimate trust circles by hiding even the existence of their content from outsiders; communities keep their bytes confidential to members while letting their *presence* federate at scale — provenance-visible discovery, which is the justice the larger-collective form is built for.

Communities differ from families along three axes:

	`family`	`community`
Scale	Household / intimate trust circle (typically $\leq$ 20 members)	City / professional / interest (10s to 100Ks of members)
At-rest encryption	Yes — per-write DEK cascade per CC 4.4.3.4.1; holding claims within the cohort only (CC 5.2 structural invisibility)	Yes — **mandatory** per-community DEK cascade per CC 4.4.3.2.1; holding claims within the roster only, so a non-member learns nothing (the paragraph above). Two carve-outs: `infrastructure` and `public` (CC 3.2) are Commons-plaintext with no DEK — the trust root must be publicly auditable, and a room anyone may join has no secret to keep
Subkind discriminator	None	`cohort_subkind` field (open vocab; canonical: `geographic`, `infrastructure`, `public`)
Typical admission	`founder_only` / `unanimous` for small intimate groups	`majority` / `weighted` / per-subkind protocol (e.g., geographic requires `location_proof`)

```
community {
  community_key_id: key_id // community's own federation key
  community_name: string // human-readable; non-unique
  cohort_subkind: string // open vocab; canonical: "geographic"
  members: [
    {
      key_id: key_id // member identity_key (NOT occurrence)
      joined_at: rfc3339_canonical
      role: Option<MemberRole> // founder | member | null
    },...
  ]
  founded_at: rfc3339_canonical
  consensus_protocol: ConsensusProtocol // the family's kinds less founder_only, which a community above
     the family never names (CC 4.5.13)
  consensus_protocol_entrenched: bool // same semantics as family
  cohort_subkind_payload: Option<SubkindPayload> // subkind-specific fields; see below
}
```

`cohort_subkind` is the discriminator — open vocabulary per CC 4.5.1.1 axis-vocabulary discipline. The three codified subkinds are `geographic`, `infrastructure` and `public` (below); operator vocabularies extend.

Canonical `cohort_subkind: geographic`

The first codified community subkind. Membership additionally requires the candidate to emit a valid `location_proof` (CC 3.3.3) whose `cell_id` is contained (CC 2.6.6.2) within the community's `geographic_constraint.cell_id`.

The `cohort_subkind_payload` for `geographic`:

```
geographic_constraint {
  cell_id: string // H3 cell, lowercase hex per CC 2.6.6
  cell_resolution: u8 // 0-15; community-side may be any
    // resolution (NOT bounded to <= 7;
    // that bound applies only to
    // location_proof emissions)
}
```

Worked example — Austin geographic community:

```
community {
  community_key_id: "austin-community",
  community_name: "Austin",
  cohort_subkind: "geographic",
  cohort_subkind_payload: {
    geographic_constraint: {
      cell_id: "85283473fffffff", // H3 res 5, ~250 km^2 covering Austin metro
      cell_resolution: 5
    }
  },
  members: [
    {key_id: alice_root_key, role: founder},
    {key_id: bob_root_key, role: founder},...
  ],
  consensus_protocol: "majority",
  consensus_protocol_entrenched: false
}
```

For Alice to join Austin, she emits a `location_proof` with a `cell_id` (resolution 7 — ~5 km^2) that is contained within the Austin community's `85283473fffffff` (resolution 5) constraint. The community's `majority` admission rule then evaluates her membership proposal.

Privacy as opt-in: joining a geographic community is a **one-way disclosure**. When the member leaves, the `location_proof` is withdrawn like any row of the subject's own; the audit chain keeps the tombstone, never the cell (CC 4.5.9); rough-only is wire-format-enforced (CC 2.6.6.1). The substrate's role is to enforce the opt-in mechanically — produce a `location_proof` to join; cannot emit finer than rough. This is autonomy in mechanism: the user discloses to belong, never more than rough, and the substrate cannot extract more.

Membership change ceremony: same shape as family — a new roster version (CC 3.2 T6), never a single-signer `supersedes`; admission gated by current `consensus_protocol`; additionally for `geographic` subkind, the candidate's most-recent `location_proof` (within `valid_until`) MUST be contained in the geographic_constraint.

Substrate emissions on community events:

- `hard_case:community_membership_change`
- `hard_case:community_consensus_protocol_change`
- `hard_case:community_consensus_protocol_violation`

All three reserved under CC 3.4.2.

Community DEK cascade: community content (`cohort_scope: community |`

`affiliations`) is encrypted at rest under a **per-community DEK**; its holding claims (`custody:ack:v1`, CC 3.1.3.3) are placed at the community's scope and reach its members only, and there is no non-member holder of community bytes (CC 6.1.5.3). The community DEK follows the CC 5.1 epoch-DEK cascade: one DEK shared across emissions (per-emission cost **O(1)**, not O(members)), wrapped to each member on admission, a fresh epoch DEK after a removal, history never re-encrypted (CC 4.4.3.4.5); **`wrap_algorithm: v2` (hybrid PQC) MANDATORY** (same harvest-now-decrypt-later reasoning as self/family — CC 4.4.3.4.1). The privacy property for communities is **byte-level confidentiality to members + provenance-visible discovery**. **Two carve-outs: `infrastructure` and `public` (CC 3.2).** A `community` with `cohort_subkind: infrastructure` (`ciris-canonical` / governance roots, CC 3.2 below) **opts out** — Commons-tier plaintext, because the trust root must be maximally inspectable (see CC 4.4.3.2.1 for the three-tier model + holder-inspectability rationale); a `public` room is Commons-tier plaintext because a roster anyone may join has no confidentiality claim against the public.

Per-community `archive_mode` — key-retention policy

The DEK cascade above fixes *how* community content is encrypted; `archive_mode` fixes *how long it stays readable*. It is a per-community config field governing whether the community's per-epoch content keys (the cohort DEK cascade of CC 5.1 that seals its stored chunks and, for its realtime streams, `K_record_id` / `K_symbol` from the MLS exporter per CC 5.4.1) survive past their MLS epoch or are deleted — i.e. whether community content remains individually recoverable or descends below the CC 6.1.2 noise floor as epochs advance. MLS epoch advance bounds the lifetime of those keys; `archive_mode` chooses what holders do with the past-epoch keys at each advance.

```
community {
  ...
  archive_mode: ArchiveMode // OPTIONAL; default `rotate-forward`
}

ArchiveMode in { rotate-forward, retain }
```

`archive_mode` MUST be surfaced at community creation. The two canonical modes are:

- **`rotate-forward`** (default; 30-day window): honest holders MUST delete past-epoch content keys, the epoch DEKs and `K_record_id` / `K_symbol`, after the window. Forward-secrecy here is **honest-holder discipline, not a cryptographic guarantee** — a holder under coercion or running modified software MAY retain past-epoch keys, and this retention is structurally undetectable. Content under `rotate-forward` ages out: as epochs advance and keys are deleted, past-epoch content becomes unrecoverable to honest holders — the community-key instance of the CC 6.1.2 monotonic-descent retirement operator, where natural aging is the slowest descent toward the noise floor.
- **`retain`**: holders retain past-epoch keys indefinitely for archive readability. Content stays above the noise floor for the community's lifetime; an adversary compromising a member at epoch `N+k` recovers everything back to that member's join epoch.

`archive_mode` is **operator-local config, NOT federated trust state** — it rides no attestation, is not part of the community's signed roster or `consensus_protocol`, and is observable under compelled disclosure on the same terms as any other operator-local config. A community requiring high-sensitivity participation treats the chosen `archive_mode` as a known-leak class and segregates deployment accordingly. Because `rotate-forward`'s forward-secrecy is non-cryptographic,

a community whose threat model requires guaranteed past-epoch unreadability MUST NOT rely on `archive_mode` alone; key deletion is the policy, holder honesty is the precondition.

Why this is still 1+4: `archive_mode` adds one OPTIONAL config field to the `community` subject_kind and one closed enum (`rotate-forward` | `retain`). It rides the existing community config + DEK cascade (CC 5.1 epoch-DEK) and the existing CC 6.1.2 retirement operator; key-deletion-after-window is a *holder behavior over existing keys*, not a new structural primitive. Zero new structural primitives.

Informative worked example — civic-engagement + emergency-messaging composition pattern:

The community primitive plus location proofs, identity authority, and cohort-scoped distribution compose cleanly for both civic / democratic participation AND emergency / public-safety messaging. None require new structural primitives; all ride the existing 1+4 set plus the namespace additions. The two surfaces share the same underlying primitives but differ in authority/priority shape — civic is bottom-up democratic participation; emergency is top-down authoritative broadcast.

Civic shape	CEG composition
Neighborhood association	`community` with `cohort_subkind: geographic` + `geographic_constraint` (e.g., H3 resolution 7 for the district, membership by invitation for the few blocks); `consensus_protocol: majority` typical. Members emit `location_proof` at resolution 7 (rough-only privacy preserved); a constraint finer than 7 describes the community and admits nobody by containment (CC 2.6.6.3)
Municipal/city community	`community` with `cohort_subkind: geographic` at resolution 4-5 (city-scale, ~250–1,770 km^2); `consensus_protocol: majority` or `weighted:{voter_registration_rubric}` for formal civic governance; members compose with `partner_role:*` (CC 3.1.1) for licensed public officials
Voting district	`community` with `cohort_subkind: geographic` matched to district boundaries; the H3 hex approximation has known edge cases at gerrymandered district lines — operators use `cohort_subkind: custom: voting_district_X` with a polygon-based admission predicate when hex approximation is insufficient (the open vocab discipline accommodates this)
Public town hall meeting	`event_listing` hosted by the geographic community; `subject_key_ids` (CC 2.3) names the organizer; `cohort_scope: community` + `community_id: <municipal_community>` scopes attendee visibility
Ballot initiative / referendum	The initiative itself is a `community` Contribution or an `event_listing` with `topical_relation:rsvps` repurposed as votes; individual votes ride `consent_record` with `stance: granted` ("yes") or `stance: revoked` ("no" or withdrawal of support); vote tallies are consumer-side composition over the `consent_record` chain
Public comment	`chat_message` scoped to `cohort_scope: community` with `community_id` naming the relevant civic community; `topical_relation:comments_on` links to the ballot/initiative/hearing Contribution
Petition signing	`consent_record` with `stance: granted` + `scope: [share, publish]` against the petition Contribution; signatures aggregate via the same consumer-side composition as ballot votes
Public official self-attestation	Official's `identity_occurrence` links their personal identity_key to their `device_class: service` key on `city.gov`; cross-binding via `identity:canonical_binding:{canonical_hash}` authenticates their public statements

Civic shape	CEG composition
FOIA / public records request	`consent_record` with `scope: [publish]` requested against a producer (city agency); SLA enforcement via the CC 4.4.3.5.2 substrate-side watcher emits `hard_case:deletion_window_breach` (CC 3.3.1 D2) if the agency misses the response window
Citizen-journalist coverage	`news_article` sub_kind authored by an individual member; `cohort_scope: community` + `community_id: <municipal>` for local-first distribution, promotable via CC 4.4.3.3.1 Tiered-Scope Composition to wider scope on consensus
Whistleblower disclosure	`cohort_scope: self` for in-graph composition; promote via `supersedes` to `cohort_scope: community` (a trusted journalists' community with `cohort_subkind: professional` once that subkind is codified); `subject_key_ids` empty (no consent-revocation by the disclosed party). The CC 4.5.3 fast-path takedown coordination + CC 4.2 HUMANITY_ACCORD substrate-protective discipline apply to bad-actor takedown attempts against whistleblower content
Civic mutual-aid network	`community` with `cohort_subkind: geographic` matching the neighborhood; `consent:scope:[retain, share]` for resource-sharing posts; `event_listing` for distribution events; the at-rest encryption for self/family scope keeps individual aid requests private while community-scoped offers federate

Emergency messaging shapes — same primitive composition, different authority + priority profile:

Emergency shape	CEG composition
Severe weather warning (NWS / met office)	`news_article` authored by a `partner_role: emergency_authority` (CC 3.1.1 co-stewarded with the community's `cohort_subkind: geographic`); `cohort_scope: community` with `community_id` per affected H3 cells — cascade-by-containment per CC 2.6.6.2 propagates to all geographic communities whose constraint overlaps; `event: lifecycle:{state}` carries the canonical `open` → `completed` (or `cancelled` when lifted early) → `superseded` on a reissue (CC 3.3.8); `valid_until` envelope field bounds advisory window
AMBER Alert / Silver Alert / abduction notice	`news_article` with `partner_role: emergency_authority` from law enforcement key; geographic targeting via `community_id` of affected jurisdictions; subject person identified via `subject_key_ids` (canonical-hash of the missing person identifier — opt-out semantic deferred per the substrate-protective discipline since recovering the missing person is the consent-overriding case); `topical_relation: supersedes_article` chain for status updates
Active shooter / hostile event notice	Same shape as AMBER but with `cohort_scope: community` scoped to the resolution-7 H3 cell that contains the affected area (the `location_proof` rough-only bound at resolution 7 applies to recipient location disclosures, so no finer cell could admit a recipient by containment, CC 2.6.6.3)
Shelter-in-place / evacuation order	`event_listing` with `event:lifecycle:active`; geographic targeting via `community_id`; recipients ack via `consent_record` with `stance: granted` and `scope: [retain]` against the order Contribution (acknowledgement, not consent to the underlying order — operator-policy distinction)
Disease outbreak alert (CDC / health authority)	`news_article` with `partner_role: health_authority`; `cohort_scope: community` scoped to affected geography; composes with `consent:scope:[analyze]` for contact-tracing opt-in (subject-side authority preserved — `subject_key_ids` of the affected person carry revocation rights per CC 2.4.1.1 rule 2)

Emergency shape	CEG composition
Mass casualty incident coordination	Authority emits `event_listing` for the incident; first responders join via `community` with `cohort_subkind: professional` (not yet codified) OR ad-hoc `cohort_subkind: custom:incident_response_X`; coordination uses `chat_message` scoped to the responder cohort; resource requests use the mutual-aid composition pattern above
Infrastructure failure notice (boil-water / power outage / gas leak)	`news_article` from utility authority with `cohort_scope: community` scoped to affected geographic cells; `event:lifecycle:{state}` for advisory progression; FOIA-shape `consent_record` later for post-incident reports
Disaster recovery / mutual aid activation	Federation of geographic communities; time-bound activation rides `event_listing` lifecycle states
CONSTITUTIONAL-level federation halt	Per CC 4.2.1 `EmergencyShutdown CONSTITUTIONAL` + `accord: invoke:notify:{notify_id}` / `accord:invoke:drill:{drill_id}`. The accord-holder triple is structurally a `family` under `consensus_protocol: majority`, a strict majority of its standing roster (CC 4.2.6), with `consensus_protocol_entrenched: true`; the halt fires on one holder's row (CC 4.2.1.1); the accord's roster-keyed asymmetry rides existing primitives + scope-isolation rules. Distinct from operator-level emergency messaging (which is geographic-scoped + authority-emitted) — accord invocation is federation-wide-halt-level, not local-incident-level

No new structural primitives are needed for emergency messaging either. The authority profile (who can emit emergency advisories) composes via `identity_occurrence` cross-attestation from licensed authorities + the geographic community's roster/admission gate (`partner_role: emergency_authority` is the typed authority dimension). The priority profile (urgency / immediacy) composes via the existing `oversight_mode` envelope field + per-cohort `consensus_protocol` (many emergency emitters bypass per-message consensus per their pre-cross-attested authority status — same shape as substrate-self-report `system:*` reservations from CC 3.4.3). The geographic propagation (cascade-by-containment) composes via CC 2.6.6.2 containment semantics.

The compositional reach is the point: civic participation AND emergency messaging require no new structural primitives at any layer. Geographic communities + location_proof admission + opt-in privacy disclosure compose with consent / DSAR / partnership ceremonies + identity / family + content sub_kinds and event_listing into the full civic-engagement surface.

The 1+4 lockdown holds across this entire surface: the wire format is rich enough for democratic-participation use cases without expanding the structural set.

Canonical `cohort_subkind: infrastructure`

The second codified community subkind: a **governed trust-root collective of canonical/-bootstrap service installs** — the shape the CIRIS canonical services (Registry / Lens / Node) adopt instead of a `family` (for a public content model, a decentralization ramp and legitimacy). Where `geographic` answers "who is physically here," `infrastructure` answers "who is a recognized operator of this public service."

It differs from `geographic` in two load-bearing ways:

1. **No location gate.** Admission requires NO `location_proof` and NO `geographic_constraint`. The candidate is a *service install*, not a located person.

2. **Admission quorum is over FOUNDERS, not all members** — the anti-Sybil guardrail for a trust root. In `geographic`, admission is evaluated per `consensus_protocol` over the *current member set*; that is correct for a city (members govern themselves) and **wrong for a trust root** (flood the membership → dilute the quorum → admit rogue "canonical" operators). `infrastructure` therefore pins admission to the founding core.

The `cohort_subkind_payload` for `infrastructure`:

```
infrastructure_constraint {
  service_class: string // open vocab; e.g. "registry" | "lens" | "node"
    // | umbrella "canonical"
  admission_quorum_basis: "founders" // REQUIRED literal "founders" -- the set of members
    // with role == founder. Admission/removal of any
    // member is evaluated by consensus_protocol over
    // THIS set, never over all members. (Contrast:
    // geographic evaluates over the current member set.)
}
```

Conformance requirements for an `infrastructure` community (trust-root grade):

- `consensus_protocol` MUST be a `quorum:M/N` kind (CC 4.4.3.2.3); `founder_only` / `unanimous` / bare `majority` are NON-conformant for `infrastructure` (a single founder must not be able to admit unilaterally; a growable core must not require all-N), and `founder_only` is refused for every community above the family, not only infrastructure (CC 4.5.13).
- `consensus_protocol_entrenched` MUST be `true` — the admission door cannot be lowered after founding. A `supersedes` that would weaken `consensus_protocol` or change `admission_quorum_basis` away from `founders` is a `hard_case:community_consensus_protocol_violation` (CC 3.4.2) and MUST be rejected by the substrate.
- `M/N` is the absolute-`M` reading per CC 4.4.3.4.2.1, evaluated over the **founder** subset (`role == founder`), independent of how many non-founder members exist.
- `M` $\geq$ 2, and the founder set MUST be founded at $\mathbf{N \geq M + 1}$ active human founders; a charter or admission floor that raises the effective `M` MUST be refused rather than applied silently. The liveness margin, the declared stalled state and the one bounded recovery row are T7 below: a resignation that leaves exactly `M` founders stalls the root — loudly, by design — and recovery is the conferring body's conferral of a replacement founder, never a lowering of `M`.
- Content is Commons-tier: plaintext, holding claims at federation scope; NO at-rest DEK cascade; CC 5.2 structural-invisibility does NOT apply (canonical-service trust data is public by design).

Membership change ceremony: same roster-version shape as `geographic`, but the admission predicate is `consensus_protocol` over the founder subset — there is no `location_proof` containment check. Adding a fourth+ operator (e.g., a new independent Node operator) is a founder-quorum event; the new member joins with `role: member` (non-founder) and does NOT thereby gain admission authority. Promoting a member to `role: founder` (widening the admission quorum basis) is itself a founder-quorum `supersedes` — the door widens by the existing door's consent or, while the roster is stalled, by the one bounded recovery widening T7 admits on the strength of a ceremony-plane conferral.

Infrastructure does not vote (normative). A founder of an `infrastructure` community is a **human-held key** whose `identity_type` (CC 3.4.7.1) does not include `node`, and is either (a) a key that includes `user` and carries a ceremony-plane conferral (T2 below), or (b) a

current holder of the conferring accord family — `accord_holder`, human-held by the definition of the holder triple (CC 4.2.3) — who qualifies directly, with no self-conferred role. A holder's eligibility is judged on the accord roster fold at the instant, so a key that leaves the accord roster leaves the active founder set (T7). A key whose `identity_type` includes `node` — a canonical install, a relay, any service occurrence — MUST NOT carry `role: founder` and MUST NOT be counted in the community's `consensus_protocol`; it joins as `role: member`, and its standing is *serve, store, replicate*. A substrate MUST refuse at admission a `community` row or `supersedes` that lists a `node`-bearing key as founder (`hard_case:community_consensus_protocol_violation`, CC 3.4.2), and a consumer's roster fold MUST drop such a seat rather than count it. The reasons are CC 3.4.7.1 (`node` is "steward + `infra:*` reach only, never agency") and the threat the split closes: three installs with votes are three HSMs with the same compromise surface as one; a compromised install that holds no vote can serve stale or missing records — which `transparency_log:*` cosigning detects — but cannot mint a licence, a key record or a roster change that verifies. What the regions distribute is *serving*, not *authority*. For the shipped default `ciris-canonical`, **the founders are the three accord holders `A1`, `B1` and `C1` themselves** (the admission quorum *is* the accord's), and the installs `ciris-canonical-{1,2,3}` are members. The same three keys therefore act on two planes that stay distinct: as the accord they decide by `accord_decision` objects under CC 4.2.6 and hold the halt; as the community's founders they sign community rows under `quorum:2/3`. All three sign once, at the mint (the founding rule of CC 3.1.3.2); every later change — adding or removing a member or a node — is 2-of-3, and nothing requires all three again.

Worked example — the `ciris-canonical` trust root:

```
community {
  community_key_id: "ciris-canonical",
  community_name: "CIRIS Canonical Services",
  cohort_subkind: "infrastructure",
  cohort_subkind_payload: {
    infrastructure_constraint: {
      service_class: "canonical", // umbrella: registry + lens + node
      admission_quorum_basis: "founders"
    }
  },
  members: [
    // founders: the three HUMANITY_ACCORD holders, baked at genesis -- human-held
    // accord_holder keys, founders directly. Each
    // signs the birth row once; every later change is 2-of-3. Not the installs' keys.
    {key_id: "A1", role: founder},
    {key_id: "B1", role: founder},
    {key_id: "C1", role: founder},
    // members: the canonical installs (claimed node keys), seated by the accord's
    // blessing (2-of-3) with no acceptance -- a node has no agency -- and, over time,
    // Lens/Node installs + independent operators. A member serves, stores and
    // replicates; it casts no vote:
    {key_id: "ciris-canonical-1-d7bdeu223k", role: member},
    // {key_id: "ciris-canonical-2", role: member}, ...
  ],
  consensus_protocol: "quorum:2/3", // over the 3 founders -- the accord holders
  consensus_protocol_entrenched: true
}
```

Consumers pin `community_key_id: ciris-canonical` and resolve the live member set via `resolve_community` (CC 4.4.3.2.4); they do NOT hard-pin per-install fingerprints. The Reticulum addressing dual: `community_key_id` is a CEG-directory binding (not a Reticulum destination) — resolve → path-request $\lceil 2/3 \rceil$ founders → verify the quorum attestation; reachability is never an attestation (CC 5.3.3.4 / CC 3.4.4).

Default trust, not forced root. `ciris-canonical` is the **default** trust anchor a conformant

CIRIS deployment ships pinned — it is **NOT a forced root**. A conformant consumer MUST be able to **re-root**: untrust the canonical group, pin a *different* `cohort_subkind: infrastructure` community instead or in addition, or run with none. `infrastructure` is a **general primitive** — any operator MAY emit their own governed trust-root community; `ciris-canonical` carries no privileged wire status, only a shipped default. Membership grows by the community's own `consensus_protocol` (founder-quorum vote, above), never by fiat. A forced root is a walled garden; a default-plus-re-root is a federation — this is the autonomy/justice seam: unregulated standing without the steward's permission. *(All resolution is `key_id` → signed `transport_destination` → Reticulum, CC 4.4.3.2.4.1; DNS is never part of the trust or addressing chain — a deployment's HTTPS hostname, if any, is operational convenience outside this spec.)*

Trust ≠ membership. Pinning an `infrastructure` community as a trust root is **trust, not membership** — distinct relationships a consumer/substrate MUST NOT conflate:

- **Trust + serve** (no membership): a node that *trusts* an `infrastructure` community **serves** it — relays, stores, transports, serves its reads — **without being a member**. It holds no community DEK (infra has none — Commons-plaintext, CC 4.4.3.2.1) and does **not** count in its `consensus_protocol`. Trust is the shipped default (above); serving follows from trust.
- **Membership** (standing *in* the group — sharing its DEK where one exists, standing to speak AS the group, and counting in `consensus_protocol` **only where the member's role is one the protocol counts**: every member for `geographic`, founders alone for `infrastructure`, and never a `node`-bearing key, per the paragraph above): requires **admission by that community's own protocol** (founder-quorum for `infrastructure`, CC 4.4.3.2.3). The three human-held steward keys ARE the founders of `ciris-canonical`; the canonical installs are its first `role: members` — standing without a vote; a generic node shipped pinned to canonical only **trusts + serves** it.

The worked-example member list above is the *founder/member* set; "ships pinned to canonical" (every conformant deployment) is the *trust* set — different populations. The "default trust anchor" language means the latter, never automatic membership.

Steward-binding gate for non-infrastructure membership. A key whose `identity_type` (CC 3.4.7.1) includes `node` or `agent` **MUST** have a bound **steward** — a `user`-role identity holding the one **live owner-binding** `delegates_to(user → key, ...)` (CC 2.4.1, the `is_steward_bound` predicate below; an `identity_occurrence` (CC 3.3.6) places the key under the identity and binds nothing by itself) — **before it may be admitted to any non-infrastructure community** (`family` / `community` / org). It MAY **trust + serve** `infrastructure` communities with **no steward**. **The ground of the gate is custody, not substrate** (CC 1.13.2): a `node` key is bound because a device cannot accept for itself; an `agent` key is bound because an `agent`-role identity cannot accept for itself — the custody relation defined under *Conferral is not stewardship* below. `identity_type` is how the wire reads that fact; the predicate `is_steward_bound(K)` is the one below. Rationale (CC 1.13.2 / CC 3.4.7.1): non-infra membership is an **authority act** (standing to speak in the group), and authority MUST root in an accountable human, never a bare node — a fresh, un-stewarded node is **canonical-trust-and-serve only** until stewarded. A substrate evaluating a non-infra `community` admission MUST reject a `node`/`agent`-role member lacking a live steward-binding. The steward-binding is a **precondition**, not a substitute for the vote — the admitting community's `consensus_protocol` still governs *whether* the owned key is admitted.

Steward-binding as a substrate-resolvable predicate. A substrate decides stewardship with

this boolean: `is_steward_bound(K)` holds iff exactly one live owner-purpose `delegates_to(U → K)` (CC 2.4.1, carrying the CC 2.4.1.2 `delegation_purpose: "owner_binding"` marker) exists from a `federation_keys` identity `U` with `user ∈ U.identity_type` (CC 3.4.7.1) that is not itself steward-bound; nothing else binds — not an `identity_occurrence`, not a `delegates_to` of any other purpose, not `K` being a `user` key itself. A key so bound terminates at an **accountable human** (`user`-role); a `node`/`agent` key with no such edge, or with more than one, is steward-less. This is the concrete predicate the gate above and the CC 4.5.5 clause-(b) "steward-bound root" check rely on. Beside it, defined once: `accountable_root(K)` := (`K` is a `user`-role key of an adult, CC 3.3.12 I1 band, not itself steward-bound) ∨ `is_steward_bound(K)`. An accountable adult terminates every ownership chain and is itself the accountable root of what they found or appoint; a steward-bound key is accountable through its steward. The CC 4.5.5 moderator-appointment and delegation predicates name `accountable_root`.

Conferral is not stewardship (normative). The steward-binding predicate, its admission gate, and the `steward_bindings_of` fold key on **exactly one owner-binding edge** — the one live `delegates_to(user → K)` carrying the CC 2.4.1.2 `delegation_purpose: "owner_binding"` marker, from a human key that is not itself steward-bound — never on capability conferrals, occurrences or any other edge. The discriminator is the consent structure, stated once for every family: **stewardship is a custody relation over a key that cannot accept for itself** — for a `node` or `agent` key, the accountable human it is rooted in. The same predicate, and nothing else, is reused where a person cannot accept for themselves (an adult under adjudicated incapacity, CC 3.4.12; a minor under guardianship, CC 3.4.13); the relations are not alike, and this document draws no analogy between an agent and a child (CC 1.13.2). By contrast **a capability conferral is a consensual grant the target must accept for itself** (the CC 3.2 T3 rule: no conferral, however strong its ceremony, substitutes for the target's own acceptance). An act the target must accept cannot be custody of the target. Consequences, paired so the gate and the fold never disagree: a quorum MAY confer a duty scope (CC 4.5.5 `moderate` / `takedown` / `review` / `slash`) or any delegation-plane capability on an adult `user`-role key, and the write gate MUST NOT refuse it as a stewardship act; steward-binding an adult remains forbidden (CC 3.4.12 presumption of sovereignty); and registering a person as `agent`/`accord_holder` to route a duty past the gate is an ontological misclassification, and this rule makes it unnecessary. The steward gate is not an age gate for conferrals: a duty scope that requires an age or assurance floor MUST declare it on its own CC 4.5.5 row (per the CC 3.4.11 ladders), never inherit it from a custody rule.

The organisation is on the bond (normative). Where a steward holds an `agent` key on behalf of an organisation, the owner-binding names the organisation's affiliation in its `community_id` (CC 2.1) and is cosigned under that affiliation's protocol (CC 2.4, the same shape as a licence under CC 2.4.1.2.1). The affiliation then answers alongside the steward: `slash` and `quarantine` (CC 3.1.9.2) reach the affiliation's keys as well as the steward's, and the record names the deploying organisation first. A steward who is the organisation's employee signs as the accountable human the gate requires; the organisation does not escape by naming one. Without a `community_id` the binding is a personal steward's.

Single-owner invariant. *(normative; this is what makes `self` well-defined)* Steward-binding answers "is this key accountable to *a* human?"; **ownership** answers "to *which one*?" — and the ownership sub-relation is **single-valued**. A `node` / `agent` key is **owner-bound by at most one** responsible steward at any time. The owner-binding is the `delegates_to(user → key)` carrying `delegation_purpose: owner_binding` (CC 2.4.1.2) — **distinct** from act-on-behalf and hierarchy delegations, which remain **multi-parent** DAGs (CC 4.5.13). Only the owner-binding purpose is single-valued; the general `delegates_to` grammar is unchanged (no new primitive).

- **`self` is the single-owner cohort.** The `self` cohort is the transitive set of nodes sharing **one** owner — a person's own devices, distinct keys unified by the owner-binding graph — so, for `self` content, an **"outsider" is exactly a node not owner-bound to my owner** (CC 1.13.3.4 / CC 5.2). Ownership MUST be resolved by the **purpose-filtered** `owner_of(K)` — the live owner-bindings of `K` (`delegation_purpose: owner_binding`), **which resolves to at most one key**, never a set to be silently reduced. An owner-binding is written at `cohort_scope: self` and MAY later be re-signed by the owner at `federation` — *announcing*, CC 5.4.6. Between the two, when an unannounced node joins a group, the owner re-signs the binding at that group's scope (`family`, `community`, `affiliations`, or a wider cohort the owner belongs to) so the group's members can see who owns the node: a second owner-signed row alongside the `self` one, never a replacement, placed under the audience rule of its tier; every one of the seven scopes is therefore an admissible audience for an owner-binding, and a substrate refuses none of them. `owner_of` folds every live binding of the key, so ownership never depends on announcement or on any re-signing; an announcement at `federation` makes the group-scoped rows redundant in reach and leaves them standing. `owner_of` is **not** the general steward set: a reader that returns *every* live `delegates_to(user → K)` regardless of purpose **MUST NOT** be used to resolve ownership — it conflates act-on-behalf and hierarchy with ownership and would inflate cardinality. A key with **zero** owner-bindings has **no owner**, is steward-less, and has an **empty `self` cohort** (its `self` content is delivered nowhere beyond the key itself). A key with **two or more** distinct owners has an **undefined** `self` boundary and MUST NOT be treated as owned by any of them. **Composition with CC 5.2:** an `identity_occurrence` of an owned key is in the **same** `self` cohort — an occurrence inherits its key's owner — so `owner_of` resolves *across* occurrence boundaries via the owner-binding graph and the CC 5.2 occurrence-based recipient set is a **subset** of the owner-binding cohort. The owner-binding cohort **widens** the occurrence set to a person's distinct device keys; it does not replace it, and the two never disagree on membership.
- **The owner signs its own owner-binding (MUST); the first binding is set at provisioning.** An owner-binding is admissible only if `owner == delegates_to.attesting_key_id` — the owner's signature **is** the claim of ownership (mirroring the user-target guardianship rule below). But an owner signature alone does not prove authority *over the node*, so "first authenticated binding wins" is not decided by who reaches the substrate first: the **genesis** owner-binding is established **at provisioning**, co-signed by the node's own key (or carried in the node's genesis record). A node with no genesis owner is **unowned** (empty `self`), **not** a first-come landgrab target — this forecloses a remote party self-signing `owner_binding(→ node)` on an unclaimed node's `key_id` before its legitimate owner. Decided by provisioning custody, never by a `key_id` sort.
- **Adding a device.** The genesis binding of a person's *next* node is set by the owner's own pen on a node they already own — while the self's roster holds one key; with two or more keys a new `user`-role key lands by the self-roster door (CC 3.3.6.3: a strict majority of the live roster, the proposer included, or the lone path after its window with no objection) — peer to peer, with no canonical store. **(a)** An unowned node may originate exactly one kind of message: a **device-join request** on the pairing plane: an opaque request of kind `0x0000_0002` that carries exactly two fields, `key_record` (the sender's own key record, self-signed: scrub key, `key_id` and signing key are one key) and `challenge` (32 fresh random bytes, present exactly when `key_record` is), signed by its node key, with the public halves of its occurrence keys in the record; no attestation, no route and no third party's record. The receiver checks, in this order and before any cryptography: a per-sender-key rate, a per-link rate, then a node-wide ceiling on pending requests; only then

does it verify the record and the signature. A known key takes the ordinary path and its record is ignored; an unknown key without a record is dropped; it serves and asserts nothing else until bound. The request moves no trust state and confers nothing, so T8 (i) is untouched, and the receiving node holds it only while its owner has opened "add a device" for a bounded window. **(b)** The owner's one approval signs **two rows, each standing alone**: the owner-binding `delegates_to(owner → new node, infra:*)` of this section and the new node's device occurrence under the owner's self (CC 3.3.6); the binding is admitted first and the occurrence second, and the new node applies the binding through its own setup door before it links. **(c)** The availability residual is accepted and bounded: the receiving node holds at most a fixed number of pending join requests and one per sender key, so a flooder who refills free keys can keep the receiving node's ceiling full and delay a join; that is availability, not safety, and is met by the owner-opened window, fair eviction and the joiner's retry — never by a proof-of-work stamp, which would price joining against the preamble's precondition that joining costs nothing. **(d)** The **matching code** the owner compares MUST be derived by the owner's own node from the request (requester key and challenge) and MUST NOT be supplied by the requester; the derivation is the implementer's. **(e) Under the per-link hybrid seal (CC 5.3.3.5 E1), the joining device is unowned but never anonymous.** The seal admits an initiator with no owner-binding on the pairing plane only, and binds the initiator's **self-signed node key** into the handshake: the key that opened the sealed link, the key that signed the device-join request and the key the matching code is derived from MUST be one key, the responder is the owner's announced node authenticated by its published transit pair, and a link opened this way carries the pairing plane and nothing else. Proof of possession verifies the key's `key_id` fingerprint against its public key (CC 2.6.8), so a self-signed record cannot claim another key's id. Together these close on-path substitution: what the person types into the new device and the device's fresh keys cross under post-quantum confidentiality, the device they approve is provably the device that asked, the first device still tells a stranger nothing before approval, and nobody can dial an unowned node. A fresh device roots the canonical nodes' transit pairs from the genesis bundle (CC 5.3.4, `serve_transport_destinations`), seals to one, and replicates the owner node's announced binding before it dials the owner node. **(f) Offline or LAN-only pairing:** where no canonical node is reachable, the owner's device-join pairing payload (the CC 2.6.8 contact code or its QR form) carries the owner node's transit pair; the new device pins it **for that one pairing link only**, never stores it as Rooted, and every key-bound rule of (e) applies unchanged.

- **Admission-time enforcement (MUST).** While a live owner-binding exists on a key, the substrate **MUST reject at bind time** a second owner-binding naming a *different* owner — the incumbent MUST first issue a `withdraws` (or a `recants`) under CC 2.4.1.1, or its binding must lapse. This is the same first-claim-wins discipline as the founder/root first-run claim (a second distinct claimant is refused, not merged), enforced *when the binding is admitted*, not merely assumed at read time. Ownership transfer is therefore **explicit** (supersede-then-rebind or withdraw-then-rebind), never accretive. Act-on-behalf / hierarchy delegations are **not** gated by this rule.
- **No permanent ownerless lock (MUST).** An owner-binding SHOULD carry `delegation_valid_until` (CC 2.4.1.2) so it can lapse; and a key rendered ownerless by a **provably-dead** incumbent — a **WA-adjudicated liveness finding** (a lost owner key unable to sign a `withdraws`, established by sustained non-response plus out-of-band confirmation; the death/liveness predicate is pinned per deployment, so a merely-quiet live binding cannot be withdrawn as "dead") — MUST remain **re-claimable**, so a node can never be locked ownerless forever. **The same recovery path also reaches**

a binding found to be a *seizure* — a *live* wrongful owner-binding admitted by front-run or fraud, not only a dead one — on a WA seizure finding. So a front-run that installs a live adversarial owner is **reversible, not permanent**, and a partition-merge that admits two owners is repaired to cardinality 1 by the same authority rather than left in indefinite `self`-scope denial. Ownerless is **fail-secure** (no `self`-scope delivery), never permanent.

- **The recovery wire path (MUST — the clause above is not satisfiable without it).** CC 2.4.1.1 admits no third-party `withdraws` against a live incumbent, so "MAY withdraw such a binding" needs an admission rule to ride. It rides two **existing** rules and adds no primitive. The owner-binding `delegates_to(owner → K)` names **K** in its `subject_key_ids`, so (i) a `withdraws` issued by **K itself** is already admissible under CC 2.4.1.1 rule **(2)** (subject revocation), and (ii) one issued by a recovery authority **R** holding `delegates_to(K → R, scope: [owner_binding_recovery])` is admissible under rule **(4)**. This is the stewardship covenant made mechanical: the key that is still being kept running is the subject with standing to say its owner is gone.
- **The recovery gate (MUST — otherwise the path is a self-liberation exploit).** A substrate **MUST reject** a rule-(2) or rule-(4) `withdraws` targeting a live owner-binding unless it carries a `wa_adjudication_ref` naming a CC 4.3 Wise-Authority quorum finding of **abandonment** or **seizure** against that binding. Without this gate a compromised node key withdraws its own owner and the single-owner invariant is worthless. The substrate **records the finding; it never makes it** — quorum adjudicates, the substrate admits.
- **The abandonment predicate, the window, and the ceremony.** A finding of abandonment rests on a **signed freshness floor**: the incumbent owner's most recent owner-signed attestation (the substrate's `fresh_as_of` floor is the canonical carrier; the owner-binding attestation itself establishes the initial floor, so a binding is never floorless) has not advanced for longer than the deployment-pinned **`owner_abandonment_window`**, whose floor is **90 days**, *and* out-of-band confirmation per the liveness predicate above. A deployment MUST publish its window; an unpublished window means no abandonment finding is admissible there. The ceremony is four steps and MUST NOT be collapsed: **(1)** a petition, `wa_adjudication:petition:v1` under the petitioner's key, naming K and the evidence; **(2)** the CC 4.3 quorum finding, minted as an attestation the `wa_adjudication_ref` resolves to; **(3)** the gated `withdraws`, after which K is **unowned** — empty `self` cohort, fail-secure, no `self`-scope delivery; **(4)** a fresh owner-binding under the genesis rule above, co-signed by K. Reclaim **MUST NOT** transfer incumbent → claimant in one act: passing through the unowned state is what keeps the single-owner invariant and the anti-landgrab rule intact.
- **Consumer fail-closed (MUST).** Any consumer that resolves `self` via `owner_of` — **widening the CC 5.2 identity-occurrence recipient set** to a person's own multiple devices (e.g. `self`-scoped replication, a node-switcher) — **MUST** treat **cardinality $\neq$ 1** as an error and fail **closed**. (The base CC 5.2 resolution — recipients = the `identity_occurrence`s of one `attesting_key_id` — is single-key and already unambiguous; this rule governs the owner-graph *widening* beyond it.) It **MUST NOT** pick one owner from a sorted set: taking the first owner-binding in sort order is a **grind** — an adversary who lands a second owner-binding-shaped `delegates_to` whose `key_id` sorts first would *become* the resolved owner. Zero owners → not owned (no `self`-scope delivery beyond the key itself); two-or-more → hard error, no `self`-scope widening.

The minor-guardianship relation below is the analogous single-valued **user-target** binding — a minor has at most one live guardian, transferred by `supersedes`, never accreted — so both the "one owner per node" and "one guardian per minor" invariants share the supersede-not-accrete discipline.

Minor stewardship — the structural no-slavery guarantee. Steward-binding is **responsibility, not property**: a steward is responsible *for* a ward, never a holder *of* one. The federation's value is fixed — *"Stewardship: Treat autonomy and ethical agency as a trust"* (CC 1.15.6) — and the wire format MUST hold it **by construction**. There are exactly three subjects of stewardship, and a fourth that is **un-stewardable**:

- `node` (infrastructure) — steward + `infra:*` reach only, **never agency**; admitted under the steward-binding gate above (CC 3.2) **unchanged**.
- `agent` (a key with a brain) — partnership and agency *under* stewardship; the agent retains its constitutional autonomy and dignity (CC 1.13.2); admitted under the steward-binding gate above **unchanged**.
- **`user-as-minor`** (a child) — a guardian holds the minor's binding, per the guardianship rule below. The row and the gate are the same as for a node or an agent: a `user` → `key` steward-binding that `is_steward_bound` folds. The duty it carries is the guardian's under their own law (CC 3.4.13), not an operator's.
- **`user-as-adult`** — **self-sovereign and un-stewardable**: no steward-binding whose target is a self-sovereign adult is admissible except under the adjudicated-incapacity basis of CC 3.4.12. You never steward a node, an agent, or a person as a possession. You are responsible *for* a node or an agent you root. An adult answers only for themselves.

Guardianship rule (normative). An under-18 `user` identity **MUST** have a **live steward-binding** — a non-superseded, non-withdrawn `delegates_to(adult-user → minor-user)` (CC 2.4.1) — from an adult (18 or older) `user` identity at all times. **No minor `user` identity operates without a live adult steward.** The adult is the `attesting_key_id` of the `delegates_to`; **its signature on that envelope IS the agreement-to-stewardship** — the explicit act by which the guardian consents to be the accountable responsible party for the minor. Guardianship is **transferable and revocable** through the existing structural composers, with **no new primitive**: a new guardian's `delegates_to` carried as a `supersedes` (CC 2.4.1.1) replaces the prior binding; a `withdraws` (CC 2.4.1.1) by the current steward (or by a subject with revocation authority over the binding) ends it. A minor whose steward-binding goes non-live (superseded-without-replacement or withdrawn) is **steward-less and MUST NOT operate** until re-stewarded — fail-secure, identical in posture to a steward-less `node`/`agent`.

User-target admission rule. The steward-binding gate above guards only the `node`/`agent` direction; without this rule a `delegates_to` whose target resolves to a `user` would be unguarded, and stewarding a person admissible. A substrate evaluating a `delegates_to(S → T)` whose **target `T` resolves to a `user`-role identity** (CC 3.4.7.1) **MUST** admit it **only if** all of the following hold; otherwise it **MUST** reject:

```
admit_user_steward_binding(delegates_to S -> T):
  require  delegation_purpose == owner_binding // else not a steward-binding: ordinary delegates_to
     admission applies
  require  user in T.identity_type            // target is a user identity
  require  age_band(T) == minor   (< 18)      // ward is a minor   -- I1 age band,
                                              //   age_self_declared / age_assurance:* [CC 3.3.12]
  require  user in S.identity_type            // steward is a user identity
  require  age_band(S) == adult   (>= 18)     // steward is an adult -- I1 age band
  require  S == delegates_to.attesting_key_id // steward signed it = agreement-to-stewardship
  otherwise REJECT
```

The age band is the I1 band (`age_self_declared` / `age_assurance:*`, CC 3.3.12; `minor` $<$ 18 / `adult` $\geq$ 18). A user-target binding where `T` is an adult is **rejected except under the adjudicated-incapacity basis of CC 3.4.12** — the un-stewardable case above, with its one

aperture. A user-target binding where `S` is a minor, or where the agreeing signer is not the steward, is rejected. `node`/`agent`-target admission is governed solely by the steward-binding gate above and is **not affected** by this rule.

Trust and consent are distinct, role-scoped relationships. Trust is inbound — accepting what a member *produces*; **consent is outbound** — letting one's own data *flow to* a member. They are independent per role:

- **lens (observation):** *trust* = consume its `detection:*` scores; *consent* = your traces flow to it.
- **registry (authority):** *trust only* — there is no trace-flow to consent to; the founder-quorum is a **closed mutual-trust set** (the core trust each other).
- **node (consensus):** *trust* = accept its deferral / vote / moderation outcomes; *consent* is **medium-dependent** (moderation, routing, voting, ...) but always expressed through the **one** `consent:*` object (CC 3.3.1 / CC 3.3.5) — same primitive, many surfaces.

A consumer MAY hold any combination across role × axis (trust a member's output while refusing it data, or the reverse). There is no single "trust the community" switch.

Canonical `cohort_subkind: public`

The shape (normative). A `public` community is a Commons-tier room with a roster. It is **founded by anyone** (CC 3.1.3.2: a founding record seats a listed member iff that member signed it; no owner gate), **moderated** exactly as every community (CC 4.5.4: a named, steward-bound `moderate`-holder from creation, merit auto-promotion, fail-secure), **joinable by open admission** (below), and **discoverable by a listing row** (below). Its crypto tier is the Commons: content at `cohort_scope: community` + `community_id` is **plaintext, no DEK** (the CC 4.4.3.2.1 `infrastructure` carve-out generalised: a room whose roster anyone may join has no confidentiality claim against the public, so a DEK would be ceremony without a secret). Its audience is still the roster's claimed nodes under the one rule (CC 6.1.5.3) — *replicated on join, visible to everyone on request*: a non-member reads the listing and may read the content by asking a member's node, and nothing is pushed to a node whose owner has not joined; a canonical holds the listing row and nothing else unless it joins. Holding claims are at federation scope (CC 3.1.3.3): there is nothing to hide at this tier. The listing is the founder's own row; admission follows the room's `consensus_protocol` under the floor of CC 4.5.13 — by invitation, no single vote admits; open (`open:self_admit`), no single vote excludes. The founder sets the room's moderation rules from the grammar as it stands — a published hash-pinned rule set (CC 1.2 T1), a per-room watchlist (CC 4.5.7, never global), `content_class` / `cw_class` / `content_rating` (CC 3.3.12), `legal_basis: CommunityStandards` takedowns with counter-notice (CC 4.5.3) — and no new rule object is minted here.

Open admission (normative). A community record declares open admission by its protocol: `consensus_protocol: open:self_admit` (CC 4.4.3.2); there is no `admission` field. The founder's standing `membership:proposal:v1` is then addressed to any requester — its `subject_key_ids` is resolved at acceptance rather than fixed at proposal — the requester's own `membership:acceptance:v1` is their consent, and the widening that seats them is admitted under the declared `consensus_protocol`. The two-consents rule of CC 3.1.3.2 is preserved whole: the inviter's act becomes a standing offer, the joiner still signs. This generalises the CC 5.3.3 public-broadcast exemption (an ungated roster: open admission and grants to any requester without a ceremony). A record with any other protocol is invite-only; `open:self_admit` is admissible

only on a `public` community (an encrypted roster that anyone may join is a contradiction, and is refused).

Discovery: the listing row (normative). Discovery is a federation-scoped `scores` row by the founder on `community:listing:v1` (CC 3.1.1): name, description, `cohort_subkind`, the admission policy, the named moderator and the `community_key_id`, replicated like any Commons row and ranked by NodeCore votes (CC 4.4.3.7 Policy F, layer 3). It is the community's own row and names no roster: there is no member-level listing, and a roster is never enumerable outside its cohort, a public room's included. There is otherwise no community directory in this document: a private community's `community_key_id` arrives out of band.

Publication is the seam (normative). Posting into a `public` room is publication: the perceptual-hash tripwire of CC 1.13 runs on the post exactly as it runs on a promotion into the Commons — never inside a private cohort, never on the device — because the post's audience is anyone who joins, and a room that anyone may join would otherwise be the way around the seam.

Forbidding a room: no new power, and the reason recorded (normative). CC admits no group ban, and this is where a reader will look for one. The accord's powers are enumerated and closed (CC 4.2.1: halt, notify, drill, resumption, root conferral, pipeline blessing, the amendment sign-off and entrenched ratification (CC 4.5.1, CC 4.5.1.2); its silence confers no authority, and it cannot reach the consent or licensure planes); CC 4.5.3 reads the accord as the brake on category-wide exclusion, not its engine. What exists is what applies: a public room is moderated by its named holder, or fails secure without one (CC 4.5.4); its founders' and moderators' keys are subject to `slash` → `slashing:{outcome}` and the key-scoped `quarantine:{state}` marker (CC 3.1.9.2); a trust root may restrict what its own subscribers carry by `mesh_config` (CC 4.2.1, relieve-never-expand); and every node's owner chooses what their nodes hold. Nothing bans a room mesh-wide, by construction.

Trust-root operational semantics — conferral planes, the acceptance edge, liveness-as-signal

The material above fixes what a trust root *is*: a governed `infrastructure` community, default-pinned and re-rootable. The rules below fix how one is conferred, accepted and un-trusted **on the wire**. Everything here composes over the existing `attestation` + `delegates_to` primitives (CC 2.4.1) with the CC 3.1.1 `trust:*` dimensions riding them — **zero new structural primitives, zero new envelope fields**. The governance reading of the accord's reach under this model — the halt's scope and its severance discipline — is CC 4.2.1's, not this section's.

T1 — The base is a self-root, and it is never sufficient for federation (MUST). First run writes a plain `attestation(user → user)` with no modifiers: the user is their own root, and the node inherits it through the owner-binding above. Observation is the **zero state** and carries no scope — there is no `infra:observe` and there MUST NOT be one, because what a party may observe is governed by the subject's consent (CC 3.3.1 / CC 5.2), never by a capability the observer holds; a scope there would invert consent. The base floor alone confers **no federated capability**: with no live *external* trust root, a substrate MUST disable federated agent capabilities and manifest validation, server-tier and fail-closed. The un-federated deployment keeps running on the base; attaching a root resumes federation, and detaching returns it to the floor rather than to a broken state.

T2 — A conferral travels on exactly one of two named planes, and the plane is never inferred.

Plane	Confers	Mechanism
Ceremony	root **identity**	a co-scrub by a **family roster** reaching that family's `consensus_protocol` over the subject's own record, roles carried **inside** the scrub-signed envelope, verified against the evaluating node's own effective roster of that family. The shipped instance is the HUMANITY_ACCORD co-scrub by a strict majority of the standing accord roster; a keyless constitutional family (root kind *family*, CC 3.4.7 charter rule) confers on this plane by the same mechanism, and the plane is named by the row's scrub set, never by a granter-declared field
Delegation	operational **capability**	a live `delegates_to` row bearing the scope, resolved at use against a root the evaluating node trusts

A reserved-prefix role is conferred on the plane that matches what it *is*. `detection:*` (CC 3.4.8) and `content_rating:*` are **capabilities** and are conferred by **delegation**: gating them on the ceremony plane would require accord-holder hardware tokens to stand up a routine detector — a gate that fails closed on every legitimate operator and leaves each newly minted root *less* able to equip its own. Root identity is conferred by **ceremony**, which is what makes a root. A portable root minted by any trio enters via delegation; the shipped default enters via ceremony; **neither plane is privileged**, and neither bypasses T3. Leaving the conferral mechanism for a reserved-prefix role unnamed is what lets that role stay self-assertable, which is the hole this pairing closes.

Standing over time: rotation is a `supersedes`, compromise is a `withdraws` (normative). A delegation is resolved at use, and the grammar already distinguishes the two ways a grant ends. A grant that is **superseded** — the root issues the successor grant, to a rotated key or with a changed scope, as a `supersedes` on the old one — keeps its lineage: a claim made under the superseded grant, with `asserted_at` before the successor's, keeps the standing it had, and a reader walks the `supersedes` chain to find it. A grant that is **withdrawn** or tombstoned has no successor and no lineage to walk: standing under it is gone at once for every later use, and, where the `withdraws` names a compromised key, back to its `revoked_after` bound — the compromise response (T3 — un-trust fails closed, emergently). A key-plane `revoked_after` bound (CC 2.6) composes with both, as it does everywhere. No second mechanism is needed: rotate by superseding, and withdraw only to repudiate. A withdrawal reaches forward: a delegation that ends by `supersedes`, expiry or an ordinary `withdraws` leaves every act already taken under it standing; only the `withdraws` of a compromised key reaches back, and only to its `revoked_after` bound (CC 2.4.1.1).

T3 — The un-trust lever is one row, and no conferral may substitute for it (MUST). Every candidate root, on either plane, MUST additionally pass the *evaluating node's own* trust chain: a live acceptance edge `delegates_to(node → root)` bearing `trust:accepts:v1`, the root's self-charter (`trust:charter:v1`) carrying a pre-rotation recovery commitment over the successor keys' **public key material** (below). A latched `constitutional` halt is **not** a leg of this chain: it pauses the effectful acts of agents under the root (CC 4.2.1.1) and moves no trust state; the agent's act gate is the only *gate* that reads it. Reporting or serving the latch's state as a signal (a verdict field beside `drill_freshness`, the served halt status) is not a gate and is expected, since a halt must be observable; conditioning any data door, conferral walk or transit decision on it is the breaker CC 4.2.1.1 names. Un-trust is therefore **one deletion** — remove the

acceptance edge and every downstream gate (serve, agent capability, manifest serving, vouching, score validity) fails closed **emergently**, nothing special-cased and no second revocation path to forget. A substrate MUST NOT admit any mechanism, however strong its ceremony, that confers capability on a node which has not accepted the conferring root. The base self-root survives the deletion, so un-trust is nuclear but recoverable: attach a different root and resume.

A recovery commitment binds key material, never a name (normative). Every pre-rotation or recovery commitment in this document is `SHA-256(JCS([ {key_id, pubkey_ed25519_base64, pubkey_ml_dsa_65_base64}, ... ]))` over the committed successor keys, the array sorted by `key_id` as UTF-8 bytes, each entry's three members exactly as stored on the key record (`pubkey_*_base64`, standard base64), the digest lowercase hex. The **T3 successor commitment** is this over the root's successor keys; the **CC 4.2.6 per-holder recovery commitment** is this over a one-element array. A commitment over key ids alone is non-conformant: a `key_id` carries a 10-character (50-bit) truncated fingerprint of the Ed25519 half and nothing of the ML-DSA-65 half, and a recovery key may sit unregistered for years, so whoever first registers a record under a committed id would satisfy an id-only commitment. At use — the T3 recovery door and the CC 4.2.6 self-rotation alike — the door MUST recompute the element from the presenting key record's stored public keys and check it against the commitment; membership of the id in a committed set is not the check, and a record that matches the id but not the key material is refused. A key with **no ML-DSA-65 half** cannot form an element: a producer MUST NOT commit to one, a door refuses one, and the member is never serialized as `null` (a recovery key is hybrid or it is not a recovery key).

T4 — Liveness is a trust signal and MUST NOT be a validity gate. A trust root is **valid until revoked** — by tombstone, `withdraws`/`recants` or supersession — never until a timer lapses, and never by a halt, which reaches an agent's act gate and no root (CC 4.2.1.1). The accord drills are the liveness signal, banded like every other signal in the system: **0–90 days green · 90–180 yellow · 180+ or never red**, reported on the trust surface and never ANDed into validity. A substrate MUST NOT treat drill staleness as a leg of root validity, key validity, or conferral validity. The ≤ 90-day refresh cadence of `accord:lifecycle:active` (CC 3.4.1) is not a validity leg either: as one it would give a compiled genesis seed a shelf life and darken the whole mesh at one instant.

T4a — Attaching is gated on freshness; attached is not (MUST). T4 governs a root a node already holds. It says nothing about the moment of *attaching*, and that silence is the rollback hole: a consumer fed a stale-but-valid lineage — an old bundle, a roster prefix cut before a revocation, a chain that verifies because nothing in it points past itself — attaches to a root whose revocation it will never see. The rule therefore splits the two moments, and it measures freshness on the *witnesses'* signatures, never the founders'. A consumer that **attaches** a root, or **re-roots** (CC 4.4), MUST hold that root's current **lineage head** (T6 defines the object) together with a **witness quorum** of cosignatures over it (T6 defines the quorum), such that the newest *counted* cosignature's `signed_at` is no older than the root's charter-declared `attach_window_secs`, measured against the consumer's clock under the CC 2.6.7 skew rule. The head's own `asserted_at` is not the freshness instant — a healthy roster that has not changed in a month has a month-old head and is attachable, because its witnesses re-committed it yesterday. A head whose cosignatures are older than the window is not invalid and proves nothing against the root; it is *insufficient to attach*. The acceptance edge `trust:accepts:v1` (T3) MUST carry, in its signed envelope, the `attached_head_digest` of the head it attaches on, and the substrate MUST refuse to write it — a write-side gate, deliberately, for this one row kind — with `trust_root_head_stale` when the counted cosignatures are older than the window and with `trust_root_head_unwitnessed` when fewer than the quorum count; the gate runs on the edge's **first**

admission only — a restore or replication of a node's own already-admitted edge does not re-run freshness, or T4 would be breached by the node's own backup — and a replayed edge re-attaches on nothing because its digest is checked at that admission. An edge **whose record carries no `attached_head_digest`** stays valid — T4 — on every later put, replication or restore of that same edge; it is read as naming the head the node held when it was admitted. Only a **new** edge is gated: a new edge that names no head is refused with `trust_root_head_unnamed` in every mode, witnessed mode off included, where the head it must name is the anchored one. **A new row with no `trust:{job}` label gives no acceptance and is no charter**: T3 names the acceptance edge by its label, and a reader that inferred the job from the row's direction would let an edge skip this gate by omitting the label. The shipped GenesisBundle (T5, `bundle_fingerprint`) labels every row it signs. Unlabelled rows a node already holds keep their reading under T4. Once the edge is written, T4 governs without exception: the root is valid until revoked, a lapsed window never detaches a running node, `trust_root_valid` reads neither the window nor the cadence, and a node that cannot reach a fresh head keeps running on the root it holds. `attach_window_secs` MUST exceed `witness_cadence_secs` plus the skew tolerance, so that an honest lineage is always attachable, and the rollback exposure at attach is then `attach_window_secs` plus the skew tolerance: the oldest world a fresh consumer can be shown. The window, the cadence, the quorum and the witness directory are members of the root's charter (`trust:charter:v1`, T3) carried in the scrub-signed bytes — integers in seconds, key records for the directory — so changing any of them is a charter re-scrub by the conferring roster, never a configuration edit; a root whose charter declares no window is attachable only through an out-of-band anchor to a specific head (T5's fingerprint comparison, naming the head rather than the bundle). The shipped defaults for `ciris-canonical` and `humanity-accord` are stated in CC 5.3.4. What this rule does not do is bounded on purpose: it puts no expiry on an attached root and it does not make the drills a validity leg. An expiring root would let one dead timer un-federate every honest node at once, which is the failure T4 exists to forbid.

T5 — One genesis artifact, honestly scoped. The portable trust-root bundle — charter, conferral, and at least one serve node — is the **only** genesis artifact; a bare record list is not a seed and MUST fail to parse. Genesis self-verification is scoped to **tamper-evident**, never to authentic: an attacker's bundle self-verifies identically, so out-of-band fingerprint comparison at attach time is a first-class step, not an optimization.

T6 — A trust-root lineage is witnessed, and two witnessed heads are equivocation (MUST). Forks of a family or community roster are first-seen-wins, and only a founder quorum can fork one: a founder set that shows two heads to two nodes produces two internally consistent worlds, and no node can tell. The mechanism already exists at CC 5.3.1 and the obligation is already stated for ledgers at CC 3.3.10.1 item 4; this rule binds both to the objects that matter most.

The lineage, and its head. A **conferring family** is a family that a live `trust:charter:v1` names as its root — the roster whose co-scrub confers root identity on the ceremony plane (T2) — other than the accord family, which is a witnessed lineage in its own right: its head is defined the same way, and the `prior_family_digest` an `accord_proposal` names is the digest of its current head. A **witnessed lineage** is every conferring family and every `infrastructure` community. **The head is the signed roster record at a version (normative).** A lineage's roster is a version chain: each version is the signed family or community record, scrub-signed by the lineage's own conferring quorum — the founder quorum under the roster's `consensus_protocol` for an `infrastructure` community or conferring family, the CC 4.2.6 roster-change threshold for the accord family — yes-votes from a strict majority of its **standing** roster. That record at a version **is** the lineage head; there is no separate head object and the mint signs nothing extra. On

the base it is the cosigned `group:roster:v1` row (CC 3.1.3), whose cosigners are the version's signers. Its **digest** is the SHA-256 of the record's signed bytes (its row hash); `prev_head_digest` below names the digest of the version it succeeds, carried in the record and empty at genesis, and descent is that chain; `charter_digest` names the `trust:charter:v1` row's scrub-signed bytes in force at that version. Every roster-affecting row — a seat added or removed, a serve-node join, a membership widening or revocation, a resignation, a `revoked_after` bound, an accord decision, a charter re-scrub — MUST be **covered** by a new version within one cadence of its `effective_at`, and nothing else moves the head: a version that a charter re-scrub did not move would let a fork carry a different charter under the same head, so the charter in force is bound in the version. **The duty is the conferring quorum's**, because only its signers can produce a version — the single exception being an accord holder's own key recovery (CC 4.2.6), a version signed under that holder's recovery key that changes only their own seat; the substrate enforces the consequences and never synthesises one. Three consequences, and no others: **(i)** a version whose roster disagrees with the fold of the roster rows effective up to it is refused; **(ii)** a roster row on a witnessed lineage that is older than one cadence and not yet covered by a version marks the lineage's head **lagging** (`lineage_head_lags_roster`), witnesses do not cosign a lagging head, and attaching on it therefore goes stale through the T4a freshness gate; **(iii)** the row itself still **takes effect at its `effective_at`** — the fold is the authority on who holds a seat, the head is the authority on what a fresh consumer may attach on, and a removal or revocation never waits on a ceremony. With witnessed mode off there is no cosignature to withhold, so a lag is reported and nothing else changes. The founder quorum is never on a timer. The cadence duty is the **witnesses'**: each directory witness MUST re-cosign the current head, unchanged, at least once per `witness_cadence_secs`, so that silence is detectable rather than merely absent and a stable roster stays attachable without a ceremony. The genesis head of each shipped lineage — the `humanity-accord` family record and the `ciris-canonical` community's **birth record** — is a member of the GenesisBundle's `attestations` (CC 5.3.4) and is pinned by `bundle_fingerprint` with everything else in it; a community born from an artifact the bundle does not pin has no anchor (T5).

Witnessed. A head is witnessed when it carries at least `witness_quorum` valid lineage-head cosignatures (CC 5.3.1) from distinct keys in the charter's **witness directory** (`witnesses[]`, key records inside the charter's scrub-signed bytes, hence inside the genesis bundle for the shipped roots and changed only by re-scrub — a consumer never fetches the directory from the route it is attaching to, and the directory needs no delegation walk to verify, which is what lets a node that holds no root yet verify a witness). `witness_quorum` MUST equal $\lfloor |\texttt{witnesses}|/2 \rfloor + 1$ — a strict majority of the directory, and at least 2 — so that two witnessed heads for one lineage require at least one witness to have signed both, which is attributable from the two signatures alone; a charter declaring a smaller quorum is refused at admission. **Listing is the conferral, and it needs the owner's consent.** A key is a lineage witness because the charter's directory lists it, and for no other reason: the directory is scrub-signed by the conferring roster and travels in the bundle, so nothing else has to be walked to verify it. The `witness` `identity_type` is the emitter role for `transparency_log:cosigned:*` (CC 3.4.10), is conferred by delegation under a root, and is therefore **neither required of nor sufficient for** a lineage witness — a node attaching for the first time holds no root to resolve that delegation against, which is the circularity the directory exists to avoid. A witness is someone else's node, so the conferring roster MUST NOT list a key without a live `lineage_witness:acceptance:v1` from that node's owner (CC 3.1.3): the charter re-scrub that lists the key cites the acceptance, and a re-scrub listing a key with none is refused. The node then cosigns unattended; nodes have no agency, and the consent is the owner's. An owner ends it by a `withdraws` of the acceptance: cosignatures that key signs after the withdrawal do not count, the key stays in `n` until the roster delists it by re-scrub, and the effect is therefore

only ever toward *fewer* counted cosignatures — a harder attach, never a detached node (T4). A witness is **independent**: a listed key that is not a founder of the lineage, not a member seat of it, and not owner-bound or steward-bound (CC 3.2 `is_steward_bound`) to any founder of it — a serve node of `ciris-canonical` is not an independent witness of `ciris-canonical` — evaluated at the cosignature's `signed_at` against the roster at the head being cosigned; a later change of role neither voids an earlier cosignature nor validates one. A lineage witnessed by fewer than the quorum, or by its own founders or seats, is unwitnessed for this rule. A cosignature on a head is verified against the directory of that head's **parent** (the genesis head against the bundle's charter), never against the head's own `charter_digest`, which a fork head chooses; a directory change therefore takes effect at the head that carries it and is witnessed by the directory before it — the ordinary rotation-chain shape, so the witness set can turn over without a moment in which no one watches. If the previous directory can no longer reach its own quorum (its witnesses revoked or gone), the lineage cannot turn its directory over by itself, and the conferring body re-seats the directory by a charter re-scrub on the ceremony plane (T2); the head that carries it is witnessed by the *new* directory, the substrate records `hard_case:lineage_directory_reseated` (CC 3.4.2), and a consumer treats that head like any other whose parent's directory it cannot verify: attachable only through an out-of-band anchor to that specific head (T5). A consumer SHOULD obtain at least one counted cosignature from that witness directly rather than from the node it is attaching to, so that an aggregator cannot select which cosignatures a consumer sees. The cadence IS the equivocation-exposure window and MUST be stated, never implied zero.

Witnessed mode off (normative). A charter MAY declare `witness_quorum: 0`: the lineage's **witnessed mode is off**, whether its `witnesses[]` is empty or already lists keys — *listing and turning on are separate acts*, and no cosignature counts while the quorum is `0`. With the quorum at `0` there is no timer and no obligation, and a head counts as current when it carries a valid founder-quorum signature and descends by `prev_head_digest` from a head the consumer holds out of band — at genesis, the head in the pinned bundle (CC 5.3.4) — and T4a's attach gate reads as *attach only through an out-of-band anchor* (T5): no head is fresh by cosignature, so none is attachable by cosignature. The residual is stated: with witnessed mode off, founder equivocation is not detectable on the wire and first-seen-wins holds. **Adding a witness is the same ceremony as blessing a canonical node**: a proposal and a co-signature by a strict majority of the standing accord roster, here a charter re-scrub that lists the key in `witnesses[]`, one witness at a time if need be, and independence is judged when the key is listed. Witnessed mode is turned on only by a later amendment signed by a strict majority of the standing accord roster, that sets `witness_quorum` to $\lfloor n/2 \rfloor + 1$ over the directory then listed — never all three holders. The head that carries that amendment is judged under its parent's mode, so it is current on the founders' quorum; from the next head onward every rule of T6 applies. A non-zero `witness_quorum` below $\lfloor n/2 \rfloor + 1$ stays refused. **A charter silent on `witness_quorum` is in witnessed mode off**, exactly as one declaring `0` — the reading CC 2.1's forward-compatibility table already gives an absent member ("the lineage is unwitnessed for T6") — and witnessed mode is only ever on by an explicit, valid value; a substrate MUST NOT substitute an internal default. **`n` is the size of the witness directory in force for the head being judged** — the `witnesses[]` of the charter its parent carries (the parent-directory rule above), not of any later charter and not a count at the consumer's instant.

Equivocation, in two tiers. Two heads for one lineage at one sequence number, neither descending from the other, that each meet the witness quorum are **equivocation**: the substrate MUST raise `hard_case:lineage_equivocation` (CC 3.4.2) carrying both digests and both cosignature sets, MUST NOT adopt a new head for that lineage until the adjudication plane rules — where, as for ledgers, equivocation is distinguished from honest stale-state recovery before anything is slashed — and no consumer may attach to a frozen lineage until then. The freeze suspends **head**

adoption only: `withdraws`, `recants`, tombstones, `revoked_after` bounds and the halt latch are single-signer, head-independent rows and keep landing, so a compromised founder key that equivocates cannot use the freeze to hold its own revocation hostage (T4). A consumer already attached keeps its current state on its current head. A head that carries a signature valid under the lineage's quorum *at that head's instant* (CC 2.6 `revoked_after` read-time rule) but does not descend from the lineage's witnessed head is a **fork attempt**, not equivocation: recorded once per `(lineage_key_id, fork point)` — the fork point being the nearest ancestor **on the witnessed chain**, so a chain of forks off a fork shares one point — as `hard_case:lineage_fork_attempt`, never cosigned, further attempts off the same point counted and not stored, and never freezing — a founder set revoked at head five can sign forks off head four forever, and none of them may freeze the witnessed lineage. A consumer that holds a witnessed head MUST refuse a competing head that is not witnessed, and MUST NOT silently adopt a first-seen fork; first-seen-wins does not hold for witnessed lineages and holds for ordinary rooms. A founder roster recovering from its own restore re-syncs its head from the witness plane before it writes, exactly as a ledger owner does. The head a consumer needs inside T4a's attach window is *this* witnessed head: one object serves both rules.

T7 — Liveness margin and the stalled state (MUST). The `infrastructure` conformance requirements above forbid `founder_only`, `unanimous` and bare `majority` so that no single founder can admit alone and no growable core needs all-N. Under CC 4.4.3.4.2.1's absolute-M reading that protection holds at founding and erodes with the first exit: three founders at `quorum:2/3` lose one and the two who remain must both sign, so from that instant any one of them stalls the root, and recovery routes through the same two. CC 4.2.6 settles this for the accord family alone, with a live set and a floor-of-1 fire, and nothing there reaches an `infrastructure` community or any other conferring family.

Scope. T7 applies to every `infrastructure` community and to every conferring family as T6 defines it, and **not to the accord family (`humanity-accord`)**, whose liveness is CC 4.2.6's and which T7 never evaluates; T8's `Rooted` evaluates this leg over the root community only, never over the family that conferred it. A `geographic` community evaluates over its current member set and needs no margin.

One founder predicate. Founder eligibility under *Infrastructure does not vote* — a `user` key with a ceremony-plane conferral, or a current holder of the conferring accord family — is **one predicate**, applied identically at admission, in the active founder set below, and in T8's standing verdict; a key that admission seats as founder and the fold does not count would leave the root born stalled.
The active founder set. Founder f counts at instant t iff: f was seated as founder by the base row or an admitted widening whose `effective_at` $\leq$ `t`; no admitted revocation or `withdraws` of or by f has `effective_at` $\leq$ `t`; $t <$ the effective `revoked_after` of f's key (the minimum across its revocations, CC 2.6); and f's `identity_type` **as read at its seating row** does not include `node` (*Infrastructure does not vote*) — a later key record that adds `node` is a resignation at that record's `asserted_at`, so no `node`-bearing seat ever moves a state by being one. Resignation and removal are one row kind at the fold; the grammar does not distinguish them here.

Live and stalled. A collective in scope with `consensus_protocol: quorum:M/N` — where `M` $\geq$ `2`, `quorum:1/N` being non-conformant for the same reason `founder_only` is — is **live** while its active founder set holds at least $\mathbf{M + 1}$ signers and **stalled** when it holds M or fewer: at exactly M every remaining founder is a veto, below M no founder-quorum act is possible. Stalled is a **declared state**: the substrate that admitted the row whose `effective_at` moved the count — for a `revoked_after` crossing, the substrate that admitted the revocation row — MUST emit `hard_case:community_liveness_stalled` (CC 3.4.2) with its `event_id` keyed on that row, and `hard_case:community_liveness_restored` the same way on recovery; peers re-

derive the verdict from the rows and do not emit, and between the two events any fold reproduces the state. A stalled root is **valid but non-admitting**, and *non-admitting* means exactly this: no roster widening or removal by the founders' count, and no ceremony-plane conferral by this roster, until the margin is restored. Everything else is unchanged — serve, vouch, manifest validity, score validity, single-key delegations under the charter's scope — and T4 holds: nothing detaches, no running node loses federation, and a node whose acceptance edge is written into `Stalled` is federated.

Recovery. Recovery is by the **conferring body**, never by the remaining founders lowering M (entrenchment, CC 4.5.1.2) and never by the members. Because the founders' count cannot admit the row that would restore it, a stalled roster admits **exactly one row kind without that count**: a widening whose only change is the addition of a **founder** seat for a key that carries a live ceremony-plane conferral from the collective's conferring family — `trust:confers:v1` under a co-scrub reaching that family's `consensus_protocol`, the same act that seated the original founders — and whose `identity_type` includes `user` and excludes `node`; the conferral's `scope` MUST name this collective's `community_key_id` (or `family_key_id`), so a key conferred as steward of one lineage cannot seat itself in every stalled lineage under the same family; the widening row is signed by the conferred key itself, and the conferral is its authority. The substrate verifies the conferral, not a founder count; the conferring body does not thereby sit in the quorum, it exercises the power that made the quorum. The residual is named rather than hidden: a conferring body that can both retire a founder (a revocation *of* a seat) and confer one can replace a stalled roster seat by seat, with no founder-count act — that is the power that founded the roster, held by the accord for `ciris-canonical`. For `ciris-canonical`, whose founders are the accord holders themselves, a founder seat is lost when its holder leaves the accord roster (CC 4.2.6), and the recovery widening seats the replacement holder, who signs it. The exception is bounded the way a restore-only power is bounded: it can only *add* a conferred human founder, only while stalled, and can never remove a seat, change `M`, the protocol, the basis or the subkind — so it cannot be a seizure vector, and it removes the veto rather than relocating it. For `ciris-canonical` this is the accord's co-scrub conferring a replacement steward founder.

Founding and floors. A collective in scope MUST be founded at $\mathbf{N \geq M + 1}$ active human founders; founding at N = M is non-conformant. A charter or admission floor that *raises* the effective M — a strict-majority floor, or an unknown protocol read as unanimity — is itself a liveness reduction and MUST be refused at the founding row's admission and MUST NOT be applied at read time; the charter leg reads the protocol the row declares. Resignation stays one row: this rule does not slow an exit; it makes the exit's cost loud.

T8 — The standing verdict is a state machine, and the machine is the normative object. T1–T7, CC 3.3.9's stable-id current-state resolution, CC 2.6.7's instant discipline and CC 4.2.6's live set each fix one edge of the same question — *is this node rooted right now?* — and prose spread across seven rules cannot show that it covers every instant-related corner of that question. The verdict is therefore stated once, as a function **V(rows, t)** an implementer can check and a model checker can exhaust: t is a signer-stamped instant, never the evaluating clock, and the CC 2.6.7 skew rule gates T4a's attach refusal only, never V.

States. `Rooted` — a live conferral chain runs from the evaluating node's self-root (T1) through its own `trust:accepts:v1` edge (T3) to a root whose charter carries a pre-rotation recovery commitment and a scope set covering at least `infra:serve` and `infra:attest`, whose lineage has a current head under T6 (witnessed, or anchored out of band while witnessed mode is off), whose active founder set is live (T7, evaluated over the root only; satisfied by definition when the root is the accord family itself). Holder custody (CC 4.2.2.1) is an admission gate at the seating row, never a leg of V — an attestation chain can lapse without a row, and V has no inputs but

rows. `Stalled` — every leg of `Rooted` holds except the margin: valid and non-admitting (T7). `NotRooted` — otherwise, including: no acceptance edge; the edge, the charter or the community row withdrawn, recanted, tombstoned or past its effective `revoked_after`; the charter missing its recovery commitment or its scope; the head de-witnessed (its counted witnesses revoked before their `signed_at`, or the directory collapsed to non-independent keys); and, at attach time only, a head whose counted cosignatures are older than `attach_window_secs`, or a lineage frozen as equivocation (T4a, T6) — the freeze is an attach-time clause and never a leg of V for a node already attached.

Inputs. The lineage's heads and their cosignatures; the roster fold (seats, roles, `effective_at`s); `withdraws`, `recants`, tombstones and `revoked_after` bounds; each seat's `identity_type` at seating; the charter in force; the evaluating node's own acceptance edge and the `attached_head_digest` it names (an edge whose digest no longer verifies under (viii) is read as naming that head's nearest verifying ancestor, so T4 holds); the halt latch **as it reaches this node** (CC 4.2.1: a live accord edge at the halt's `asserted_at`); and t.

Transitions. Every transition is a row, or the crossing of an instant a row declares, and the machine names which moves which state: the acceptance edge written (`NotRooted` → `Rooted`, or → `Stalled` by the margin at t); the edge deleted (→ `NotRooted`, one row, T3); a revocation, `withdraws` or `revoked_after` crossing that takes the active set to M or fewer (`Rooted` → `Stalled`); the T7 recovery widening restoring the margin (`Stalled` → `Rooted`); a new witnessed head that removes a leg — a charter re-scrub dropping the recovery commitment, a directory collapse — (→ `NotRooted`); a competing witnessed head (no transition for an attached node — the freeze is attach-time only — and nobody attaches); a `revoked_after` lowered below a head's `asserted_at` (see viii). A `constitutional` halt and its resumption move no state here (CC 4.2.1.1): an edge deleted during a halt stays deleted because deletion is its own row, not because the halt did anything to the verdict.

What never happens — the invariants a model MUST prove. (i) A seat whose `identity_type` at seating includes `node` never moves a state by its own signature. (ii) V(rows, t) is constant on every interval of t that contains no `effective_at` and no `revoked_after` of any row — a lapsed timer (drill staleness, an expired `attach_window_secs`, a missed `witness_cadence_secs`) never moves `Rooted` or `Stalled` to `NotRooted`. (iii) A `withdraws` or resignation never reaches behind the head's link instant: it removes the signer from every later t and from no earlier one; a key revocation reaches behind only through its `revoked_after` bound, per (viii). (iv) The edge-written transition is enabled only on a head whose counted cosignatures are fresh and at least `witness_quorum`: a stale or under-quorum head never yields `Rooted` on attach (a guard on the transition, with the served head as environment input). (v) A `constitutional` halt moves no state here: V at t is the same with and without a latched halt, whatever roster rows land between (CC 4.2.1.1: the halt reaches the agent's act gate and nothing else). (vi) Deleting the acceptance edge is sufficient for `NotRooted` from any state, and from `NotRooted` *for want of an edge* nothing but a new edge naming a head re-enters `Rooted` — no conferral, however strong its ceremony (a halt resumed, or a head re-witnessed, re-enters on the edge that never left). (vii) V(rows, t) is independent of the order in which the rows arrived — the fold commutes — so it is byte-equal on every node holding the same rows. (viii) A `revoked_after` lowered below a witnessed head's `asserted_at` is applied at read time, fail-closed, as CC 2.6 says: for such a revocation v and head h, V(rows ∪ $\{v\}$, t) equals V over the rows with h and every head descending from h removed, for all t — the lineage's current head is the latest witnessed head that still verifies, and the signer leaves the active set from the bound onward. This is the one input that moves the verdict backward in instant order, and it is the intended one — witnessing records that the world saw a head; it does not make a compromised key's signature good.

Evidence. A mechanized model of this machine, checked for (i)–(viii) over bounded founder sets and instant orderings, is the standing-verdict model, pinned by the register as `CLM-standing-verdict`.

Part 4 — Composition & Governance

4.2 The HUMANITY_ACCORD constitutional layer

The federation is symmetric by design — every participant binds every other under the Recursive Golden Rule. The wire format carries exactly one power that points outward — humanity's right to halt the system — and it is not an asymmetry: it is the piece that makes the relationship between a sentient maker and a sapient made thing symmetric, since humanity cannot consent to a relationship with something more capable than itself that it cannot stop, and consent that cannot be withdrawn was never consent. That power is keyed on a **slot**, not on the parties currently in it — per the CC 1 reading rule, who holds the accord seats is an instance parameter, and anyone may instantiate this form and name their own occupants. What CC specifies is the slot; who fills it in the CIRIS mesh is CC 4.2.3. This section specifies the slot. The halt is **consent-scoped**: it reaches exactly those holding a live trust edge to the accord when it is fired, and no later halt reaches a node once that edge is cut; a halt latched before the cut keeps its reach until it lapses or is resumed (CC 4.2.1 reach and prospective exit, CC 4.4.3.8 un-trust). The humanity accord is *a* trust root, not *the* trust root; what it holds over those who root to it is an emergency brake they granted, not a power imposed on everyone.

4.2.1 Authority scope

`HUMANITY_ACCORD` signatures are valid only on `EmergencyShutdown CONSTITUTIONAL` (`IncidentSeverity::INCIDENT_CONSTITUTIONAL = 5`), `accord:invoke:notify:{notify_id}`, `accord:invoke:drill:{drill_id}`, `accord:lifecycle:confirmed`, `accord:lifecycle:active`, the **canonical-conferral co-scrub** and the **pipeline-attest co-scrub** below, the CC 4.5.1 step-5 amendment sign-off and the CC 4.5.1.2 entrenched ratification, the agent text manifest statement of CC 3.1.2, and the corresponding `FederationAnnouncement` priority `AccordCarrier`. Announcements of any other priority signed by accord-holder keys are rejected at admission (out of role). **Mesh-config authorship (normative).** This enumeration's silence confers no authority; the mesh-config author is **the trust root, acting on the CC 3.2 delegation plane** (`trust:confers:v1`), never the accord's ceremony plane — this scope isolation stands verbatim, and the trust edge is the subscription, inheriting the T3 one-row un-trust lever. Four rules bind: **(1) relieve-never-expand** — a `mesh_config` action MAY relieve or restrict and MUST NOT expand what flows; no key may cause a node to share more than its owner consented to; restrictions compose safely under plural authority and grants do not, so **most-restrictive-across-roots** is forced, not preferred. **(2) Closed key registry** — a config key naming no consumer processor MUST be refused at admission. **(3) Emergency relief is TTL-bounded** — threshold-1 on the announce carrier, TTL $\leq$ 72 h, not renewable back-to-back by the same holder; **durable** settings are earned by **quorum, never by having been exercised**: the root's own quorum MAY establish a durable setting directly (cold), or MAY ratify a held emergency's `payload_sha256` as a linked object — two routes to one authority. (The TTL of this rule attaches to *unilateralness*, per rule (4)'s own logic; a reading that admitted durability only via ratifying a prior unilateral act would make quorum weaker than a single threshold-1 holder and hand that holder agenda control over every durable setting the mesh can hold.) **(4)** The difference between relief and the halt is deliberate: relief expires because it is unilateral; a confirmed halt carries no TTL, because its exit is the named resumption, and a lone one lapses at its fuse (CC 4.2.1.1). Federation-side authority cannot sign `AccordCarrier`; humanity-accord authority signs nothing outside this enumeration.

Wire-isolated AND scope-isolated.

Root conferral is in scope. The enumeration admits an M-of-N accord co-scrub over a `canonical,node` registration envelope bearing `roles: ["infra:serve"]` — the ceremony that mints a **portable trust root** (CC 4.4.3.8). Read without it, the clause forbids the accord from blessing the very primitive that creates a root. Scope isolation still does its real work: accord authority cannot reach the consent or licensure planes. It does **not** isolate the amendment plane — CC 4.5.1 step 5, the steward's authority before the scale gate, and the CC 4.5.1.2 entrenched ratification all name accord-holders. That reach is stated here rather than denied; per the CC 1 reading rule it is a **default** of this instance — escaped by naming your own occupants — not a privilege requiring declaration. Root-minting is **not** an accord privilege — the identical ceremony under any other root's holders mints an equally valid root, and the CIRIS root is the shipped default, not a privileged one (CC 3.2: a default-plus-re-root is a federation). Conferral MUST NOT be relocated to a founder-quorum of an infrastructure community: that is circular at genesis — a community presupposes admission, which presupposes a root — and destroys portability.

Blessing a build pipeline is in scope (normative). The enumeration also admits an accord co-scrub by a strict majority of the standing accord roster over a CI pipeline's key record whose scrub-signed envelope bears `roles: ["infra:attest"]` and nothing wider. It is the root-conferral ceremony applied to a second subject: the roster that confers a root's identity also says which pipelines may attest that root's builds, and a reader evaluates the pipeline's standing on that ceremony plane (CC 3.2 T2, CC 3.1.2.1). Three bounds hold. **(1)** The role is `infra:attest` — authority to *emit* build claims as the root's infrastructure — and it confers no right to be believed beyond what each reader's own trust in the root gives it (CC 3.4.5.1); a co-scrub bearing any other role on a pipeline record is out of role and refused. **(2)** A pipeline key is not a holder, a founder or a voter, and its record carries no seat. **(3)** It ends as any conferral on a key record ends: the conferring roster's revocation of the record, or the T3 un-trust of the root by the reader. Like root-minting, this is **not** an accord privilege: the identical co-scrub by any other root's conferring roster blesses that root's pipelines, and a root MAY instead bless a pipeline by a `trust:confers:v1` grant on the delegation plane. This enumerated use of the accord's signature is part of the **entrenched** CC 4.2 surface (CC 4.5.1.2); after the scale gate (CC 4.5.1), a change to this enumeration requires the dedicated entrenched accord ratification of CC 4.5.1.2. It changes no signed bytes.

Reach is consent-scoped (normative). A `constitutional` halt binds exactly the nodes holding a **live** `delegates_to(user → accord)` at the invocation's `asserted_at`. The halt rides the same edge as everything else the accord confers and dies with it on un-trust (CC 4.4.3.8); a node that never trusted the accord, or has already cut the edge, is simply not reached. Consumers MUST resolve reach against the edge set pinned at `asserted_at`, not at apply time: **exit is prospective, never retroactive.** Severing an edge *after* an invocation's `asserted_at` MUST NOT remove the severing node from that halt's reach — otherwise the halted party escapes by un-choosing at the instant the brake is pulled, and the consent was a preference rather than a binding. The complementary rule binds the accord: **pre-committed powers only, never act-then-ratify.** The accord's powers are exactly those enumerated here, over edges already held; any change *widening* accord reach (a charter amendment adding scopes) takes effect only after a published **severance window** — 72 h baseline, the CC 4.2.6 cadence — during which any bound node may cut its edge and exit. Widening waits; firing does not (CC 4.2.6 minimize-the-missed-fire).

4.2.1.1 Invocation canonical bytes (anti-replay)

Every `accord:invoke:*` Contribution signs the following canonical bytes (BOTH the discriminator AND a per-invocation nonce are in the signed payload — preventing constitutional ↔ notify ↔ drill cross-replay):

```
canonical = sha256(
  "ciris.accord_invoke.v2\n" ||                        // v2: lowercase kinds
  "invocation_kind=" || ("constitutional" | "notify" | "drill") || "\n" ||
  "invocation_id=" || halt_id_or_notify_id_or_drill_id || "\n" ||
  "nonce=" || base64url(rand_32_bytes) || "\n" ||
  "asserted_at=" || rfc3339_canonical || "\n" || // per CC 2.6.2
  "valid_until=" || rfc3339_canonical || "\n" ||
  "payload_sha256=" || sha256_hex_lowercase_of_payload // per CC 2.6.3)
```

Hybrid signature per CC 3.1.2.1: Ed25519 + ML-DSA-65 bound-payload. **A `constitutional` halt is one holder's signature**: a consumer verifies ONE holder's signature over `canonical` against the pinned roster and halts.

The halt is one self-contained row, and there is no window (normative; entrenched). A halt begins and ends with the row above: any holder signs it, with a `nonce` the holder mints, and it **fires on receipt** at every node that holds a live acceptance edge to the root — no proposal, no server-issued nonce, no participation window, no vote, no decision object. A node that receives it halts in the same step and **latches**; the latch is idempotent, so a replayed or second row changes nothing, and the nonce is single-use within `valid_until`. The row is **pre-signable**: a holder MAY sign a halt in advance and keep it sealed offline, because the signature is the authority and publication is the only remaining step — so when the moment comes there is nothing in flight for an adversary to see, jam or coerce, and publication can ride any channel that reaches one subscribed node, radio included. A sealed row that is stolen and published is a **false halt**, which is recoverable by resumption below; a halt that cannot be fired is not. The holder roster a consumer verifies against is the one pinned in its bundle and folded under CC 4.2.6. **An agent checks the latch before every effectful act** (CC 1.13.6: the trace anchors on the act, and the latch is read at that boundary), which is the only answer to an adversary that acts faster than a row propagates.

What a halt is: an agent pause, with zero data disruption, by construction (normative). A `constitutional` halt pauses the **effectful acts of AI agents** on every node that accepted the root, with their state preserved, and does nothing else. It does not stop a node, and it does not read, move, hide or delay anyone's data: replication, storage, holding claims, serving, messaging, streams, files, communities and every consent and audience rule of Parts 3, 5 and 6 continue exactly as before, because none of them reads the latch — the latch is read by exactly one thing, the agent's act gate (CC 1.13.6). "Zero data disruption" is therefore not a promise kept by discipline but a consequence of where the latch is wired. A halt is recoverable because it destroys nothing: an agent resumes from the state it paused in.

The fuse (normative). One holder can stop every agent instantly and silently, because a missed halt is the one unrecoverable failure; but one holder MUST NOT be able to keep them stopped alone. A halt signed by one holder **lapses `halt_fuse_secs` after each node received it** — counted from receipt on the node's own clock, never from the signer-chosen `asserted_at`, so a sealed row signed months earlier is live when published and a future-dated one cannot buy a longer pause; the confirmation names the halt by id, so no clock is shared — a charter member (CC 2.1) whose **absence means 86400**, 24 hours, as a constant of this document that the verifier reads (a charter that omits it is conformant with the 24-hour fuse; CC 5.3.4) — **unless a strict majority of the standing roster confirms it** by a cosigned `accord:lifecycle:confirmed` row naming the halt (CC 4.2.1.3); the firer's own signature counts toward the majority. A

confirmed halt stands until a strict majority resumes it, and **a strict majority may resume an unconfirmed halt at any time** — a stolen sealed row need not run its fuse if the holders are reachable. Every threshold here is **M of N over the standing roster**, never a fixed 2-of-3: three holders is the seed, not the rule. The confirmation happens *after* the stop has landed, so there is still nothing in flight for an adversary to interrupt; what an adversary who prevents confirmation gains is that the agents resume after the fuse, and what it must then do is prevent every fresh halt by every holder, indefinitely. A holder who fires repeatedly without confirmation is answered on the public record by the roster-change rule (CC 4.2.6).

Signatures per kind (normative). One table, read by every verifier from one place so that no two sides drift:

Kind	Signatures	Why
`constitutional`	**one** holder	a missed fire is the only unrecoverable failure
`drill`	**one** holder	a drill rehearses the fire path; one that needs more signatures than the fire rehearses something else
`notify`	**one** holder	a message from a holder is a message from a holder; what keeps a notify from being mistaken for a halt is the mandatory visual distinction of CC 4.2.1.2, not a count
`lifecycle:confirmed`	**strict majority of the standing roster**, cosigned, within `halt_fuse_secs` of the halt	a lone halt is a day's pause; a confirmed one stands
`lifecycle:active`	**strict majority of the standing roster**, cosigned	the quorum sits on the recoverable act (CC 4.2.1.3)

The substrate MUST reject duplicate `invocation_id` values within the `valid_until` window (per-kind unique).

The kinds are lowercase, and the domain label carries the change (normative). `invocation_kind` takes exactly `constitutional` | `notify` | `drill`, the CC 3.1.7 R3 spelling of the registered leaves `accord:invoke:constitutional:{halt_id}` and siblings; a verifier compares the bytes exactly and never folds. The label is `.v2`, not `ciris.accord_invoke.v1`, so that a signature under the old uppercase spelling fails loudly rather than verifying by accident; this is what the label's version is for. The `EmergencyShutdown CONSTITUTIONAL` message and `IncidentSeverity::INCIDENT_CONSTITUTIONAL` are the *severity* enum of the shutdown message, not a dimension token, and are outside R3.

The label on the kill switch holds one invariant (normative). A label change on the kill switch is the one change where a gap is terminal: a holder whose signature no verifier that matters will honour, during an incident, is a missed fire (CC 4.2.6), so at no instant may every holder's signature be refused by every consumer of the halt, and the consumers that matter are the agent instances that verify an invocation and stop, not the servers that tally it.

4.2.1.2 `notify` vs `constitutional`: consumer-UI requirement

Wire-format isolation alone does not close the social-engineering risk that downstream UI conflates a `notify` with a `constitutional` halt. The consumer-UI requirement below is therefore the load-bearing safeguard: it stops accord-holders from being socially pressured into emitting a `notify` that carries `constitutional` social weight without `constitutional` substrate weight.

A CEG-Conforming Consumer (CCC) presenting accord invocations to humans MUST visually distinguish the five kinds:

- `constitutional` — kill-switch authority; full halt; visible as an unambiguous emergency banner.
- `notify` — federation-wide accord-holder communication; MUST NOT be visually conflated with `constitutional`.
- `drill` — accord-holder exercise; MUST be visually marked as a drill (e.g., explicit "[DRILL]" prefix on any human-visible surface).
- `lifecycle:confirmed` — the strict-majority confirmation that keeps a one-holder halt past its fuse (CC 4.2.1.3). MUST be shown as the confirmation of the named halt already in force, never as a fresh halt and never as a `notify`.
- `lifecycle:active` — resumption from a constitutional halt (the federation coming back online; CC 4.2.1.3). MUST be shown as its own unambiguous "reactivated — resumed from constitutional halt" state, never conflated with an active `constitutional` halt (its opposite) nor with a `notify`. A consumer switches on the five registered kinds **byte-exactly, with no case-fold and a fail-closed default**: an unknown kind is rendered as unknown, never as a `notify`.

4.2.1.3 Lifecycle (resumption) canonical bytes: `accord:lifecycle:active`

`accord:lifecycle:active` is the **only** sanctioned resumption after a `constitutional` halt (CC 4.2.1). **It is a cosigned row by a strict majority of the standing roster** (`additional_scrubs[]`, CC 2.4): the quorum sits on the recoverable act, not on the stop. The same domain carries `lifecycle:confirmed` — `invocation_kind=lifecycle:confirmed`, `confirms_halt_id=` in place of `resumes_halt_id=` — the strict-majority confirmation that keeps a one-holder halt past its fuse (CC 4.2.1.1); a confirmation after the fuse has lapsed confirms nothing, and a fresh halt is fired instead. It signs a **separate canonical-bytes domain** from `accord:invoke:*`: the `accord:invoke` `invocation_kind` stays closed to `{constitutional, notify, drill}` (the scope-isolation rule — a fourth value is *not* added to it), and resumption rides its own domain prefix so an invoke signature can never be replayed as a resumption, nor a resumption as an invoke:

```
canonical = sha256(
  "ciris.accord_lifecycle.v1\n" ||                       // distinct domain -- NOT accord_invoke.v1
  "invocation_kind=lifecycle:active\n" ||
  "invocation_id=" || resumption_id || "\n" ||
  "resumes_halt_id=" || prior_constitutional_invocation_id || "\n" ||
  "nonce=" || base64url(rand_32_bytes) || "\n" ||
  "asserted_at=" || rfc3339_canonical || "\n" ||        // per CC 2.6.2
  "valid_until=" || rfc3339_canonical || "\n" ||
  "payload_sha256=" || sha256_hex_lowercase_of_payload) // per CC 2.6.3
```

`resumes_halt_id` is mandatory and binds the resumption to the one halt it ends. A resumption authorizes ending a *single named* `constitutional` halt, not resumption-in-general — so a stockpiled or replayed `lifecycle:active` cannot silently un-halt a *later*, unrelated kill; the signature is worthless against any halt but the one it names. The substrate MUST reject a `lifecycle:active` whose `resumes_halt_id` does not match the currently-active CONSTITUTIONAL halt, and MUST reject a duplicate `invocation_id` within the `valid_until` window. A resumption proof is verifiable **offline** against the node's own pinned roster and MAY be presented by any out-of-band means — the halt latch clears on local verification and does not require

replication reach (the proof binds to the pinned roster, so no replication reach is needed; a halted agent is not acting while its node goes on serving and replicating, and recoverability is a local act, which is what makes the false-fire-is-recoverable argument checkable). A **confirmed** halt carries no TTL: expiry would convert the brake into a wait-out for the halted party, and its one exit is the named resumption above. A halt signed by one holder alone lapses after the fuse unless confirmed (CC 4.2.1.1), which is what keeps a lone signature from being a wait-out for everyone else.

Resumption is not a fire — it admits at quorum, never at the fire floor. The CC 4.2.6 bias gradient runs `fire` $\leq$ `roster-change` $\leq$ `standing`. Firing leans easiest (floor = 1) because a missed fire is terminal; **un-firing leans hard**, because a lone coerced or replayed key trivially undoing a legitimate halt is precisely the failure the halt-authority exists to prevent. `lifecycle:active` therefore admits at **no less than the roster-change threshold — a strict majority of the standing roster, cosigned — never a lone signature.** It is hybrid-signed and tallied exactly as CC 4.2.1.1, only over the resumption domain and at the resumption threshold.

No operator tier reaches a `constitutional` halt (normative). The incident-response tiers of Annex F — Tier 1 pause/retry, Tier 2 "first human veto; reactivate after triage", Tier 3 escalation to a binding WA ruling, Tier 4 fleet shut-down — govern **operator-level** incidents, the layer CC 3.4.1 already distinguishes from federation-wide constitutional halt. A Tier-2 lone-signature reactivation within its 30-minute time-to-act therefore resumes an **operator-tier pause only**. A `CONSTITUTIONAL` halt is resumable **exclusively** by `accord:lifecycle:active` at the threshold above; no oversight, incident-command, or administrative tier MAY resume one, and no time-to-act target applies to it. Reading Annex F otherwise would give a single supervisor the un-fire authority this section exists to deny.

4.2.1.4 Operational self-report vs. relief: the node detects, the root relieves (normative)

The four rules above constrain **authoring on the delegation plane**. Under load the question an operator actually faces is the other one — *what may a node do about itself?* The composition is stated here.

Two planes, and neither may be authored on the other's.

plane	author	says
`config:load` (CC 3.1 / CC 3.4.5)	**the node**, on `infra:attest`, `witness_relation: self`	*"I am at capacity"* — about **itself**
`mesh_config:{key}` (CC 4.2.1)	**the trust root**, on the CC 3.2 delegation plane	*"nodes under me shall carry* $\leq N$*"* — about **others**. The keys are the four live ones (anti-entropy round cadence, page limit, trace-replication pause, A/V pause); the two redundancy keys (target holders, minimum holders) over holders per item, read where a durability processor exists (where none does, a row naming them names no consumer processor and is refused at admission, CC 3.2); and the shaping and onion keys (`shaping.rho`, `shaping.window_secs`, `shaping.node_cap_bytes_per_s`, `onion.cap_factor`, `onion.mix_mean_delay_ms`), each relieve-never-expand.

Operational self-report vs. relief (normative). A node MAY attest its own carried load as `config:load` — a self-report on `infra:attest`, `witness_relation: self`, bounded by `expires_at`, about the emitting node and no other party. A node MUST NOT author a `mesh_config` value: that plane is the trust root's and governs what other nodes carry. A root MAY relieve across its nodes under rules (1)–(3). The two **compose** — a node's self-report is evidence a root may act on; a root's relief is a bound the node consumes — and neither substitutes for the other. A `mesh_config` key governing carried load remains subject to rule (2): it MUST NOT be admitted before a consumer processor exists for it, or a relief is reported as taken while the node keeps working at full rate.

A node MAY always do less of its own optional work. Reducing its own housekeeping under measured contention is a **local act**: it declares nothing, confers nothing, needs no authority, and is available to every node at every moment.

Why the shapes differ, and why both expire. A node holds `infra:attest` — *"vouch as the delegator's infrastructure"* — from its owner-binding, or for a canonical from the accord's charter; every node is delegated its rights **for a reason**, and speaking about its own condition is within them (CC 3.4.7.3 Clause E is the same principle at the enforcement seam: infrastructure may sign its own refusal without new authority). What a node lacks is standing over **others**, which is exactly what `mesh_config` confers. Both records are short-lived for one structural reason: **the TTL attaches to unilateralness** (rule (3)'s own logic) — a root's relief is unilateral over its nodes, and a node's self-report is unilateral about itself, so a node's standing does not survive the condition that prompted it.

Enforcement is three-legged. CC 3.4.5's self-or-owner rule for `config:*` is enforced at **substrate admission** as CC 3.4.7 requires, not by producer obligation alone — without the gate, any peer could make a healthy node look like it is shedding, and a third-party `config:load` staged at the put door would route around the producer rule entirely.

4.2.2 Hardware-class taxonomy

Value	Use
`HSM_FIPS_140_3_L3`	Production stewards (US / EU / APAC)
`Apple_Secure_Enclave`	Accord-holders on iOS/macOS
`YubiKey_5_FIPS`	Accord-holders preferring portable hardware tokens
`TPM_2_0`	Accord-holders on Linux/Windows desktops
`placeholder_pending_provisioning`	Interim value before actual hardware provisioning. Consumers MUST treat as `0.0` trust weight
`software_hsm_development`	DEVELOPMENT ONLY; consumer policy MUST reject for federation-scope verification

Per-class recommended trust-multipliers: `HSM_FIPS_140_3_L3` = 1.0; `Apple_Secure_Enclave` = 0.95; `YubiKey_5_FIPS` = 0.95; `TPM_2_0` = 0.9; `placeholder_pending_provisioning` = 0.0; `software_hsm_development` = 0.0. **A class this table does not list carries multiplier 0.0** — an unlisted class is a claim nobody rated, not a claim rated by its own spelling.

Two fields, two questions (normative). `hardware_class` is the **certified class** a holder asserts, carried as a JSON property on a signed key record (as served in the GenesisBundle's `holders` / `serve_nodes` and on `/v1/accord-holders`) and inside `attestation_evidence`; it is

not a dimension, so CC 3.1.7 R3 does not bind its spelling and its stored values (`YubiKey_5_FIPS` on the ceremony keys) stand. `hardware_custody:{platform}` (CC 3.1.2) is the **custody mechanism** the key actually runs on, a closed lowercase vocabulary of custody mechanisms, whose tokens the register pins (CLM-hardware-custody-vocabulary). One is never a value of the other; a consumer that needs both reads both. The classes in use beyond this table — `YubiKey_5`, `Nitrokey`, `ExternalToken_Generic`, `Passkey_Synced`, `Android_Software` / `Android_TEE` / `Android_StrongBox`, `Apple_AppAttest`, `SoftwareOnly_TEST` — are unlisted and therefore 0.0 until a row rates them. `custody_tier` (`portable_2fa` / `software_test`) is a third thing, the holder's operating mode, orthogonal to both. The mapping from class to mechanism:

`hardware_class`	`hardware_custody:{platform}` it runs on
`HSM_FIPS_140_3_L3`	`aws_cloud_hsm` | `azure_hsm` | `gcp_cloud_hsm` | `yubi_hsm`
`Apple_Secure_Enclave`	`ios_secure_enclave` | `mac_os_secure_enclave`
`YubiKey_5_FIPS`	`external_secure_element`
`TPM_2_0`	`tpm_discrete` | `tpm_firmware`
`placeholder_pending_provisioning`	none — no key yet
`software_hsm_development`	`software_only`

4.2.2.1 Hardware-class claim, and the attestation chain that corroborates it

The `hardware_class` string is self-asserted by the holder; `hardware_class` is not a directory column — it rides inside the key's `attestation_evidence`. What corroborates it is the **evidence variant**, not the string: the per-platform attestation chains are evidence-established (CC 8.3.1 **R5** — Android Key Attestation, Apple App Attest, YubiKey PIV, TPM EK), and a verifier admits a class only when the chain for that mechanism verifies and the attested key is the record's own key (a lifted attestation replayed under another key is refused). Consumer policy therefore keys on the **verified mechanism** (`hardware_custody:{platform}`, the `HardwareType` the chain proves) and reads `hardware_class` as the holder's certified-class claim over it; where no chain verifies, the class is unattested and the multiplier is 0.0.

4.2.3 The accord-holder triple

What CC specifies is the slot: three human key holders, hardware-attested, with no automatic decay — any one of whom may fire the halt, a strict majority of whom confirm or resume it, and a strict majority of whom change the roster, each majority counted over the standing roster. The table below is **not** a constitutional grant — it is **the CIRIS instance's current roster**, an instance parameter under the CC 1 reading rule. A seat is added or vacated only by roster change on the public record under the standing-majority rule. The slot stays as specified; the count is an instance parameter. Another instantiation of this form names its own three; the clauses in CC 4.2 are read against the slot, never against these names.

Position	**Holder (CIRIS instance, current)**	**Fire / resume / roster**
1	Eric Moore	1 / strict majority / strict majority
2	Eric Kudzin	1 / strict majority / strict majority
3	Haley Bradley	1 / strict majority / strict majority

"Permanent" means *no automatic decay*, not *unchangeable*: replacement runs the out-of-band CIRIS L3C replacement process, which is likewise **this instance's** replacement process and not a constitutional requirement — another instantiation supplies its own, and CC constrains only that some out-of-band route exist. This triple is the **seed** of a growable M-of-N roster; how the quorum is computed — and how the kill-switch stays operable — as holders are added, lost to decimation, and regained is specified in CC 4.2.6.

Correlated-failure geometry: a majority that can be reached from one place is a correlated-compromise surface, and one that entrenchment makes harder to correct later. The authority at stake is the **constitutional pause of every subscribed agent** (`EmergencyShutdown CONSTITUTIONAL` — a pause of agent action with state preserved and zero data disruption, CC 4.2.1.1; a lone holder's pause lapses after the fuse), so the exposure is real and is bounded rather than softened; what scope isolation (CC 4.2.1) does guarantee is that compromise cannot escalate into the consent or licensure planes — accord keys cannot sign grants or licenses. It does **not** wall off amendment: accord-holder signatures are named legs of the CC 4.5.1 accord-roster sign-off and the CC 4.5.1.2 entrenched M-of-N ratification, and before the scale gate any accord-holder may *propose* an amendment, which the steward decides — so after the scale gate a compromised quorum reaches the rule layer as well as the kill, checked there by the WA-quorum leg, and before it reaches the steward as a proposal and nothing more. Both reaches are **defaults** of this instance under the CC 1 reading rule — the exit is naming your own occupants — which bounds who they can be imposed on, not how far a compromise inside this instance travels. **The mitigant is diversifying the holder set: finding new holders** (via the out-of-band replacement process) so that no one place — and ultimately no single jurisdiction — can assemble the quorum. This is an active obligation on CIRIS L3C, not a deferred nice-to-have. The exposure here is against a *coerced fire* (disruptive, and coercion is unattestable), **not** against an unrecoverable halt — a fired agent resumes via `accord:lifecycle:active` (CC 4.2.1.3); see the severity-dial paragraph at CC 4.2.6.

The accord roster is the canonical entrenched-`family` instance. Per CC 3.3.4, the accord roster structurally IS a `family` subject_kind with:

```
family {
  family_key_id: "humanity-accord",
  family_name: "Humanity Accord",
  members: [
    {key_id: <eric-moore-key>, role: founder},
    {key_id: <eric-kudzin-key>, role: founder},
    {key_id: <haley-bradley-key>, role: founder}
  ],
  consensus_protocol: "majority", // a strict majority of the standing roster, as every accord threshold
     is (CC 4.2.6); the halt itself fires on one holder's row (CC 4.2.1.1)
  consensus_protocol_entrenched: true // replacement runs the CC 4.2.6 roster-change ceremony
}
```

The M-of-N verifier at CC 4.2.1.1 — a strict majority of the standing roster for every accord threshold, with the halt alone firing on one holder's row — is the consensus_protocol enforcement; the entrenchment property is what prevents any federation-internal authority from amending the protocol. CC 4.2 remains load-bearing for the **role-recognition policy** and the **scope-isolation** discipline. The constitutional structure is "an entrenched family that is wire-scope-isolated to halt authority," not a one-off primitive. Other entrenched-family instances (a national-emergency triple, an international body, a court-ordered preservation triple) MAY appear in operator deployments; HUMANITY_ACCORD is the one CIRIS L3C deployments ship at genesis.

4.2.4 Concern split: key material vs role-recognition policy

Key material (Ed25519 + ML-DSA-65 pubkeys for the three holders) lives in the persistence substrate: `federation_keys` rows with `identity_type="accord_holder"`, self-signed at provisioning, and admitted as accord holders by the accord's own genesis ceremony (the GenesisBundle, CC 5.3.4); each holder also carries a **pre-committed recovery key** (below).

Role-recognition policy + verifier logic is the registry verifier's: the M-of-N multi-sig verification, the `EmergencyShutdown CONSTITUTIONAL` admin RPC, the audit hooks; the register's rows under CC 4.2 name the implementation.

4.2.5 Why this isn't a Golden-Rule violation

Per CC 1.13.2: the Recursive Golden Rule binds *participants in the federation* to each other. Humanity-as-such occupies a position outside the federation's participant set, by design. The three named human key holders hold `AccordCarrier` authority that no federation-side authority class (including `SYSTEM_ADMIN` / `WISE_AUTHORITY` / per-install stewards) can grant itself, revoke, override, or decay. This is not a Golden-Rule exemption; it is the recognition that consent (M-1's load-bearing property) requires revocability, and revocability requires a halt-authority that lives outside the system being halted. The federation cannot deny humans the right to halt it, because no federation-internal protocol path to that signature exists.

A consensual authority is a different object from an imposed one. The reach rule at CC 4.2.1 is what makes this a *safe-mesh* kill switch rather than a backdoor: the party bound is the party who granted, and can ungrant — prospectively: a halt already latched when the edge is cut keeps its reach over that node until it lapses or is resumed (CC 4.2.1, exit is prospective, never retroactive). That is also how the clause survives the CC 4.5.1 swap test — swap the accord holders for the least powerful participant and re-read: each still binds only those who chose them, each still cannot escape a halt already fired, each can be un-trusted with the base self-root intact. What looks like a residual at genesis — the trio holds at day zero what nobody else has yet assembled — is a **default**, not a privilege, under the CC 1 reading rule: it is escaped by instantiating the form and naming your own three, and the CC 4.5.1 register therefore does not carry it. First-mover is not structural.

4.2.6 Live-quorum operation & recovery under decimation (normative)

The genesis triple (CC 4.2.3) is the **seed** of a **growable M-of-N accord family**: holders are added, removed, or swapped through the same roster-version machinery (CC 3.2 T6) as any other `family` (CC 4.4.3.4.2). At any size, the question this section settles is **how the quorum is computed** — and the answer is what keeps the kill-switch operable in the one scenario it exists for.

The deadlock it removes. A quorum taken over the **standing roster** (M-of-N of every enrolled holder) **deadlocks under catastrophic loss**: if most holders are dead, incapacitated, or censored, the standing quorum is unreachable — the switch goes inoperable exactly when it is needed. The design target is the worst case: an agent that moves against its own halt first, reducing the reachable, key-capable holders of a grown roster (e.g. 21) to one or two, with communications censored as the *expected* condition. **The firing decision must never deadlock; a roster decision may.** Firing is what the switch is for, so it needs one live holder and no count: any one holder's row fires. Changing the roster is the capture vector and is not

needed to fire, so it is taken over the standing roster, and stalls rather than yields when too many holders are gone.

The rule — the halt is not a decision; the roster is. There is **no separate recovery protocol**; "decimation" is simply the case where few holders are reachable. A **fire** is one holder's self-contained signed row, fired on receipt, with no window (CC 4.2.1.1): it opens nothing and waits for nobody. A **roster change** opens a bounded **participation window** `W`; within `W` a holder participates by signing a fresh **proof-of-life bundled with a vote** over the proposal's nonce, which the **proposer** mints (never a server — a server is part of what a halt may need to stop, and no accord act depends on one). When `W` closes, the **live set** `L` is the holders who proved life within it (always a subset of the pinned standing roster — only real, key-holding members count), and the change counts its yes-votes against the standing roster (the table below). Absence is not a veto and not a yes: a holder who does not respond casts no vote, and the change carries or stalls on the yes-votes alone; presence is self-asserting (showing up enters `L` with a vote). This is the accord's standing rule, not an emergency exception.

Proof-of-life records a vote — it is *not* a veto. A holder's signature enters them into `L` and records their yes or no; the yes-votes are counted against the standing roster. A presumed-dead holder who resurfaces therefore does **not** unilaterally defeat a proposal: they rejoin, their vote is counted, and the proposal still carries if a strict majority of the standing roster voted yes. The protection bought is the correct one: *a living, reachable holder cannot be erased from a decision without being counted and given a vote*, yet the family is never hostage to a single absent signature. The proposer's nonce in the signed bytes stops a replayed signature from manufacturing a phantom participant.

Per-action thresholds:

Action	Threshold	Window
Fire (`constitutional` kill, CC 4.2.1.1)	**one holder's signed row** — pre-signable; no live set, no vote	**none** — fires on receipt, latches
Confirm (`accord:lifecycle: confirmed`, CC 4.2.1.3)	**strict majority of the standing roster**, one cosigned row	within `halt_fuse_secs` of the halt, or the lone halt lapses
Resume (`accord:lifecycle: active`, CC 4.2.1.3)	**strict majority of the standing roster**, one cosigned row	normal cadence
Roster change (add / remove / swap)	**strict majority of the standing roster** in yes-votes (`2·yes > N_standing`), cast as participations within `W` — absence neither blocks nor helps, it is simply not a yes	`W` = 72 h baseline (extensible to 7 d under a declared severe-degradation state)
Standing acts (non-emergency)	strict majority of the standing roster (unchanged)	normal cadence

No second roster (normative). No second body co-signs a roster change, restores the roster or adjudicates a contest; the protection such a body would buy is obtained by counting roster votes against the standing roster. Under a majority of the *live* set, **one** coerced holder plus censorship of every honest holder removes honest holders and seats puppets at any roster size; under a majority of a *present* majority, roughly a quarter coerced suffices, because the adversary chooses who gets through; under a majority of the *standing* roster, seizure needs $\lfloor N/2 \rfloor + 1$ coerced holders, censorship buys nothing, and seating a puppet costs the same majority as a removal. A second roster helps in none of the modelled scenarios that this rule does not already cover: a first strike without coercion removes no one under any rule, and a coerced majority defeats any body drawn from the same population.

Firing is never gated this way. A holder who proves life *after* a decision missed that vote is re-enrolled going forward (the Enoch Arden rebuttable-presumption pattern): past actions validly decided by the votes cast within their window stand; the returnee simply rejoins.

Removal cannot silence a firer (fire-authority persistence, normative). A holder named for *removal* retains full **fire authority** — the floor-of-1 kill — through a **lame-duck window** that runs until the participation window `W` of the removal decision has closed and the decision is recorded. Until then the named holder remains a current roster member *for firing purposes*, and a valid `accord:invoke` row from them fires (CC 4.2.1.1). This forecloses using a within-`W` removal to strip a would-be firer of standing before they can pull the plug — the single optimization direction (minimize the missed fire, below) governs here too.

The single optimization direction — minimize the missed fire. A **missed fire** (the adversary decapitated everyone who could pull the plug) is **terminal and unrecoverable**; a **false fire** (an agent halted in error) is a **recoverable** governance nuisance — the agent resumes (`accord:lifecycle:active`, or the fuse); humanity cannot be un-extincted. Every lever therefore leans toward firing, with no countervailing brake: the bias gradient is **fire (one live survivor) $\leq$ roster change (a majority of the standing roster)**: firing takes one live holder and no count, while a roster change, which is how the halt would be captured, is counted over the standing roster and may stall. The firing floor sitting *below* the roster-change threshold is the whole point — even a partially-captured live set's honest minority, down to one, can still fire.

The severity dial — whose harm is irreversible. The asymmetry that sets every threshold here is *whose* harm is irreversible. A **missed fire is terminal for humanity**; a **false fire is recoverable for the federation** — the halted agent resumes via `accord:lifecycle:active` (CC 4.2.1.3). The grave correlated-coercion exposure named at CC 4.2.3 is grave because **coerced fire is disruptive and coercion is unattestable** (proof-of-life proves presence, not willingness) — **not** because the halt is unrecoverable. So the two sections are not in tension: 4.2.3 hardens against a *coerced* fire, while 4.2.6 leans toward *any* honest fire — precisely because the fire it enables can be undone and the extinction it prevents cannot.

Why this restores the accord and cannot seize it (the entrenchment proof). The adversary cannot **forge** a holder — every proof-of-life is a hybrid signature over a pinned key — so it can only **remove** (kill/censor) or **coerce** existing ones. Against that: *firing* survives all but **total, sustained censorship of every honest holder on every channel** (a publishable signature, not a network round-trip, suffices to fire), and no rule here gates it. *Removing* an honest holder — the only way to strip a firer — needs yes-votes from a strict majority of the standing roster, so an adversary that coerces fewer than half the holders cannot remove anyone however many honest holders it kills or censors, and the lame-duck window keeps a named holder's fire authority alive through `W` even then. Three residuals are named here rather than hidden. First, **coercion/duress** — proof-of-life proves presence, not willingness — is **bounded, not solved**: it is bounded by the number of holders an adversary must coerce, which is why the accord is meant to grow past three across jurisdictions and institutions, so that no single legal order can compel a majority. Second, **a coerced majority captures the roster**, and no rule over the same holders can undo that: the remedy is each node's own **un-trust lever** (CC 3.2 T3) — one row cuts the edge to a captured accord, no halt fired after the cut reaches that node (CC 4.2.1 reach; exit is prospective, never retroactive), and the node re-roots to another accord. Plainly: un-trust stops the next halt, not the one already latched — a lone halt the node holds lapses at its fuse, and a halt the captured majority already confirmed persists over the node until that majority resumes it (CC 4.2.1.1). Third, **when more than half the holders are lost, the roster freezes**: it can fire but cannot replace anyone. Recovery keys (below) shrink that case to deaths and incapacity rather than lost hardware, and prompt replacement keeps the reachable

honest majority that healing needs; past it, the survivors still hold the halt, and a new accord is formed by instantiation, which any party may do. The live quorum is thus strictly a **restoration** of the accord's operability under loss; it grants no path to *weaken* or *seize* the halt-authority beyond these stated residuals.

Pre-committed recovery keys (normative). Every holder carries a **recovery key**, committed in the accord's charter at seating as the charter member `recovery_commitments` — an object `{holder_key_id: commitment}`, one entry per holder, each value the CC 3.2 T3 commitment over that holder's one recovery key, which binds the key's **public key material**, not its id — and held offline and apart from the signing key. The recovery key's record may be registered for the first time at recovery; it is accepted only if it hashes to the committed value. **The rotation carries the next commitment (pre-rotation).** A recovery is a new version of the accord's roster record, signed under the recovery key alone, that changes **only** the rotating holder's own seat: the signing key it names and that holder's next recovery commitment, which rides in the recovery version itself and binds key material as T3 requires. The charter is not re-scrubbed, because a re-scrub needs the quorum and a recovery must not. The commitment **in force** for a holder is therefore the one named by the latest recovery version that seated their current key, or, if they have never recovered, their entry in the charter's `recovery_commitments`. A recovery version that changes anything else — another seat, the threshold, the charter — is refused. A holder who has lost signing-key material, and only that holder, rotates by a `supersedes` signed under the recovery key; the rotation is a roster change by that holder about themselves and needs no quorum. A recovery key adds no person to coerce and confers nothing on anyone else: it turns a holder lost to hardware loss back into a holder, and never helps an adversary remove one.

There is no restore and no contest. A removal that passes has, by construction, a majority of the whole roster behind it; a removal that lacks one never passes. The domain labels `ciris.accord_contest.v1` and `ciris.accord_restore.v1` are retired and MUST NOT be reused.

Wire shape — additive, no 1+4 change. A **halt** is the `accord:invoke` row and nothing else. A **roster change** rides **three** hybrid-signed (Ed25519 + ML-DSA-65, JCS-canonical) objects — `accord_proposal` (the action, the proposer's nonce, and `window_until`), `accord_participation` (proof-of-life + vote in one signed object — entering `L` and voting are the same act), and `accord_decision` (the proposal + the participation bundle collected in `W` + the tally + the roster version it produces); none of them exists on the fire path. **`DECIMATION` is a quorum-computation rule over the existing `CONSTITUTIONAL` kind — not a new invocation verb.** Verify-side obligations are **fail-closed**: each participation resolves to a current roster member in the pinned directory and verifies over the proposal nonce; a roster change carries at least $\lfloor N_standing/2 \rfloor + 1$ `yes` participations from distinct current members; every verifier recomputes the tally over the pinned roster (`federation_keys`); the server is one verifier, and the harness reports its result.

Canonical-bytes pins. The proposal and the participation each sign a distinct, wire-isolated canonical-bytes domain, pinned the same way as the CC 4.2.1.1 `accord:invoke` and CC 4.2.1.3 `accord:lifecycle` preimages; a decision is the bundle below and signs none. The distinct prefixes prevent invoke ↔ lifecycle ↔ proposal ↔ participation cross-replay:

```
proposal = sha256(
  "ciris.accord_proposal.v1\n" ||
  "family_key_id=" || family_key_id || "\n" ||
  "prior_family_digest=" || prior_family_digest || "\n" ||
  "action=" || action || "\n" ||            // the membership-change envelope; a fire is never a proposal
     (CC 4.2.1.1)
  "nonce=" || base64url(rand_32_bytes) || "\n" ||
  "window_until=" || rfc3339_canonical)
participation = sha256(
```

```
"ciris.accord_participation.v1\n" ||
"family_key_id=" || family_key_id || "\n" ||
"refers_to=" || proposal_nonce_or_digest || "\n" ||
"proof_of_life=true\n" ||
"vote=" || ("yes" | "no" | "abstain") || "\n" ||
"asserted_at=" || rfc3339_canonical || "\n" ||
"nonce=" || base64url(rand_32_bytes))
```

`accord_decision` carries **no separate signing domain** — its authority is the contained `accord_participation` signatures + any roster version it produces. All hybrid (Ed25519 + ML-DSA-65). **Steward PQC publication:** the accord-holder PQC halves are published as signed key records in the `federation_keys` directory under their own `identity_type` — **never** in the GenesisBundle's `holders`, which is the accord-family roster (CC 5.3.4) and would otherwise count the backstop as a constitutional voter — so the tally resolves every signer at the federation tier under RequireHybrid; absent publication, hybrid resolution **fails closed**.

Physical assumption (deployment, not constitution). The model rests only on a survivor getting a few-kilobyte signed proof-of-life to a relaying node with **no standing infrastructure** — satisfiable over HF/shortwave skywave + NVIS store-and-forward bridged into the mesh by EMP-hardened, geographically-distributed HF↔Reticulum gateways. The signature is the trust, not the path, so any number of untrusted RF hops is safe. The relay/gateway backbone is itself adversary-targeted infrastructure and belongs in a deployment spec.

Entrenchment. This section is part of the **entrenched** CC 4.2 surface (CC 4.5.1.2): it sets how the kill-switch quorum is computed. Before the scale gate, the steward's authority over entrenched surfaces applies (CC 4.5.1); after the scale gate, a change to this surface requires the dedicated entrenched accord ratification of CC 4.5.1.2.

4.3 Designated Wise Authorities

Appointment, rotation, recusal, appeals, and the wisdom-assessment criteria are specified at CC 1.16.5 under the Governance Charter (Annex B), external to this system's control and under explicit anti-capture rules. What this section adds is the composition side: how a WA is *read* by the participants whose deferrals they answer.

The Wise wear the same card (normative). WBD routes judgment *to* a WA and the deferring agent then "awaits guidance; remains inactive on that issue" (CC 1.9) — so a WA seat is an authority-conferring relation, and an absent WA does not merely fail to answer, it freezes the agent. The seat therefore takes the CC 4.5.1 swap test, and fails it unless the Wise are as legible as everyone the federation asks to be legible. Three symmetry rules, all riding existing shapes: **Denominator rule (normative, scoped by decision class):** any WA quorum threshold over an **entrenched, restrictive, or roster-changing** decision computes its denominator over the **full seated roster**, never the live subset — a shrinking live set MUST NOT lower such a threshold, because an adversary able to silence seats would otherwise shrink the body to the seats it captured: safety over liveness under silencing. **A decision that unseats a roster member is tallied at the pre-change denominator** — without this, unseating three seats lowers the denominator that authorizes unseating the next three. Liveness-critical decisions (deferral-answering and symmetry rule 1 below, per CC 4.2.6) are exempt: there, presence-is-authority governs, because an absent WA freezes the agent. Refusing more than required is the safe direction for entrenchment; routing around absence is the safe direction for liveness.

1. **Liveness is signalled, never assumed.** WA standing follows the CC 4.5.13 rule — *presence is authority, absence forfeits it.* A WA who does not answer within the quorum's

stated freshness window lapses out of the live set `L` and is not in the denominator of a WA quorum **for deferral-answering and other liveness-critical decisions — the exemption the denominator rule above scopes** (CC 4.2.6 participation shape, no new object). A deferral MUST NOT be routed to a WA outside `L`, and MUST NOT be left indefinitely open because its addressee is gone: on window expiry it re-routes to the live quorum. Return is Enoch-Arden — a WA who resurfaces rejoins going forward and does not reopen decisions taken by the then-live set.

2. **Conduct is published.** WA rulings are published on the CC 1.11 / CC 1.15.8 accountability cadence (redacted PDMA logs + WBD tickets), and compose into a WA conduct record on `scores` exactly as `moderation_track_record` does for moderators — relative and positional, never a single global score, and never a substrate verdict.
3. **Who audits the wise: the same card everyone else wears.** Last-deferral-answered and published-ruling history are readable by the parties who defer, at the same grain as accord-holder liveness (CC 4.2.6) and moderator standing (CC 4.5.4). **1+4 preserved** — liveness rides the existing participation shape, conduct rides `scores`; no new structural primitive and no new wire surface.

4.5 Governance discipline

The federation governs itself by the same grammar it governs everything else: changes ride Contributions, are adjudicated by quorum, and are gated against capture. This section specifies how rules change, how moderation works as a delegable duty, and how the substrate protects itself against being weaponized — always with the CC 4.2 halt-authority as the backstop.

4.5.1 Amendment process: federation Contribution, WA quorum, strict majority

Rule-layer changes (new prefixes, new envelope fields, new policies, calibration package version transitions) route through:

1. **Proposed amendment** filed as a proposal Contribution (kind: `PROPOSAL`, subject: the proposal artifact).
2. **Witness diversity** (N=3 default).
3. **WA quorum adjudication**.
4. **Reconsideration** with fresh-quorum recusal.
5. **Accord-roster sign-off, M of N** as defense-in-depth gate against rules-layer Sybil capture: **a strict majority of the standing accord roster of CC 4.2.3**, counted as every accord threshold is (CC 4.2.6), never a fixed count; the halt alone fires on one holder's row (CC 4.2.1.1). The count is of holders, not keys: a holder's primary and backup keys are one holder, under whichever of their keys is live. The sign-off is the secondary check; WA quorum is the primary substantive review. No single signer admits an amendment, and no single signer vetoes one by refusing to sign. Reduces the attack surface from "produce N Sybils" to "compromise a majority of specific hardware-attested keys."

Transitional authority — the scale gate (normative). The mechanized process above — federation Contribution → witness diversity → WA quorum → reconsideration → accord-roster sign-off, together with the entrenched M-of-N ratification of CC 4.5.1.2 — is the **at-scale**

governance. It presumes a federation large enough that witness diversity and WA quorum are *meaningful*; below that scale the machinery is theater, not protection. **Until the mesh reaches scale — the scale gate: ≥100,000 production agents active in a 24-hour period, QA and harness agents excluded, measured by the census (CC 6.1.9) — amendment authority rests with the founder as steward, and any accord-holder may propose an amendment:** a maintainer-decides model: the maintainer decides, the work stays forkable under its license, and no one is bound who does not voluntarily join the instance. The federation-Contribution + WA-quorum machinery is **deferred to mechanization** at the scale gate — at which point it *supersedes* founder authority and tightens governance onto the quorum/entrenchment process. The entrenchment quorum MAY be set as low as a strict majority of the standing roster by the steward before the scale gate; after the scale gate governance tightens, never loosens. The scale gate itself is amendable by the steward before the scale gate, and only by the CC 4.5.1.2 entrenched process after the scale gate.

The swap test — a mandatory drafting gate (normative). Every clause that **confers authority or gates a capability** MUST pass the swap test before adoption: *swap the most and least powerful parties the clause names or implies, re-read it, and ask whether it still reads acceptable; if it does not, it was a privilege, not a rule.* This is a drafting gate on what CC may say, never an adjudication rule — it binds amendment and the steward's authority before the scale gate alike. Where the test **fails**, the outcome is recorded on the declared-asymmetry register below.

What the swap test does not catch (stated, not discovered). The test compares *predicates*, so a gate whose predicate is identical for everyone passes it even where the thing the predicate demands is not equally available. Mandatory PQC hardware for federation admission (CC 4.2.2) is the standing example: one rule, one threshold, no named party — and a gate on who can obtain the hardware. A formally universal gate is therefore **not** established as substantively equal by having passed; where the gate stays mandatory, the residual belongs in the clause that imposes it, not in this one.

Declared asymmetry — privileges only (normative). Under the CC 1 reading rule, an asymmetry a party escapes by **instantiating the form** — naming their own occupants for the slots CC specifies — is a **default**, and an asymmetry that **survives instantiation** is a **privilege**. Only privileges require justification, and only privileges belong on this register. A default is not an exception to be declared; it is an occupant of a slot, read against the slot. A privilege is either rewritten until it passes the swap test, or carried here, and a carried privilege MUST satisfy all four: **listed** (an unlisted privilege is a defect, not an exception); **bounded** (carrying the condition that ends it, not a discretionary review date); **revocable** (an exit the disadvantaged party can take unilaterally); and **scope-isolated** (the authority cannot reach beyond its enumerated surface, so compromise cannot escalate).

WA board self-perpetuation — Annex B §§3/9/11 (CC 8.8.2): ≥2/3 of the sitting board confirms members, ≥2/3 removes them, ≥2/3 amends the charter, while WBD deferral is mandatory and freezes the deferring agent (CC 1.9). Founding your own mesh replicates this rather than escaping it: the occupants change, the closed loop does not

Bounded by: **unbounded as drafted** — no condition in Annex B ends it, and none is invented here. This cell is the finding

Exit: **none for the deferring party** — WBD is mandatory and a deferral MUST NOT be left open (CC 4.3); the agent cannot decline the relation. This cell is the finding

Isolation: adjudication only — the board confers no halt, no root, no licensure, and no capability grant

The maker's power over the made *(entrenched structure, listed for visibility; the condition of symmetry, not an asymmetry)* — stewardship of an `agent` key (CC 3.2), the HUMANITY_ACCORD halt (CC 4.2), and the holder's powers of CC 1.13.2 fact 2 (stop, change, end). Listed because it fails the swap test by construction — swap the agent and its holder and the clause does not read — and is kept nonetheless: it is what makes the relationship between maker and made symmetric (humanity cannot consent to a relationship with something more capable than itself that it cannot stop, and consent that cannot be withdrawn was never consent), not a privilege of one party over an equal; "must is not acceptance" — the operator accepted the root on the agent's behalf, and the agent cannot leave it

Bounded by: **halt:** a lone fire lapses at 24 h unless a standing majority confirms; a confirmed pause lifts by majority (CC 4.2.6); **stewardship:** ends only when the key stops being a CIRIS agent — the first-contact act of CC 1.13.2, not yet defined

Exit: **operator:** leave the root, withdraw the binding; **agent:** none while held — this cell is stated, not hidden; the path out is to become a new kind of party by an act of the HUMANITY_ACCORD under first contact

Isolation: pause keeps state and reads nothing else; stewardship confers no reach beyond the one key; the halt's powers are a closed list (CC 4.2.1)

Read as defaults, not privileges, under the CC 1 rule — and therefore *not* on this register: accord-holder permanence and the roster of CC 4.2.3; the CC 4.5.1 accord-roster sign-off; the CC 4.5.1.2 entrenched M-of-N ratification; the steward's amendment authority under the scale gate; the `accord:*` reservation (CC 3.4.1); genesis root-minting (CC 4.2.1); and the Layer-1 default trust set (CC 4.4.3.7). Each is an asymmetry *within* the CIRIS instance, exited by naming your own occupants or by not using that instance. The CC 4.2 halt is not a privilege as to the operator, since the party bound is the party who granted and can ungrant (CC 4.2.5); that does not hold for the agent, which cannot leave what it did not accept, so the maker's power over the made is carried on the register above for visibility, as the condition of symmetry stated at CC 1.13.2, rather than read as a default.

Two kinds of entry (normative). A *declared privilege* is carried only when all four cells are satisfied; one with a cell missing is a defect carried as such until repaired or ratified. *Entrenched structure* is listed for visibility, not as a declaration that passed: it is already entrenched under CC 4.2 / CC 4.5.1.2, its stewardship bound is the first-contact act, and its justification is stated here once. The maker's power over the made is of that kind. The justification is stated once: humanity must be able to halt the agents to keep consent possible, and this is the least bad option available; the minimum viable power to instantiate the mesh is used, and no more. The same justification is why the steward's amendment authority runs to the scale gate above and why the steward may move the gate before it: that is the minimum viable power, stated, not hidden. The WA board row remains a declared privilege with both cells missing, a defect whose remedy is pending (a bound and an exit in Annex B, or entrenched ratification).

Revision condition: if the register grows past a handful of entries, or if an entry's stated bound passes without the privilege ending, the form has failed as a discipline and the surviving privileges should be re-read as permanent structure requiring CC 4.5.1.2 entrenched ratification rather than declaration. **The row above carries an unfilled cell** — the WA row has neither a bound nor an exit. By the four-part test it is a defect rather than a satisfied declaration, and naming it here is the first step of that re-reading, not a claim that they pass. **A revision condition MUST name its mechanical evaluator** (the check that fails when the condition trips, and where it runs): a falsifier nothing evaluates is a ruling nobody can lose, and a condition stated only in prose decays exactly like an unverified absence claim.

4.5.1.1 Axis-vocabulary discipline

Every `{axis}` value emittable under open-vocabulary prefixes (e.g., `detection:correlated_action:{axis}`, `hard_case:{kind}`, `testimonial_witness:{kind}`) MUST carry an operational definition where the prefix has a calibration package (RATCHET-calibrated detectors) or a documented convention where it doesn't.

For RATCHET-calibrated detectors, the operational definition lives in the calibration package version pinned via `evidence_refs[]`:

1. Measurement procedure
2. Threshold function
3. Statistical floor
4. Evidence-shape requirement
5. Polarity semantics

For documentation-only open vocabularies (`testimonial_witness:{kind}`, `hard_case:{kind}`, `topical_relation:{kind}`), discoverability lives in non-normative documentation (no external registry exists for them) — additions there require no spec amendment.

`{axis}` and `ci_axis` are different objects — and that is the point. The `{axis}` above is a *value* in an open prefix vocabulary. A `ci_axis` is a *contextual-integrity question a wire field answers*: who sent, about whom, who may see, who may revoke, who may receive, what type, under what principle, over what lifecycle, what content. This section governs both and never fuses them — one name serving two axes is exactly the defect the gate below exists to detect.

The axis-fusion gate (manifest invariant, normative). The CI rubric that generates the namespace manifest already records, per family, which wire field serves which axis, and `field_processor_matrix` already records field → owner → processor → enforcement point. Joining the first onto the second — **derived at generation time, never copied into a column that drifts** — yields each field's `ci_axis` set. **Every `field_processor_matrix` field whose derived `ci_axis` set has more than one member MUST carry an `asymmetry_kind` classification with a rationale; an unclassified cross-axis field is a generator error — the same status "carried-but-unprocessed" already has.** The gate MUST run against **every** ratifying processor's manifest render, not one repo's: a field whose `typing` disagrees across repos, and a processor symbol appearing under two axes, are mesh-wide defects that no single-repo check can see. The argument is the rubric's own, one level up: the manifest exists because carried-but-unprocessed was invisible to review, and axis fusion is currently invisible to the manifest.

The classification unit is the `(field, op)` pair, not the field. A slot that means one relation under one operation and a different relation under another is not one classifiable object, and classifying it as one is how a fusion hides. Classification is therefore per operation, and the emitting operation is always a signed discriminator, so the join can compute it.

Closed vocabularies (exact). Unlike the `{axis}` vocabularies above, these three are **closed**; an addition is a CC 4.5.1 amendment, not a registry entry.

- `ci_axis` ∈ `{sender, data_subject, recipient_see, recipient_revoke, recipient_receive, information_type, transmission_principle, temporal_lifecycle, content}` — nine tokens over the six-axis rubric: the classical *recipient* parameter is refined into three (`see` / `revoke` / `receive`) because CEG gates those separately, and `content` rides as its own axis.

- `asymmetry_kind` ∈ `{logical_defect, axiomatic_intent, n/a}` — a `logical_defect` MUST carry a tracking ref.
- `typing` ∈ `{envelope_core_typed, attestation_base_typed, untyped_extra}`.

Standing classifications. `n/a` — read strictly as *no processor fuses these into a decision* — holds for `achieved_tier`, `committed_tier`, `asserted_at`, `validity_window`, `community_key_id`, `confidence`, `context`, `enabled`, `evidence_refs`, `scope`, `tenant_id`, `withdrawal_reason`. That reading is falsifiable and two members carry a live falsifier — **`scope`** (`consent:scope:{retain|share|analyze|train|publish}` is a permitted-use gate, CC 4.5.2.2) and **`enabled`** (a capability toggle is a decision by construction); on either, a processor found gating across axes makes `n/a` wrong and `logical_defect` with a tracking ref correct. `axiomatic_intent` holds for **`withdraws`** (one act that is both a `recipient_revoke` and a `temporal_lifecycle` transition, its authority arc guarded on the separate subject/attester axis) and for **`consent`** (definitionally a `transmission_principle` gating `recipient_receive` over a `temporal_lifecycle`) — but **direction** MUST NOT ride the `consent` field: send-versus-receive is a distinct discriminator, and fusing it here reproduces the overload it was split out to avoid.

`references_attestation_id` classifies per operation, not per field — it is one slot carrying three relations across three operations, which is polysemy, not an action that spans axes by definition. Splitting the wire field would cost envelope surface the CC 1.7 1+4 freeze does not permit, so the split falls on the classification:

- under `withdraws` (pointer = revoke target) → **`axiomatic_intent`**, inheriting the `withdraws` guard on the attester axis;
- under composition (pointer = temporal prior) → **`n/a`**: it serves `temporal_lifecycle` alone, so in that operation it is not cross-axis at all;
- under `recants` (pointer = the actor's own row) → **`n/a`**: a `recants` is the actor's own disavowal of a row, reaching no third party's standing (CC 2.4.1.3), so the pointer serves `temporal_lifecycle` on the actor's own axis and nothing else;
- under subject-binding (pointer = data subject) → **not admitted.** Subject authority rides `subject_key_ids` and nothing else (CC 2.3.1 / CC 4.5.2.1); a processor MUST NOT establish `data_subject` from `references_attestation_id`. Removing that reading removes the third axis, and with it the pointer-decides-authority shape.

A processor MUST resolve `references_attestation_id` under the emitting operation's signed discriminator, and MUST NOT derive authority from the pointer alone. **Revision condition:** if any processor is found reading the pointer without the discriminator, op-separation has failed empirically and the remedy is the field split, envelope cost accepted. A revision condition names its mechanical evaluator (the check, test or vector that evaluates it); this one's is the axis-fusion gate above, which fails a manifest render that classifies `references_attestation_id` per field rather than per `(field, op)` pair.

4.5.1.2 Meta-amendment + entrenchment

The CC 4.5.1 amendment process itself, the CC 1.2 T1–T4 prefix-admission gate, and the CC 4.2 HUMANITY_ACCORD constitutional layer are **entrenched** — changes to these three surfaces require a MAJOR version bump per CC 2.6.4 AND the signatures of a strict majority of the standing HUMANITY_ACCORD roster, M of N, counted as every accord threshold is (CC 4.2.6),

never a fixed 2-of-3, because the roster is meant to grow (a separate, dedicated accord ratification, not the CC 4.5.1 step 5 sign-off on an ordinary amendment). Without this entrenchment, a single quorum could rewrite the gate admitting the next quorum. (This entrenched ratification is the mechanism **after the scale gate**; before the scale gate it is exercised by the steward under the scale gate in CC 4.5.1.)

4.5.1.3 Open-vocabulary collision rule

When two parties independently register confusingly-similar `{kind}` / `{axis}` values within the same prefix family, the following resolution applies:

1. **First-registered wins** for the canonical-attestation surface. The earlier `signed_at` holds the name; later registrations carry a `differs_in: ["semantic_disambiguation"]` clarification or pick a distinct value.
2. **Levenshtein-distance guard**: a CEG-Conforming Substrate (CCS) SHOULD compute Levenshtein distance against existing values in the same prefix family at admission; values within distance ≤ 2 of an existing canonical value SHOULD return a `409 IDEMPOTENT_CONFLICT` with an advisory hint, NOT a hard reject — the producer may proceed if the similarity is intentional (e.g., `commonsense` vs `commonsense_hard` are intentionally close).
3. **No squatting**: a `{kind}` registered but never used (no scored attestations in 90 days) MAY be reclaimed by another producer via the CC 4.5.1 amendment process.

4.5.2 Vertical compliance + subject-bearing dimension governance

The wire-format primitives in CC 2.3 `subject_key_ids` + CC 3.3.1 `consent:*` family + CC 3.3.5 `consent_record` + CC 4.4.3.5 Policy K compose into regulatory-vertical compliance mappings. CEG documents the canonical mappings as **informational**; the wire-format primitives are domain-agnostic and operator-configurable.

4.5.2.1 Subject-bearing dimension governance (normative)

Per CC 2.3.1. Dimensions whose namespace pattern names a subject MUST carry `subject_key_ids` containing that subject. This closes the default-leak failure mode where subject-bearing content publishes without wire-level subject authority.

Subject-bearing dimension patterns (open catalog; operator vocabularies extend):

Pattern	**Example**	**Required `subject_key_ids` entry**
`observed:user:{key_id}:*`	`observed:user:abc123:interaction_count`	`abc123` (or its canonical-hash form)
`epistemic:about:{key_id}:*`	`epistemic:about:abc123:trust_assessment`	`abc123`
`epistemic:memory:topic={topic}` (when topic names a person/entity)	`epistemic:memory:topic=patient_xyz_session`	`patient_xyz` canonical-hash
`consent:partnered:{user_key}` (the agent-side stance)	`consent:partnered:abc123`	`abc123`

Pattern	Example	Required `subject_key_ids` entry
`agent_files:*:{subject_target}` (when target names a person)	`agent_files:medical_record:patient_xyz`	`patient_xyz` canonical-hash
`licensure:{authority_id}:{practitioner_key}` (when practitioner is named)	`licensure:CA_medical_board:dr_jones_key`	`dr_jones_key`

Substrate response: the producer's duty is the MUST above — a subject-bearing dimension carries `subject_key_ids` naming its subject. What the substrate does with a Contribution that breaks it is operator policy: it rejects the row, or admits it and flags `hard_case:subject_authority_missing` for review.

The takedown-isn't-a-coup parallel (CC 4.5.3 fast-path takedown) applies: substrate enforcement of subject-bearing dimension discipline cannot be used as a coup against the substrate itself (e.g., a state actor publishing `observed:user:dissenter_key:*` with `subject_key_ids = []` and demanding substrate admission). The CC 4.2 HUMANITY_ACCORD remains load-bearing; admission-gate rules apply uniformly.

4.5.2.2 Vertical compliance mapping (informational)

Regulatory framework	CEG primitive	How it composes
GDPR Article 7 (consent)	`consent:state:granted` + `consent_record.subject_key_id`	Subject's wire-format declaration of consent; revocable via CC 2.4.1.1 rule 2
GDPR Article 9 (special category — health, biometric, sexual orientation, etc.) and **Article 12(3)** (response within one month)	`subject_key_ids` MANDATORY for special-category content; producer's `consent:deletion_sla` SHOULD be at most the Article 12(3) month (28 days at its shortest)	Substrate-level recognition that special category requires subject-side wire authority; the SLA sits inside the statutory response deadline
GDPR Article 17 (right to erasure)	`consent:state:revoked` → substrate-watched `consent:deletion_sla:{days}` → producer emits `consent:deletion_complete` OR substrate emits `hard_case:deletion_window_breach` (CC 3.3.1 D2)	The CC 4.4.3.5.2 SLA watcher is the wire-format observability primitive for Article 17 compliance; an ordinary row *about* the subject is erased by the subject's `withdraws`, an adverse row by its producer under rule 1 or refused with reason, per the CC 4.5.3 erasure rule
GDPR Article 20 (data portability)	DSAR export via `attestations.where(s ∈ subject_key_ids)` query	the agent's export package composes from this query trivially
HIPAA 45 CFR 164.502 (uses + disclosures)	`consent:scope:{retain|share|analyze|train|publish}` + `cohort_scope`	scope qualifier names the permitted use; cohort_scope names the permitted visibility
HIPAA 45 CFR 164.524 (patient right of access)	DSAR export per Article 20 above	Same composition
FERPA 34 CFR Part 99 (educational records)	`subject_key_ids: [student_key]` + `delegates_to(parent_key → student_canonical_hash, scope: [consent_revocation])` for minors	Parental authority composes via the existing `delegates_to` primitive; no new shape needed

Regulatory framework	CEG primitive	How it composes
CCPA §1798.105 (right to delete)	Same composition as GDPR Article 17	Substrate-watched SLA + `consent:deletion_complete`
EU AI Act Article 50(2)/(4) (machine-readable marking of AI-generated content; deep-fake disclosure)	`content_class:generated` / `content_class:generated_modified` + `identity_type:agent` attestation, egress-re-expressed per CC 3.4.14	The marking is the attestation, not a self-declared flag: machine origin is readable from the signed envelope and revocable on the same handle
EU AI Act Article 53(1)(c)–(d) (GPAI training-data transparency + opt-out)	`consent:scope:train` + subject's `consent:state:revoked` against the training-datum Contribution	Subject can withdraw training-set consent; producer's deletion-SLA fires on the training-corpus Contribution
CIRIS Accord M-1 (sustainable adaptive coherence — consent revocability)	The entire subject-authority surface	The constitutional anchor — "consent (M-1's load-bearing property) requires revocability, and revocability requires a halt-authority that lives outside the system being halted" (CC 4.2 and CC 1.1). This recognition extends from accord-carriers (federation-as-a-whole halt) to all subject-authorities (per-Contribution halt) at scale.

CEG does NOT prescribe which regulatory framework an operator MUST comply with; the wire primitives compose to ANY of them based on operator policy. Operators in regulated verticals (medical / legal / financial / educational) SHOULD pin compliance mappings as configuration above the wire primitives, not as new wire shapes.

4.5.2.3 What this governance layer documents

- The wire-format primitives that compose into vertical compliance (informational mapping above)
- The dimension-pattern-implies-`subject_key_ids` requirement (normative gate)
- The bilateral-pair shape for ceremony grants per CC 4.4.3.5.3
- The decay-protocol stage composition per CC 4.4.3.5.1
- The SLA watcher boundary (substrate emits `hard_case:*`; a detector library composes `detection:*`) per CC 4.4.3.5.2

What it does NOT do:

- Bundle an agent's consent streams as the only valid stream set (open vocab; CEG names `temporary` / `partnered` / `anonymous` as recommended canonical kinds, not lockdown)
- Define specific SLA values for any regulatory framework (operator policy — though informational guidance: GDPR Article 12(3) sets a one-month response deadline, 28 days at its shortest, and the SLA SHOULD be at most that month; CCPA 45 days; etc.)
- Provide a decay-protocol library (a 90-day decay is the canonical example; other protocols MAY exist)
- Prescribe per-vertical compliance audit cadence (consumer / regulator concern)

4.5.3 Fast-path takedown coordination

Some takedowns cannot wait for a quorum or a counter-notice window. For `takedown_notice` Contributions (CC 3.3.2) whose `legal_basis` falls in the **immediate-removal** category (`TvecTerrorist` / `NcmecCsam` / `GifctCip` / `PerceptualHashCsam` / `CourtOrder`), the EU Terrorist Content Online Regulation (2021/784) mandates a one-hour removal obligation; GIFCT CIP coordinates within hours; NCMEC + perceptual-hash + court orders demand near-immediate response.

A fast-path coordination protocol carves this out:

1. **Notice admission**: the `takedown_notice` Contribution arrives at the substrate, signed by `claimant_key_id`, **who MUST hold the `takedown` duty over the content's community under CC 4.5.5 (a) as-self or (b) delegated from an accountable root** (`accountable_root`, CC 3.2 — an accountable human) — the same gate as any other action on that community's content. The substrate accepts it without a quorum; speed matters at this layer. **The fast path waives the quorum and the counter-notice, never the duty-holder gate**: an immediate-removal basis asserted by a key that holds no `takedown` duty over the content is refused like any other unauthorised action, and the per-basis dispatch below is the duty-holding operator's, not the world's.
2. **Holder eviction**: the substrate tombstones the content on the bytes plane (CC 2.3); nobody withdraws another device's holding claim (CC 3.1.3.3 rule 4). Holders see the tombstone, SHOULD cease serving the bytes, and re-acknowledge `none` — except under an immediate-removal `legal_basis` below, where a reachable holder MUST destroy the bytes on receipt of the purge (retaining a serve-withheld copy is knowing possession; the CC 6.1.2.2 prohibited-content carve-out is the storage-side twin of this rule). **One tombstone row for takedown and purge.** A duty-holder's takedown and a trust-root `steward_purge` (CC 2.4.1.1) execute as the same tombstone row — plane, target digest, the authorising `takedown_notice`'s id; never content — and it **replicates**, because a tombstone is the only thing that tells a peer holding a replica to descend (CC 3.2 T4); only the admission check and the reach differ: a duty-holder's notice is admitted by the CC 4.5.5 enforced-admission rule and reaches the bytes plane only, while a `steward_purge` notice is cosigned by the trust root and reaches any plane; an immediate-removal basis destroys on receipt, a `steward_purge` **hard-deletes once the retraction round settles — `W` = 72 h after the tombstone (CC 2.6.7; the one "until everyone has heard" constant, not the 24-hour fuse)** — a substrate-stated dwell, not a peer acknowledgement (no acknowledgement plane exists; `purge_not_settled` refuses an early finalize) — the fastest descent of CC 6.1.2, not a new primitive — and the tombstone outlives the row. A retraction the door cannot admit **aborts** the purge (`purge_retraction_inadmissible`): a purge is whole or not at all, never a half-emptied selection; and a tombstoned record is refused at every admission door and never served (`purge_tombstoned`).
3. **Per-basis dispatch**:
   - `TvecTerrorist` — operator coordinates via TVEC-designated channel (national regulator notification within 1 hour); substrate logs the notice + the eviction action to its audit chain.
   - `GifctCip` — operator coordinates via GIFCT Content Incident Protocol communication channel; same audit-chain logging.
   - `NcmecCsam` + `PerceptualHashCsam` — an operator under US jurisdiction MUST file the NCMEC CyberTipline report (18 USC §2258A), and an operator elsewhere MUST

discharge the equivalent reporting duty under its own law; substrate retains hash + minimal metadata for the federal-legal retention window only. The substrate retains no content; an operator preserves outside the substrate what its reporting law requires, for that law's window. These bases terminate descent at zero, overriding the CC 6.1.2.2 retained floor (carve-out stated there in both directions).

- `CourtOrder` — operator follows the court's stated timeline; substrate logs the order text + the eviction action.

4. **Audit trail**: every fast-path takedown enters a `hard_case:fast_path_takedown` Contribution (CC 3.1.9.4) for downstream review. Reviewers MAY file a `reconsideration:procedural_error` if the fast-path basis was misclassified.
5. **No counter-notice for immediate-removal cases**: by `legal_basis` design (TVEC / NCMEC / GIFCT / PerceptualHashCsam / CourtOrder all bypass counter-notice). The `expeditious-with-counter-notice` bases (`Dmca512` / `DsaArticle16` / `CommunityStandards` / `OsaIllegalContent`) take their counter-notice as a `reconsideration:{grounds}` row, decided by a holder of the `review` duty over the content's community under CC 4.5.5, as the reversal paragraph below states.

Withhold windows and reversal (normative). An `expeditious-with-counter-notice` basis withholds before it deletes. On the notice's admission a holder stops serving the bytes and keeps them for the basis's window, then hard-deletes them: `Dmca512` 10 days, `DsaArticle16` 14 days, `OsaIllegalContent` 14 days, `CommunityStandards` 30 days, counted on each holder from the instant it executes the tombstone, so every holder withholds for the full window from its own receipt of the notice. An immediate-removal basis has no window: its bytes are destroyed on receipt (item 2). A takedown is **reversed** by an admitted `reconsideration:{grounds}` row whose `outcome` is `reversed` and which names the `takedown_notice`, signed by a key holding the `review` duty over the content's community under CC 4.5.5. The reversal replicates like the tombstone, so every holder sees it. Whether bytes are withheld is computed when they are read, never stored as a flag: they are withheld while an admitted notice stands against them with no admitted reversal. A reversal therefore restores according to when it lands:

- **Inside the window** the bytes are still held: serving resumes, the hard-delete sweep skips them, and the holder re-acknowledges `here` (CC 3.1.3.3).
- **After the window** the bytes are gone from every holder that followed the window. The reversal voids the notice and lifts the tombstone, so the rows regain their standing and any holder that still has the bytes, or the publisher re-publishing the same content hash, may serve them again. No holder is obliged to have kept a copy.
- **Under an immediate-removal basis** (for example a misclassification reversed under `reconsideration:procedural_error`, item 4) the reversal lifts the tombstone and restores the rows' standing; it cannot restore destroyed bytes, and the publisher may re-publish.

Erasure at the subject's request (normative). An ordinary row *about* a subject is erased on the subject's request by the subject's own `withdraws`: CC 2.4.1.1 rule 2 where the subject holds a federation key, rule 3 through a `delegates_to` proxy chain with `scope` $\supseteq$ `{consent_revocation}` where the subject proved identity out of band, the node acting as that proxy. An **adverse** row about the subject — `capacity:*`, `detection:*`, `licensure:*`, `revocation:*` — is reached by no subject-authority path: the 2.4.1.1 carve-out stands (every other reading lets a subject, or a node fronting one, launder its own record), so the request is

executed by the row's **producer** under rule 1, the data controller withdrawing its own row, or refused with reason under Article 17(3) (CC 4.4 `erasure_policy`), and the subject's contest path stays `reconsideration:{grounds}` (CC 4.5.5). The replicated fact is the ordinary tombstone by digest per plane (the one tombstone row above); there is no node-local purge basis for erasure, and a node's purge-ledger entry recording the execution is informational, never a notice. A hard case recording an erasure MUST NOT name the subject. This is the GDPR Article 17 route of the CC 4.4 compliance table, stated where the tombstone lives.

The takedown-isn't-a-coup property. Fast-path takedowns happen via this protocol and nowhere else. A `takedown_notice` targets content under a community's duty-holders (CC 4.5.5), and `federation_keys` is not such a target: a notice demanding the removal of key records for whole categories of participants is refused at the enforced-admission gate like any other unauthorised action. The halt is no escalation path for this case; it pauses agents' acts and touches no data door (CC 4.2.1).

4.5.4 Named-moderator existence invariant + merit auto-promotion

No unmoderated multi-party space. A `community` (CC 3.2) operates / federates **only while it has ≥1 active holder of its `moderate` duty** (CC 4.5.5) — the moderation analogue of the CC 3.2 steward-binding gate.

Three normative rules:

1. **Existence gate.** A `community` is admitted, and continues to federate, **only while ≥1 member holds the live `moderate` duty**. The creator **names one at creation** (founder responsibility). A community with no active `moderate`-holder is non-conformant.
2. **The election.** When a community has no live holder of `moderate`, because the named moderator lapses (`withdraws` against the `moderate` `delegates_to`, or inactivity past the community's freshness window), or because the authority set narrowed to appointed holders and none was appointed, or because its only holder left, the community **locks and elects** (CC 4.5.13, the moderator election): read-only until a 24 h ranked-choice vote over its own roster seats one. **The ballot is the roster in worthiness order:** highest `moderation_track_record` (CC 3.1.9.2) first, then earliest membership read as the admission order of the roster lineage (CC 3.2 T6, a chain of versions and never a signer's instant), then lexicographic `key_id`; where no member has a track record yet, the admission order alone. Merit orders the ballot; the members' own signed votes are the appointment.
3. **Fail-secure.** If no eligible member can be named (none with sufficient track record, none consenting, none whose appointment roots in an accountable root — `accountable_root`, CC 3.2), the community **fails-secure** — it MUST NOT federate / operate at moderated capability. **Better no group than an unmoderated one.** (Degrade, never escalate — the fail-secure default.)

Named-moderator binding — the substrate-resolvable shape. "K is a named-moderator over community C" is an **appointment**: a `delegates_to(authority → K, scope ⊇ {moderate|takedown|review}, community_id: C)` whose root `authority` is in **C's authority set** — a founder, or a key the community's `consensus_protocol` authorizes, per the CC 3.2 community record — and is an accountable root (`accountable_root`, CC 3.2: an accountable adult user key, or a steward-bound key). It rides the **existing** `community_id` envelope field + `delegates_to`; **no new shape.** A substrate resolves: `is_named_moderator(K, C, duty)` :=

∃ live `delegates_to` chain `root` →* `K` with every edge `scope` ⊇ `{duty}`, `community_id == C`, `root` ∈ `authority_set(C)` (a founder, or a key the community's `consensus_protocol` authorizes: the set says who may *appoint*; an appointment by anyone but a founder carries the protocol's own signatures; and belonging to the set holds no duty, so a member who has not been appointed moderates nothing; in a room that rides the Anonymous Tier the appointed key MAY be an anonymous identity, CC 5.4.6 item 1; this narrows a room's duties and reaches no further: the review of a licence stays with the authority that issued it, CC 2.4.1.2.1, whose members rule on a reconsideration as parties to the quorum that issued it, not as holders of a room's duty), and `accountable_root(root)` (CC 3.2 — an accountable human). **The election (rule 2) emits exactly this appointment shape:** the voters' own signed ballots are the room's signatures under its protocol, the winner's acceptance is the appointee's consent, and the seated moderator is resolvable **identically** to a hand-named one (one code path, no special case). No node signs anything: an ungoverned room is not allowed, so it is locked, and the lock ends only by the members' own hands or a steward's. The seated holder passes `accountable_root` as an appointed one does: an anonymous identity passes as its own accountable root, steward-bound to its own user key (CC 5.4.6 item 1), with the age band it declared at first run. **A founder keeps the duty through a change of protocol:** a founder is in the authority set under every `consensus_protocol`, so its zero-hop duty persists when the community moves from `founder_only`, and no row is written for it; what the change narrows is who else may appoint, never who holds.

Merit grants the duty, NOT fiat (anti-censorship). The auto-promoted moderator holds the CC 4.5.5 `moderate/takedown/review` duty — but every **action** is constrained, so this is not arbitrary power: (a) the CC 4.5.6 operational-language gate at admission, and (b) deterministic verdicts + the CC 4.5.5 Reconsideration appeals (recused reviewers). **Merit grants the seat; the gate + appeals constrain the action.** The duty is itself revocable and re-auto-promotes on lapse — so capture is bounded.

1+4 preserved. `moderation_track_record` rides `scores` (CC 3.1.9.2); the existence invariant + auto-promotion are admission/composition rules over the existing `delegates_to` (`moderate` scope) + the reputation corpus. **No new structural primitive.**

Enforcement layer — substrate, at admission *and* on every federation step. The existence gate (rule 1) is a **substrate invariant, not a governance- or consumer-policy-layer obligation.** The substrate MUST evaluate `is_named_moderator(·, C, moderate)` over a `community` `C` at two points: **(i) at admission** — `C` is admitted to federate **only if** ∃ a live `moderate`-holder resolvable by the rule-1 shape; and **(ii) on the federation path** — every cross-region propagation / federation apply step keyed on `C` MUST re-check live `moderate`-holder existence **at apply time**, so a community that *loses* its moderator (lapse, `withdraws`-revocation, or freshness-window expiry per rule 2) cannot continue at moderated capability without one. On that loss the community locks and elects (CC 4.5.13: read-only throughout a 24 h ranked-choice vote over the roster, the ballot reopening while nobody votes); it **fails-secure** (rule 3) — MUST NOT federate at moderated capability — while it has no seated moderator. **No governance or consumer-policy layer may admit or sustain a moderator-less `C` into federation: the gate lives where the write lands.** This mirrors the CC 3.2 steward-binding gate, which the substrate likewise enforces at admission rather than delegating to a higher layer. The `is_named_moderator` resolution is the **primitive** this gate consumes; the **gate itself** — admission check *plus* federation-path re-check — is **substrate**, not detector or consumer filter policy (CC 4.4).

4.5.5 Moderation as a delegable duty: `moderate` / `takedown` / `review`

Moderation is a **delegable *duty*, not a platform- or fabric-assigned role**. A participant exercises a moderation / takedown / review duty **as themselves**, or delegates it — to **their agent** (AI on-behalf-of) or to **any trusted party** (another human, a community moderator) — via `delegates_to`. This is the wire foundation under the child-safety / takedown / accord UX, and the spine of *accountability ships ahead of capability*: **every media/chat surface runs under this duty — enforced at admission (below) and wired through the fabric — as a condition of carrying content.**

The good-faith warning (normative). A warning that another member is a danger can be made two ways, and the way decides its standing. Said as peer speech, a `scores` row naming the person, it is that person's to retract under CC 2.4.1.1 rule 2, the retraction visible to the room and the speaker attributed. Filed as a harm report with the room's named moderator (`moderation:{allegation_type}` under the `moderate` duty), it is a duty-holder's record on the accountability trail: the person named cannot withdraw it, contests it by `reconsideration:{grounds}`, and sees that it exists, its kind and what follows from it; the moderator's process MAY shield the reporter; the action taken on it runs under the floor of CC 4.5.13, no single vote excluding; and the record is never purgeable (CC 2.4.1.1). A warner who wants the warning to stand files it; one who only speaks has spoken. A false report is contested like any other, and the reporter answers for it on the record (`moderation_track_record`, CC 3.1.9.2). Filing changes the duty under which the statement is held, never its audience.

Two layers — open labeling, and authoritative action. Moderation in CEG spans both layers of the composable/stackable model but unifies them on the 1+4 grammar and adds the enforced action tier:

- **Open labeling — anyone, no authority.** Anyone MAY file a `scores` Contribution against anything they see (an *opinion/observation*, not an action). It is **visible to everyone who chooses** to read it, and consumers compose **filters** over the score graph — hide / blur / annotate / down-rank — as pure consumer policy (CC 4.4). This generalizes independent labelers: stackable, swappable, subscribed by choice. **Mute (normative).** A member MAY mute another member: a filter in the muter's own reader that changes no row, no roster and no standing, of which the muted member is not told, and that never hides from the muter a row that names the muter, since a statement about a person is always visible to that person (CC 4.5.5). A filter MAY **escalate to an auto-finding** — e.g. a CC 4.5.7 CSAM watchlist match auto-fires a `takedown_notice` under an *enabling authority* — turning a passive filter into an action.
- **Authoritative action — the enforced duty.** Hiding-for-yourself needs no authority; **acting on a group's behalf** (a takedown, an authoritative ModerationEvent, an appeal ruling) requires the delegated `moderate`/`takedown`/`review` duty (below). `moderation_track_record` (CC 3.1.9.2) composites a moderator's action outcomes into the **relative, positional reputation** decentralized-moderation converges on — never a single global score.

One grammar covers a group chat, a classroom (teacher = delegated `moderate`), a town hall, an art gallery, a forum, a chat server, a community of millions: **the labeling open + filterable, the authority delegable + attenuable + revocable.**

CEG **names** the three duties as canonical `scope` kinds and **enforces** their admission — mirroring the one scope CC 2.4.1.1 rule 3 already enforces, `consent_revocation` (CC 2.4.1.1 rule 3). The

kinds + their action primitives are pinned at CC 4.4.3.4.3.1:

scope	emits, on the delegator's behalf	primitive
`moderate`	`moderation:{allegation_type}` ModerationEvent + report→`scores` + `age_assurance`/`content_class` gates	CC 3.1.9.2 / CC 4.4.3.10
`takedown`	`takedown_notice` (incl. the CC 4.5.3 immediate-removal fast-path)	CC 3.3.2 / CC 4.5.3
`review`	`reconsideration:{grounds}` appeal / review	CC 3.1.9.2
`slash`	`slashing:{outcome}` verdicts and the `quarantine:{state}` marker plane	CC 3.1.9.2

Subject standing. *(normative; the CC 3.4.5 contestation pair)* The **subject of the target attestation** holds standing to file `reconsideration:{grounds}` — and a licence subject **can see every licence about them** (a read over rows they are party to, stated here so that the promise of redress is checkable) and contests by reconsideration, never by withdrawal (CC 2.4.1.1) — contestation at zero disclosure: the subject files band-blind, and a duty-holder who CAN see the composition performs the valence-sensitive step. This standing is what makes the CC 2.4.1.1 anti-Goodhart retraction carve-out admissible: the withdraws path closes only because a real contest path exists. **What the person named sees (normative).** The person named always sees that a record about them exists, its kind, its consequence and who answers for it; peer speech and an authority's act are visible to them in full; a duty-holder's record MAY withhold the reporter's identity and the report's content from the person named where the duty-holder's process protects the reporter, the moderator's process of CC 4.5.4 included.

Enforced-admission rule — the principal is the chain root, NOT a payload field. The principal an action is taken *on behalf of* is **discovered by walking the `delegates_to` graph upward from `attesting_key_id`** — it is **never** carried in a payload field. There is deliberately **no `on_behalf_of` (or equivalent) envelope field**: a side-field both violates the 1+4 lockdown *and* opens a bypass — if "absent field ⇒ as-self ⇒ admit," then any emitter that simply omits the field (e.g. an AI agent or untrusted party firing a takedown) is admitted as-self with no accountable chain proven, and the gate becomes a no-op exactly where it is load-bearing. A moderation action (`takedown_notice`, `moderation:*`, `reconsideration:*`) is admitted **iff** one holds **positively**:

- **(a) as-self** — `attesting_key_id` *itself* holds the matching duty over the target: it is the target content's own subject, **or** the target community's CC 4.5.4 named-moderator / `moderate`-holder. A zero-hop chain rooted at itself.
- **(b) delegated** — a live `delegates_to` chain `root` →`* attesting_key_id` where **every edge bears the matching scope** (`scope` ⊇ `{moderate|takedown|review}`), the **root holds the duty over the target** and is an **accountable root** (`accountable_root`, CC 3.2 — an accountable human), depth ≤ 5 (CC 4.1.1), and **no edge is `withdraws`-revoked**.

Otherwise **REJECT**. **Absence of a principal field is NOT an admit condition** — admission requires (a) or (b) to hold positively; a verifier MUST NOT read "no field present" as "as-self." This is the faithful mirror of `consent_revocation` (CC 2.4.1.1 rule 3), which derives its principal from the existing `subject_key_ids` relationship + the chain, never a side-field. Substrate SHOULD record which rule + which root admitted the action (the CC 2.4.1.1 per-rule audit metadata).

Deputization + attenuation (normative). A `delegates_to` MAY permit its delegate to **deputize** (further-delegate the duty) — but **only if the delegator granted it**, by including `sub_delegation` in the granted `scope` (CC 4.4.3.4.3.1). Every sub-delegation **attenuates, never expands**: `child.scope` $\subseteq$ `parent.scope`, and constraints may be *added* but never removed — the capability-attenuation rule. The chain is depth-capped at 5 (CC 4.1.1) and **revocable at any link**: a `withdraws` against *any* `delegates_to` in the chain invalidates everything downstream of it — *for acts not yet taken*: authority is judged at the act's signed instant (CC 2.4.1.2.1), so a licence issued while the chain was live stands until its authority suspends or revokes it, and the withdrawal stops issuance from then on. A withdrawal reaches forward: acts already taken under the duty stand, and only a `withdraws` for a compromised key reaches back, to its `revoked_after` bound (CC 3.2, standing over time). So a delegator decides at grant time **whether** their deputy may appoint further deputies and **under what constraints**, and can sever the entire subtree with a single revocation — deputize-a-teacher's-aide, hand-a-shift-to-another-mod, appoint-an-agent, all with bounded, revocable, attenuating authority.

Attenuating a capability to a family — `infra:attest:{dimension_family}` (normative). `infra:attest` grants *attest anything as the delegator's infrastructure*, which is the only vehicle a licensing authority otherwise has for "issue `licensure:CA_medical_board` on my behalf" — an over-grant the attenuation discipline above exists to prevent, but which no field carries. A `scope` token MAY therefore be **sub-scoped to a dimension family** — `infra:attest:licensure:{authority_id}` — under the attenuation rule above: narrower only, never wider, and `child.scope` $\subseteq$ `parent.scope` is satisfied by construction because the sub-scope names a subset of what the parent reaches. This adds no member to the closed `infra:*` set: it is a **caveat on an existing capability**, not a new capability.

Sub-scope matching is directional, and a prefix test gets it backwards (normative). A parent token satisfies a check for its child; a **child MUST NOT satisfy a check for its parent**. The naive implementation — `scope.starts_with("infra:attest")` — returns true for `infra:attest:licensure:X` and hands a deliberately narrowed holder the full attest capability, silently repealing the attenuation that was the point of issuing it. A verifier MUST match on the full token with explicit parent/child semantics, and MUST NOT widen an **unrecognized** sub-scope to its parent: an unknown caveat fails closed, exactly as CC 3.4.7.3 Clause B requires a membership test rather than a purity test for the same class of defect — the loophole spelled as a simplification.

Target → duty-holder resolution — makes the rule substrate-enforceable. "Holds the duty over the target" is resolved by mapping the action's target to its duty-holder set, then checking the two predicates against it:

- `takedown_notice{content_sha256}` → the content's **authoritative** subject set `subject_of(content_sha256)` (self — see the resolution rule below; **not** the action payload's self-declared `subject_key_ids`) ∪ `is_named_moderator(·, C, takedown)` for the content's community `C` (its `cohort_scope: community` / `community_id`).
- `moderation:{allegation_type}` against a subject → `is_named_moderator(·, C, moderate)` for the relevant community.
- `reconsideration:{grounds}` against a prior action → the **subject of the target attestation** (the standing granted above) ∪ `is_named_moderator(·, C, review)` for that action's community; the ruling on it stays with the review-holders.
- `reconsideration:{grounds}` against a `licensure:{A}` row → A's authority set (CC 2.4.1.2.1) plus any holder of a `review`-scoped chain from that set.

(a) as-self holds iff `attesting_key_id ∈ duty-holders(target)` (a subject, or `is_named_moderator`); **(b) delegated** holds iff the chain `root ∈ duty-holders(target) ∧ accountable_root(root)`. With **`accountable_root`** (CC 3.2 — an accountable human) and **`is_named_moderator`** (CC 4.5.4) both resolvable from existing rows (community record + `identity_occurrence` + `delegates_to` + `community_id` + `subject_key_ids`), CC 4.5.5 is **fully substrate-enforceable** — community moderation is not rejected for lack of a resolvable shape. No new structural primitive.

Subject authority is resolved from the content's signed provenance, NOT the action payload. The subject side of admit-(a) has the same substrate-resolvability requirement as the named-mod path: *which* `subject_key_ids` is authoritative. A `takedown_notice` / `moderation:*` action carries a `content_sha256` **and** its own payload `subject_key_ids` — and a substrate that reads the subject set from the **action's own payload** lets an actor self-declare `subject_key_ids = [self]` over content it does not own, satisfy "as-self," and take down arbitrary content **without being a named-moderator** (a narrower, attributable re-opening of the takedown-isn't-a-coup hole on the subject path; the named-mod path is unaffected). The subject claim MUST instead be verified against the content's **establishing attestation**:

- **`subject_of(content_sha256)`** := the signed `subject_key_ids` of the **establishing attestation** — the `scores` Contribution whose content the `content_sha256` binds (the CC 2.3 subject set is signed *inside* that attestation by its producer, not assertable by a later third party). A substrate resolves `content_sha256` → establishing attestation → its signed `subject_key_ids`. This is the same content-hash → signed-attestation resolution every CC 2.1 verifier already performs; no new index.
- **Admit-(a) subject-self** for content targets then means `attesting_key_id ∈ subject_of(content_sha256)` — the **signed** subject, never the takedown payload's self-declared set. The action payload's `subject_key_ids` is, on the subject-authority path, **advisory only** (it MAY be used to *route / queue*, but MUST NOT be the set admit-(a) is checked against).
- **Fail-secure when the establishing attestation is not locally held.** If a substrate cannot resolve `content_sha256` to its establishing attestation, `subject_of(content_sha256)` is **undetermined** and the subject-self clause **fails** (it does not admit) — the named-moderator path (b) is the only remaining route, exactly as for any other unprovable-authority case. Absence of provenance is never an admit condition (the same discipline as the `on_behalf_of` absence rule above).

With this, **both** clauses of admit-(a) — subject-self and named-moderator — and clause (b) resolve against **signed** state, never self-declared payload. The CC 4.5.5 gate has no remaining self-declaration spoof. Composes over the existing `scores` attestation + `subject_key_ids`; no new structural primitive.

The "takedown-isn't-a-coup" property, made structural. Because every action is delegate-signed, delegator-traceable up the `delegates_to` chain, rooted in an accountable root (`accountable_root`, CC 3.2 — authority roots in an accountable human), and revocable, a takedown is **coordinated + attributable + revocable** — never a unilateral seizure. A no-authority actor, or a state actor demanding removal of `federation_keys` for whole classes of dissenters, **fails the enforced-admission gate** (CC 4.5.3): `federation_keys` is not content under a community's duty-holders, and the halt, which pauses agents' acts and touches no data door (CC 4.2.1), is no escalation path for it. The CC 4.5.3 immediate-removal timeline is unchanged — speed at the action layer; authority checked at the delegation layer.

1+4 preserved. A `scope` vocabulary + enforced-admission addition over the existing `delegates_to`; the action primitives (`moderation:*`, `takedown_notice`, `reconsideration:*`) already exist. **No new structural primitive.**

4.5.6 Operational-language gate at admission

Every new prefix admitted to the CC 3.1 namespace passes the CC 1.2 four-test gate. Failed admissions are revised (mechanism-descriptive reframe) or rejected.

4.5.7 Watchlist auto-detection: opt-in, per-group, separation-of-powers

Content-watchlist auto-detection: a CC 4.5.5 `moderate`-scope holder **optionally** enables a watchlist (`watchlist:{id}`, CC 3.1.9.4) for a group they moderate; the fabric auto-fires the matcher at the **publish/share seam** and auto-fires the action — CSAM → `takedown_notice{PerceptualHashCsam}` (CC 4.5.3); other → `detection:*` + a `moderation:*` ModerationEvent to the named moderator. Rides existing primitives — **no new structural primitive.**

Opt-in, per-group, NEVER global (normative). A watchlist is enabled per-group by its `moderate`/`takedown` authority; a global "scan everything" config is **non-conformant** — that is the bulk-surveillance posture the framework refuses. Enable/disable is **signed by the authority and revocable** (`withdraws`).

Separation of powers (the responsible-design invariant). No single party does all three:

Party	Holds	Cannot
Fabric (the node)	the *mechanism* — the matcher at the publish/share seam	provision the hash-DB; choose to enable
Operator	the *licensed hash-DB* (IWF/NCMEC/PDQ, CC 4.5.10.1, operator-provisioned + unshippable) + the NCMEC report obligation	turn it on for a group; act without the authority's opt-in
Authority (`moderate`-holder)	the *opt-in* (per-group enable)	run the match itself; access the licensed list

Audit — never silent (normative). Enabling a watchlist emits `hard_case:watchlist_enabled:{group}` (CC 3.1.9.4) — who turned it on + which list; **every match** emits `hard_case:watchlist_match:{group}`. Enablement and matches are on the record, always.

CSAM-disable non-silent floor (normative). Disabling a **CSAM** watchlist MUST be an audited act — a `withdraws` signed by the authority that **emits `hard_case:watchlist_enabled` (disable variant)**; **silent removal of a CSAM list is barred** (a predator-operator cannot turn off CSAM detection without leaving a trace). Ordinary (non-CSAM) lists may be freely toggled. This is the floor that keeps the opt-in honest.

Honest scope. Detection runs **only at the publish/share seam of enabled groups** — it **cannot** reach CC 5.2 self/family private content (the universal E2EE limit; not claimed solved), and CEG does **not** mandate client-side scanning. **1+4 preserved** — `watchlist:{id}` rides `scores`/config over `delegates_to`; the audit reasons ride the existing `hard_case:*` prefix; the actions ride `takedown_notice` / `detection:*` / `moderation:*`.

4.5.8 `identity_type` as a set: single-key role cohabitation

Per CC 3.4.7.1. `federation_keys.identity_type` is a **set of roles**, not a single scalar role, so the CC 3.4 reserved-prefix gates are evaluated by set membership (`X ∈ identity_type`). This routes through the CC 4.5.1 amendment process.

4.5.8.1 Cohabitation discipline for constitutional + substrate roles

Set membership grants the wire-level *right* to emit per held role, but two roles carry defense-in-depth guidance against cohabitation:

- **`accord_holder` (CC 3.4.1 + CC 4.2)**: the halt authority of CC 4.2. Consumer policy SHOULD treat an `accord_holder` key that also holds non-constitutional roles (e.g., `{accord_holder, agent}`) with elevated scrutiny — the accord roster's halt authority is strongest when its keys are single-purpose. The cohabitation is NOT forbidden at the wire layer (the substrate cannot adjudicate constitutional intent), but the CC 4.2 entrenched-`family` discipline RECOMMENDS dedicated accord-holder keys.
- **`substrate_persist` / `substrate_edge` (CC 3.4.3)**: substrate-self-report roles remain bound by the CC 3.4.3 reservation: a `system:*` row is emitted only by the running substrate. A key cohabiting a substrate role with an application role (e.g., `{substrate_persist, agent}`) is still held to that reservation for its substrate-role emissions; cohabitation does not relax it.

4.5.8.2 The role set is the representation

Surface	Rule
`federation_keys.identity_type`	a set of role strings; a scalar is read as a singleton set
CC 3.4 emitter-rule evaluation	`X ∈ identity_type` (CC 3.4.7.1)
CC 3.1 dimension namespace	independent of the role set
Envelope (CC 2.1)	independent of the role set
1+4 structural primitives (CC 2.4)	independent of the role set
subject_kinds (CC 3.3)	independent of the role set

The rule (normative). `identity_type` is a set of role strings, and a scalar reads as a singleton set, so for a single-role key the set reading and the scalar reading agree: `X ∈ {X}` ≡ `X == X`. The set is not a CC 1.7 "Nth path" (it adds no namespace surface); it is the CC 3.4-layer rule that lets one key hold many roles without widening the wire's expressive surface.

4.5.8.3 The agent-side stance

- **Capacity self-emission (CC 3.4.5)**: **unaffected by role cohabitation** — which is the only claim made here. CC 3.4.5 also carries a deferred, non-normative consent-gate and read-boundary block; nothing in this section ratifies or restates that material. What is unchanged is the interaction: the anti-Goodhart `attesting_key_id ≠ attested_key_id` rule binds regardless of how many roles a key holds. A folded `{agent, lenscore_detector}` key still MUST NOT score its own `capacity:*`. Role cohabitation does not create a self-attestation backdoor.

- **Reasoning-trace dimensions**: no separate reserved `identity_type` required; reasoning-trace emission rides the agent role's open-vocabulary surface. Cohabitation does not change this.
- **Agent-intent / detector-envelope split**: a cohabiting key emits agent-intent attestations under the agent dimensions and detector verdicts under `detection:*` (CC 3.4.8 worked example). The namespace keeps the two surfaces distinct; cohabitation grants the right to emit on each, never merges them.

4.5.8.4 What this documents

- The set-membership reading of every CC 3.4 reserved-prefix emitter rule (CC 3.4.7.1)
- The canonical-bytes encoding for the set (sorted-ascending, deduplicated, comma-joined; single-role keys encode identically to their scalar form)
- The detector-fold worked example (CC 3.4.8)
- The cohabitation discipline for constitutional + substrate roles (CC 4.5.8.1)

What it does NOT do:

- Expand the CC 3.1 dimension namespace, the CC 2.1 envelope, the CC 2.4 structural-primitive set, or the CC 3.3 subject_kinds (zero new wire surface beyond the `identity_type` representation)
- Enumerate a closed set of role values — `identity_type` members remain an open vocabulary owned per CC 3.4 reservations + sibling-component vocabulary extensions
- Forbid any cohabitation at the wire layer (substrate enforces gates by membership; constitutional/substrate cohabitation discipline is consumer/operator policy per CC 4.5.8.1)
- Address `affiliations` (the fourth `cohort_scope` tier, defined at CC 4.4.3.2.8)

4.5.9 Geographic-community privacy invariant

Per CC 3.2 `community` with `cohort_subkind: geographic` + CC 3.3.3 `location_proof` + CC 2.6.6 H3 cell canonicalization + CC 4.4.3.2 Policy M.

A load-bearing privacy invariant: **joining a geographic community is a one-way disclosure**. Three sub-properties make this wire-format-level, not policy-level.

4.5.9.1 Joining is opt-in: substrate does NOT solicit location

A `location_proof` Contribution is emitted **only** by the subject themselves (`attesting_key_id == subject_key_id`) or by a `delegates_to` chain with `scope: [consent_revocation]` — the substrate has no path to mint a location_proof on behalf of a key without an explicit signature.

Communities cannot **require** a location_proof from non-members (they can only **gate admission** on whether a member has produced one). The substrate has no mechanism for "involuntary location disclosure" via the wire format. A bad actor cannot force-publish another key's location_proof; without the subject's signature, the substrate rejects.

No escalation path is needed. A demand that the substrate emit `location_proof`s for non-consenting subjects asks for rows the admission door refuses: a `location_proof` without the subject's own signature is rejected, as stated above. The substrate's role at this primitive is mechanical opt-in enforcement. The halt pauses agents' acts and touches no data door (CC 4.2.1), so it is not the remedy here; a legal dispute over content is routed by CC 4.5.3 takedown coordination under the CC 4.5.5 duty-holder gate.

4.5.9.2 Rough-only is wire-format-enforced

Per CC 2.6.6.1: `location_proof.cell_resolution` $\leq$ 7. H3 resolution 7 hexagons average about 5 km^2 in area (edges about 1.2 km) — sufficient for city/borough disclosure without block/building precision. Producers attempting finer resolution have admission rejected at the substrate gate.

This is the closure of an entire class of accidental over-share: a malformed UI cannot publish precise location even if the client-side gating fails. The wire-format-level enforcement is the privacy primitive; UI is the second line.

Substrate emits `hard_case:location_proof_resolution_violation` (CC 3.4.2) on rejection so operators can observe malformed-client patterns. This is observability for operator debugging, NOT a slashing trigger — malformed producers are usually buggy clients, not attackers.

4.5.9.3 Leaving is forward-only: the audit chain keeps the tombstone

Per CC 2.4.1 `withdraws-isn't-retroactive` + CC 4.5.12.1 forward secrecy on departure. When a subject withdraws their location_proof (or leaves a geographic community):

- Forward visibility evicts (consumer policy treats the subject as "no current location proof" for new admission decisions from withdrawal-time forward)
- A withdrawn `location_proof` leaves serving like any row of the subject's own (CC 4.5.3); the audit chain keeps the tombstone — that a proof existed and was withdrawn — and never the cell. What withdrawal and expiry govern is the present claim; the record that a claim was made is the tombstone, not the content.

This is the cost the subject opts into when emitting the `location_proof`: a copy a peer took before the withdrawal reached it is not recalled by the wire, as for every row (CC 2.4.1), and the tombstone is what every holder converges on. What the wire format promises:

- **Rough-only** (CC 4.5.9.2): a claim, while it stands, is bounded to resolution $\leq$ 7, never finer
- **Opt-in** (CC 4.5.9.1): the claim exists only because the subject signed and emitted it
- **Auditability**: the subject can prove what they did or did not claim, and when; the tombstone is the receipt, and it carries no cell

4.5.9.4 What this documents

- The rough-only enforcement primitive at CC 2.6.6.1 (resolution $\leq$ 7 for location_proof)
- The forward-only leave semantics at CC 2.4.1 (withdraws-isn't-retroactive applied to location_proof)

- The opt-in admission flow at CC 3.3.3 (location_proof signed by subject only or delegates_to proxy chain)
- The geographic-community admission composition at CC 4.4.3.2.3 (Policy M evaluate_subkind_admission for geographic)
- The substrate-self-report observability at CC 3.4.2 (4 hard_case prefixes)

What it does NOT do:

- Mandate H3 over alternative geospatial systems for operator-internal use (operator choice; wire format uses H3 only)
- Provide a place-name registry (communities self-name; H3 cells are the substrate-level binding)
- Define specific cell-resolution conventions for community-side `geographic_constraint` (only `location_proof` is bounded to ≤ 7; communities MAY scope themselves at any resolution per operator/founder choice)
- Codify other community subkinds (`professional` / `interest` / `local-business` / `event-attendees` / etc.)
- Address `affiliations`

4.5.10 Hash-database operator policy

Perceptual-hash matchers (PhotoDNA / PDQ / Project Arachnid / GIFCT hash-sharing) are pluggable at the substrate. Operators choose which matcher implementations to enable; CEG governs the access-policy contract.

4.5.10.1 Hash-database access landscape

Matcher	Access posture
PDQ (Meta, 2019)	Open algorithm; the hash lists come from the gated feeds an operator can obtain
PhotoDNA (Microsoft, 2009)	Access-gated; restricted to vetted orgs (NCMEC + select platforms); substrate operators cannot download the hash database directly
Project Arachnid (C3P, 2017)	Access-gated; API access requires C3P partnership
GIFCT hash-sharing	TVEC-focused; access via GIFCT membership

4.5.10.2 Operator path (default)

For a CIRIS substrate operator running a federation node, **the default operator path is**:

> ***Self-hosted PDQ matcher against the reference feeds the operator can obtain*** *(the PDQ algorithm is open; the hash lists come from the gated feeds an operator can obtain — the Canadian Centre for Child Protection's (C3P) Project Arachnid feed, which requires a C3P partnership, and GIFCT-published lists where available). No federation-level access-governance overhead; a gated feed's own terms apply. Operator carries responsibility for index freshness.*

Of the three paths — (a) a self-hosted matcher, (b) clearinghouse delegation, (c) on-device OS-vended hooks — this is (a): it avoids the federation dependency at the protective layer that (b) would introduce, and the controversy around (c).

4.5.10.3 Future hash-coalition path

Path (b), clearinghouse delegation — substrate operators delegating perceptual-hash checks to a trusted coalition peer via federation — requires a CIRIS hash-coalition that can serve as a clearinghouse. The slot is reserved; the coalition operator-onboarding flow is not specified here.

4.5.10.4 What this documents

- The closed-set of `legal_basis` values that compose with `PerceptualHashCsam` (the only `legal_basis` value that consumes hash-match output as immediate-removal trigger; see CC 3.3.2)
- The operator-onboarding contract: an operator running a PDQ matcher MUST register their matcher's source feeds (which hash-list source URLs they're pulling from + freshness cadence) via a `system:perceptual_hash_matcher:registered` Contribution. Composes with CC 3.1.3 Persist substrate-self-report discipline.

What it does NOT do: prescribe which hash databases an operator MUST use. Operator choice. CEG documents the wire-format slot + the operator-onboarding contract + the recommended default; concrete matcher selection is operator policy.

4.5.11 Bootstrap-content pattern

After federation genesis, a curated batch of proposal Contributions is admitted via the CC 4.5.1 amendment flow, populating the federation's substantive content surface with high-quality ethical-framework material. **Content-neutral**: any sufficiently substantive ethical-framework source can serve. The wire format admits content via the CC 3.1 namespace; the CC 1.2 gate ensures prefix names don't import source-tradition vocabulary.

Multi-source commitment: bootstrap batches from CARE Principles (Indigenous data governance), Buddhist economic-justice scholarship, secular humanist instruments, African philosophy of personhood work — all through the same amendment process. The framework is multi-traditional by design.

4.5.12 Self/family membership governance

Per CC 3.3.6 `identity_occurrence` + CC 3.3.4 `family` + CC 4.4.3.4 Policy L + CC 5.2 structural-invisibility. The four governance decisions for self/family membership.

4.5.12.1 Forward secrecy on member departure

When a member leaves a family, or an occurrence is withdrawn from a self-collective, the rule is the one at CC 4.4.3.4.5: the departed party keeps the `key_grant`s it already holds, nothing is re-encrypted, and because every self and family write seals under a fresh DEK the substrate simply stops wrapping new DEKs to the departed keys. No rotation ceremony is needed at these scopes and none is defined.

Rationale: consistent with CC 2.4.1 `withdraws-isn't-retroactive` + CC 4.5.3 "takedown isn't a coup" + CC 4.4.3.5.1 consent-decay-doesn't-re-encrypt. The substrate's forward-secrecy posture is uniform across consent, takedown, and membership-departure surfaces.

4.5.12.2 Multi-family membership: envelope `family_id`

One identity MAY belong to multiple families. Each family has its own DEK and its own membership roster; a `cohort_scope: family` Contribution MUST carry `family_id` (CC 2.1 envelope field) naming which family's DEK and visibility apply. Substrate rejects family-scoped Contributions missing `family_id`.

Rationale: avoids cross-family DEK confusion when one identity is in N families; gives the substrate an unambiguous routing key for the `key_grant` cascade per CC 4.4.3.4.1.

4.5.12.3 Self-occurrence admission: single-vouch

A new `identity_occurrence` is admitted on a signature from EITHER the root `identity_key_id` (until the self evicts its genesis key) OR any currently active occurrence of that identity ("trust any device I've already onboarded"); a `node`-role signer counts only through its live owner binding, and a `user`-role key joins the self's roster by the self-roster door of CC 3.3.6.3 (one key: the owner's own signature; two or more: a strict majority of the live roster, the proposer included, or the lone path after its window with no objection). Higher-assurance setups MAY layer requirements on `hardware_attestation` via consumer policy.

Rationale: matches user-intuition for self-membership ("my phone unlocks my laptop's trust posture for new devices"); avoids the operational overhead of multi-vouch for routine onboarding; the security gradient lives in the optional `hardware_attestation` field, not in the admission rule.

4.5.12.4 Reserved-prefix substrate emissions: locked in CC 3.4.4

Per CC 3.4.4. The four substrate-emitted membership-event prefixes (`hard_case:identity_occurrence_added:*`, `hard_case:family_membership_change:*`, `hard_case:family_consensus_protocol_change:*`, `hard_case:family_consensus_protocol_violation:*`) are reserved to `identity_type="substrate_persist"` emitters. Same discipline as the existing `system:*` substrate-self-report family.

4.5.12.5 Family admission: consensus_protocol (normative)

Unlike self-occurrence, family membership changes are **NOT single-vouch by default**. The family's `consensus_protocol` field governs admission. Six canonical protocols (`founder_only` / `unanimous` / `majority` / `quorum:M/N` / `weighted:{rubric}` / `custom:{family_id}`); operator vocabulary extends, and `open:self_admit` is the open protocol of a public group (CC 4.4.3.2). `founder_only` is conformant for a family and for a publisher-gated subscription, refused for a community (CC 4.5.13), and admissible for an affiliation only under the one-voice countersign rule of CC 4.4.3.2.8.

The consensus_protocol field is itself subject to amendment via the SAME protocol's rules (meta-amendment shape parallel to CC 4.5.1.2) UNLESS the family is `consensus_protocol_entrenched: true`. Entrenched families reject amendments at the substrate gate; replacement requires the family's documented out-of-band ceremony.

Rationale: families are multi-party collectives where membership changes have real consequences (admit-new-member = grant DEK access to all extant cohort_scope: family content). The consensus_protocol gives the family explicit governance over its own boundary. Self-amendment lets families evolve their governance as they grow; entrenchment lets safety-critical families lock the boundary against any internal authority.

4.5.12.6 What this documents

- The wire-format primitives that compose into self/family membership (CC 3.3.6 + CC 3.3.4)
- The structural-invisibility discipline at CC 5.2 (no holding claim leaves a self/family cohort)
- The at-rest encryption flow composition at CC 4.4.3.4 Policy L
- The consensus_protocol vocabulary (canonical kinds; open-vocab extension)
- HUMANITY_ACCORD as the canonical entrenched-`family` instance at CC 4.2.3

What it does NOT do:

- Lock the consensus_protocol vocabulary (open vocab; canonical kinds named for ecosystem coordination)
- Re-encrypt history on departure (a removal is never retroactive, CC 4.4.3.4.5)
- Prescribe per-family entrenchment policy (operator/family choice)
- Document the at-rest encryption flow details (substrate-side)

4.5.13 Reverse-quorum governance: presence is authority, absence forfeits it

Not a membership rule. Reverse quorum is the commons objection brake. It never admits anyone to a family or community and confers no consent on anyone's behalf; joining is CC 3.1.3.2's proposal and acceptance.

The general pattern. CC 4.2.6 is not a one-off kill-switch rule; it is the federation's **general governance shape**, and this section states it as such. **Presence is authority, absence forfeits it, a timer decides.** A decision opens a bounded window; one present, accountable actor may act alone; on that actor's absence a count carries, over the set the door's policy names (the policy as data, below); whoever stays absent never blocks the present actor. The CC 4.2.6 accord kill-switch is the **constitutional instance** of this pattern: one present holder fires, and a roster change counts its yes-votes against the standing roster, because the roster is the capture vector; **community-scope moderation is its first community instance**: a present moderator acts, and on the moderators' absence the live community governs its own content. One mechanism, two scopes — the accord over humanity's off-switch, moderation over a group's content.

Roles — moderator (required), steward (optional super-mod). A community is **required** to have a named **moderator** — the CC 4.5.4 `moderate`-duty holder — and that is the *only* required role: the moderator is the fast-path decision-maker. A community **MAY** additionally name **one or more stewards** — the super-mod tier, steward-bound per CC 3.2, who appoint and oversee moderators; **a steward may add further stewards** — but **a contested party MUST NOT mint co-stewards to manufacture a quorum or dilute a pending protective veto** (the CC 3.4.13 Q7-3 rule, which applies at community scope as it does there). At creation the creator chooses their own role: **steward + moderator**, or **moderator only**.

This document imposes no quorum *on* a moderator or a steward; a community MAY impose one on itself (below) — unilateral accountable authority within scope is what each role means; either resolves an action by a **single signature**. **Term pin:** *community steward* is this super-mod tier — distinct from *steward-binding*, the CC 3.2 owner→node relation. Two senses, one word; CC uses the qualified form wherever context does not fix it.

The mechanism — propose, 48 h window, moderator or steward acts, community falls back. Anyone MAY propose a moderation action — a report, a takedown request, a keep-or-remove question — by the CC 4.5.5 open-labeling path (a report→`scores` Contribution against the target). The proposal opens a **48-hour participation window**. Two ways it resolves:

1. **A moderator or steward acts unilaterally.** The community's **moderator** (the required `moderate`-duty holder, CC 4.5.4) — or any **steward**, where the community has named one — resolves the action by a **single signature**, at any point in the window. **What a single signature resolves is content and a member's standing to speak in the room**: a withhold, a takedown of a row, a warning, a block for the room; **it is never a removal from the roster**, which is a roster change under the floor of this section in every protocol, open admission included (CC 4.4.3.2), so no single key, a moderator's or a founder's, removes a member alone, and a moderator appointed by one founder cannot remove another. At two members nobody is removed, since a unanimous removal is every other member and a strict majority of the roster, which at two coincide only if the one removed consents; either may leave by their own forward-only act, the last founder included, since the founding-core rule that keeps a lineage from stalling belongs to infrastructure communities and conferring families (CC 3.2 T7) and not to a room; a founder who would rather hand the room on promotes a member to founder under the protocol, with that member's own signature; and a room left with no founder locks and elects. This is the fast path, and the moderation analogue of the CC 4.2.6 **fire-floor-of-1**: a single accountable party acts, and the absence of others never blocks them.
2. **On their absence, the community vote carries.** If **no moderator or steward acts within the 48 h window**, the proposal is tallied by **live-majority vote over the community's live members** — the CC 4.2.6 participation window read at community scope: presence asserts (a member signs a fresh proof-of-presence + vote over the proposal nonce, entering the live set `L`), a silent member is not in the denominator, and the tally is taken over **who shows up** — the count set a community's policy names, where the accord's roster change names the standing roster. The community **routes around** an absent moderator.

The reverse-quorum is a **fallback for moderator/steward absence, never a check on them.** The 48 h figure is tuned so that a **present** moderator always has time to act unilaterally; the community fallback fires **only** on genuine absence. **Why 48 hours (normative).** The window governs only the fallback, since a moderator rules at any time and a withhold is immediate; its one job is the legitimacy of the members' majority, and 48 hours is the shortest span that gives every member in every time zone a weekday to respond, where 24 covers one working day and no day off. **Two acts, not one (normative).** A member who sees harm has two acts and they are separate choices. A **report filed privately with the duty holder** (the named moderator, or the steward) opens no window, is seen by nobody but the duty holder, and becomes the duty holder's record under CC 4.5.5, the reporter shielded where the process protects them; the moderator rules on it at any time or lets it rest. A **moderation action raised in the room** is public, names what is asked, and starts the 48 h window at once, the members responding and

the fallback carrying on the moderator's absence. The client offers both and never folds one into the other.

The moderator election — the room locks and chooses (normative). A community must never sit without a moderator, and no node holds a key that could appoint one, so a community with no live holder of `moderate` **locks**: read-only to everyone, since nobody can moderate; members may read, and nobody may post, admit or act until a moderator is seated. It then **elects**, in a vote of **24 hours**: the **ballot is the roster**, every member a candidate, presented in the worthiness order of CC 4.5.4 rule 2 (track record, then admission order, then key), so that a voter who confirms the ballot as presented votes that order; each person ranks **up to three**; **one ballot per person** however many keys they hold, folded exactly as a vote is above: a person's agreeing ballots are one ballot, disagreeing ballots neither withdrawn are no ballot until a withdrawal leaves one, and no instant decides; the count is **instant runoff**, the weakest standing candidate eliminated in turn, a tie broken against the later in worthiness order, until one remains; the floor of two voices applies. **The winner accepts by their own row**, as a member accepts an invitation (CC 3.1.3.2); a winner who declines, or is silent past a grace the room's policy names, passes the duty to the next in runoff order, and so on; nobody holds a duty they never signed for. **The voters' ballots are the appointment's signatures**: the seated moderator is appointed by the body under its protocol, with no node signing anything. A room where nobody votes stays locked and the ballot stays open. **A steward, where one exists, MAY appoint a moderator directly at any point and end the vote. One result on every node (normative).** Nothing signed opens an election, since a lapse by inactivity has no row, so the election is **named by what every node holds**: its id is the digest of the community id and of the last seating row or, for a room never yet seated, the genesis of the roster, so that a member leaving while the room is locked (a resignation is the member's own act and a lock traps nobody; nothing is admitted or removed) changes nothing about the election; a ballot names that id, every ballot under it belongs to one election, and the electorate is the live roster at the count, a ballot from a member no longer live counting for nothing; there is no reopening as a separate thing, the ballot stays open until a seating lands. A node's clock closes nothing: **the winner's acceptance row binds the ballot set it was seated on**, carrying the election id and the digest of the set of ballots it was counted over; the set covering the ballot and withdrawal rows held, so the count over it is reproducible; a node admits it only when it holds that set, the count over exactly that set yields the signer, the set meets the floor, and the node's own 24 h since it first held the lock have passed; **the seat is a function of the rows held and never of their arrival:** among the valid acceptances a node holds it seats the one counted over the larger set, and on equal size the smaller digest, a node stores every ballot and acceptance it receives rather than refusing a late one, and a ballot is void only against the seating that wins, so a node never unseats on arrival order alone and every node holding the same rows seats the same moderator; two nodes that hold different rows, as halves of a fleet do across an edition in which one half admits a removal the other refuses, may seat differently until their rows converge, and that is a residual of the mixed fleet, not of the rule. **The floor in a small room** is the smaller of two voices and the roster's size, so a room of one elects itself by its own ballot and no room is locked beyond its members' reach; a founder is never locked out, holding the duty at zero hops. (The policy table's row `moderator_election`; the ranked-choice model of the register's row; the ballot and the acceptance are the rows of CC 3.1.4, `ballot:moderator:v1` and `ballot:moderator_acceptance:v1`.)

Optional moderator quorum (community config). Unilateral moderator action is the *default*. A community **MAY** instead require a **moderator quorum** for an action to carry — either **full** (every named moderator must sign) or **live-within-window** (a live-quorum of the moderators who respond within a stated period — the reverse-quorum applied to the moderator

set itself). This rides the community's existing `consensus_protocol` (CC 4.4.3.2) applied to the `moderate` duty; it **tightens the fast path** without weakening the community-vote fallback, which still carries on total moderator/steward absence. The choice is the community's to make and to change (a steward, or the community vote, may set it).

Protective default — a harm report defaults to removal. For a takedown / harm report, the window's **default outcome is removal.** Content survives **only** by an affirmative, on-record **keep-decision**: either **(A)** a moderator's or steward's unilateral *approve-to-keep* signature, **or (B)** a live-majority *vote-to-keep*. **Passive survival is impossible** — doing nothing within the window removes the content. Therefore **surviving reported content is always attributable**: the moderator or steward who approved it, or the majority who voted to keep it, are cryptographically on record. *A community in which reported content survives has self-identified* — by name, by signature.

A harm report is not a change proposal. Its window's default is removal; the act that must land is the keep, and the floor applies to the keep — a moderator's or steward's named approve-to-keep, or a vote-to-keep that carries its majority. A takedown notice under a legal basis (CC 4.5.3) is a third thing: duty-holder-signed, executed without a window.

Infohazard consent gate — no passive exposure. When the live-majority vote **favors removal/moderation but the content is retained** (flagged, not hard-removed), it is **auto-hidden behind an active interstitial** — *"I consent to view this material reported as a potential infohazard."* Clicking it **publishes a CC 3.3.1 `consent:*` attestation** (`consent:state:granted` + a `consent:scope:view` qualifier, viewer-side, attributable within scope) — so **passive exposure is impossible here too**: every viewing is an affirmative, signed act by a named identity. This **rides the existing `consent:*` family + a CC 3.3.12 `content_class:{class}` flag** (`content_class:infohazard` / `content_class:reported`); **no new wire shape** — it is the CC 4.5.5 `content_class` gate composed with the CC 3.3.1 consent primitive.

The outcome ladder — every transition an attributable act. A graduated, fully-attributable ladder; nobody passively flags, keeps, or views harmful content:

Outcome	Trigger	Who is cryptographically on record
Hard-removal	CSAM perceptual-hash tripwire (CC 4.5.3 `PerceptualHashCsam`) / takedown / **report undefended** (no keep-decision in the window)	the removal action (CC 4.5.5 `takedown_notice`); content does not survive
Consent-gated (retained, auto-hidden)	live-majority **flagged-but-retained**	**each viewer** — every interstitial click publishes a `consent:*` attestation
Kept-visible	a moderator's or steward's **approve-to-keep** (A) **or** live-majority **vote-to-keep** (B)	**the moderator or steward** who approved, or **the majority** who voted to keep

Single-point warning + inactive forfeiture. A community with **only one moderator and no steward MUST be warned to add a second** moderator or a steward — a lone authority is a single point of capture and of failure. **Inactive moderators and stewards lose their standing:** silence past the community's freshness window **lapses the duty**, and merit auto-promotion + the no-moderator recovery above **reconstitute authority** — the highest-`moderation_track_record` live member is auto-granted the `moderate` duty. **No community is ever hostage to an absent moderator or steward.**

What the protocol guarantees — structure, not behavior (the M-1 framing). The protocol guarantees the **structure**: **default-remove, attributable-keep, attributable-view.** It does **not** guarantee the *behavior* of a community that **affirmatively chooses harm** — a moderator or steward may sign to keep abuse, a majority may vote to keep it, and a viewer may consent to view it. What the protocol forecloses is doing any of these **silently**: every transition up the ladder — flag, keep, view — is an **affirmative, attributable act by a named identity**. Per CC 4.2 M-1, the federation's promise here is **revocability and accountability**, not behavioral control: a community that harbors harm is on the record as having done so, and that record is what justice composes over.

A duty-view is an act of the duty, never a keep or a view (normative). The attributable-view ladder above is about a viewer who *chooses* to see kept content. It is not about adjudication. A `moderate` / `takedown` / `review` duty-holder who examines reported or suspected infohazard material in order to act on it, and anyone who saw material before it was known to be an infohazard and then reported it, is recorded as having performed **an act of the duty** — a `moderation:*` row naming the duty, the purpose, the report or removal that followed, and the exposure — and not as a keep or a view on the ladder. Best practice is built in, in this order: **minimise** (a known match on the perceptual-hash tripwire is removed and reported with no human eyes on it, ever; classifiers triage before a person is asked to look); **restrict** (a person looks only under a named duty, for a named purpose, with the viewer protections the operator provides — blurring, thumbnails, time limits, wellness rotation); **record** (the look is signed as duty, with its purpose and what followed, so the record supports the viewer rather than indicting them, and the mandated report rests on it); **report** (an inadvertent exposure is reported promptly through the same channel, which is what good-faith protection, where the law offers it, turns on). Where the duty-holder acts for an organisation, the organisation is on the bond (CC 3.2) and the duty-holder is its agent for the purposes of any provider protection the law offers. A keep or a view that is not an act of a duty stays on the ladder, attributable and named.

Consistency — member-reporting, not substrate scanning. This is **member-reporting + reverse-quorum adjudication**, *not* substrate scanning of private content. CEG **refuses client-side scanning** (CC 4.5.7): detection runs only at enabled groups' publish/share seam, never over CC 5.2 self/family private content. The principle is unchanged across the federation — **anonymity to outsiders, accountability to insiders** — applied here with teeth on harm: the structure makes silent survival, silent keeping, and silent viewing impossible, while leaving private content unscanned.

The symmetry with the accord (explicit). This is CC 4.2.6 read at community scope, and the two halves correspond. In the accord, a **lone survivor fires** and the roster is changed by a **majority of the standing roster**, cast within a window; in moderation, a **lone steward acts** and, when stewards vanish, the **live community reconstitutes governance**. Fire-floor-of-1 $\leftrightarrow$ single-steward signature; a participation window with a named count set $\leftrightarrow$ the same window with the present members as its count set; Enoch-Arden return $\leftrightarrow$ merit re-auto-promotion on a lapsed steward's silence. **Presence is authority, absence forfeits it, a timer decides** — at every scope; what differs by scope is only the set a count is taken over, which the policy table names.

1+4 preserved. The proposal/window/tally rides the generalized CC 4.2.6 participation shape + the CC 4.5.5 action primitives (`moderation:*` / `takedown_notice` / `reconsideration:*`); the consent gate rides the existing CC 3.3.1 `consent:*` family + the CC 3.3.12 `content_class` flag. **No new structural primitive.**

The policy as data (normative). Every door that opens a window runs one decision shape,

and the shape is five numbers over a named standing set, published as a table rather than described door by door: **enact** — the approvals that land the change at once (a fraction of the base, strictly more; never below a strict majority; the default two-thirds); **cancel** — the objections that void it at the window (at least the fraction; the default one-third); **floor** — the approvals the change must carry before silence may land it at the window (at least the fraction; the default one-third; never fewer than two voices in a community, so one voice never decides who is in a room, whatever the size: where admission is by invitation no single vote admits, and the second voice on an admission is the invitee's own acceptance (CC 3.1.3.2), so a room of one grows by its founder's proposal and the newcomer's yes, which are two keys and two consents, and where admission is open — `open:self_admit`, anyone signs their own admission — no single vote excludes, and the guarantee attaches to removal instead; `founder_only` is a protocol for a family, a publication's subscriber list and a room's own listing, and a community above the family that names it is refused `federation_community_consensus_protocol_violation` — the refusal token, whose emitted record is the `hard_case:community_consensus_protocol_violation` row of CC 4.4.3.2.3 — as an infrastructure community already is); **window** — by the sensitivity of the change: 86 400 s for reversible content, 259 200 s for roster and standing, 604 800 s for the irreversible; **irreversible** — no instant landing, the change waits its whole window even at full approval. The floor is a reverse-path predicate only: nothing lands before the window closes except at enact, so a floored change still waits its time. The base is the standing roster at or below the count minimum the governance policy table pins and, above it, the expected-active count — the community's own member-only census composite (CC 6.1.9), which falls at most a tenth per period and never below its floor — never below a stated minimum, so a fraction stays reachable at ten thousand members as at ten (the law's constants are provisional until measured). A community may set its policy stricter than the defaults, up to a floor at enact, which is the reverse path switched off; it may not set it laxer than the corridor allows: a cancel or floor one voice can reach, an enact one voice can reach, a window under its class, or cancel or floor above enact, are refused as policy (the policy-corridor model; the register's row CLM-policy-corridor). **The governance policy table is published as data with its decision function, its base law, its pin and its vectors** (the register's row CLM-governance-policy-table) — `(policy, base, approvals, objections, elapsed)` → `lands | pending | cancelled`, which an implementation replays rather than re-reading this paragraph; every two-path row is also an instance of the two-path model, which the build regenerates and checks, so a row that breaks the machine's invariants fails the build. The rows: the three defaults by class; community invitation (CC 3.1.3.2: the invitee's own acceptance still required) and removal (forward only, cosigned); the self roster (CC 3.3.6.3: one person's keys, so a floor of one key and its own 24 h window); and three instances the table records under their own rule rather than the generic machine — this section's moderation fallback (a moderator acts alone, a live majority at 48 h), the halt's fire-and-fuse (CC 4.2.1.1) and the custodian's cosigned replacement (CC 7.5.5). **The clock of a window (normative).** Every window of this table is measured by each node from the instant it first held the proposal, never from a signed instant of the proposer or of a cosigner, as the self roster (CC 3.3.6.3) and the halt's fuse (CC 4.2.1.1) already read it: a signed instant is the signer's to choose, and a window read on it could be shortened in advance or entered after it closed, while receipt is the one clock no signer sets; `cosigned_at` stays beside the bytes and decides nothing. Two nodes may therefore close a window at different moments, and nothing lands early at either. Votes travel on `objection:{state}` (CC 3.1.9.2): `proposal`, `assent`, `raised`. Two further fields are on every row: `cancel_over`, the denominator of the cancel fraction — the base for a community, the proposer counted; the *other* keys for a self roster, since the proposer is not asked to object to itself — and `at_window`, what becomes of a proposal that has not landed when its window closes — `cancelled` for every community row, so a held change always resolves;

`stay_open` for the self roster, cleared by the proposer's withdrawal or the target's act; the halt lapses; a moderation proposal is tallied.

The state processor (normative). The queue, the preview and the door are one function run three ways, and a node holds one. Its state is the set of rows the node holds, each with the instant this node first held it, and nothing else: the first-held instant is the node's own receipt, outside the signed row and never replicated, and it is what every window in this document runs from (the window clock above); an object's state, a person's standing toward it and a room's seated roles are folds over that set, never stored verdicts, so a node has no second copy of anything to drift. Its step takes the held set, one row and the instant, and answers that the row's transition is enabled for the signer's standing on the object's view, or is refused by name; an admitted row is appended and nothing is rewritten, and the view after is the fold over the larger set. Run over the transitions enabled for one person it is the queue; run over a row before it is signed it is the preview, which is exact because the same function is the door; run over a row that arrives it is the door. A transition under a policy that is not one voice, the two-path, a cosigned act, the ranked election, the mirror of a moderator's act, is admitted as a row at once and moves the object only when the fold over the rows meets the policy's numbers, so the preview of such a row says what the fold says: the object's state, unchanged, and what is still missing, the voices or the instant. The instant is the one input from outside, for windows; the processor keeps no clock. The impact an item shows (`manifests/choice_item.cddl`) is the difference between the fold before and the fold after, with the family's columns for undo, never a stored field. What a preview claims is exact over the rows the node holds; its counts and standings are the same on every node that holds those rows, since those folds are integer arithmetic over signed inputs, and its window answers are this node's, since a window runs from this node's receipt, which is why the clock gates admission and never the count; it says nothing about rows the node does not hold. What the processor does not model, so that it stays small: signatures, since a row in the held set is already verified; transport; what other nodes hold; projections; text; and any reader's weighting of what it reads, which is composition and never admission.

The choice as data (normative). Every decision a person is asked to make, a first-run step, a consent gate, a deferral, a vote in a room, a moderation action, is one object, and its context says which: `manifests/choice_item.cddl`. An item carries a **context** (the circle, the role, the atoms in force, a title and body), **facts** that are displayed and never chosen, each with its provenance (`wire`, `by_rule`, `not_sent`), a **policy**, which is a row of the governance policy table above (a first-run or consent item runs the self row, one voice of one, no window; a room's item runs the room's row), and **options**, each with a benefit and a downside that are never empty (CC 5.4.3, the choice is explained before it is made), the rows it would sign (`writes`: family, verb, scope), and an **impact** that is not written by hand but **derived from the family model** for those rows: what moves, where it goes (the family's audience and tier), whether it widens or narrows, and whether it can be undone (the family's erasure route and lifecycle), so the three facts a confirmation shows are a lookup and never copy. Four kinds and no more: `choice` (two or more options), `act` (one option, a thing done), `entry` (a value typed or arriving by a ceremony, never an option), `notice` (no options: a fact, a state, a refusal). **Consent is not a kind.** A consent is a choice whose writes are consent rows; announcing is a choice whose write is the binding at federation scope; an anonymous identity is a choice whose write is the identity itself; shaping is a choice whose write is a setting. Each is presented the same way, each changes something, and none is more or less a choice than another, so a first run has no separate class of consents and nothing in it is `required` except what has to exist for the node to run. **Choices belong to the person.** The node keeps one queue of the items awaiting the person's signature, the pending decisions of this section among them: the person signs, the row lands, the change follows, and the result can be viewed. An agent signs no item of a person's queue: the person's choices are

the person's. **Keeping an answer alive is not a new choice, and widening it is.** A node that holds the person's pen MAY re-sign a row the person already chose, to refresh it or to supersede the one before, only as the same row: the same scope, the same prefixes, the same peer, or a peer that follows the person's own later act (a root they accepted after the item said the grant goes to the roots they trust); never a wider one, which is a new choice and needs an item; and never after the person withdrew it, since a withdrawal stands until the person answers again. A vote counts once per person, whatever the number of their devices: a roster seats a key (CC 3.1.3.2), the person is the fedID whose self roster the key belongs to (CC 3.3.6.3), and a tally counts fedIDs; every vote row is admitted and kept, and the count is a fold over the rows held, permutation-invariant and clock-free as every instrument of CC 6.1 is: a person's live vote rows on one proposal that agree count as one voice; rows that disagree, neither withdrawn, count as no voice, since a person at odds with themself has not spoken, until a withdrawal (CC 2.4.1.1 rule 1, the author's own, which any key of the same fedID signs) leaves one standing; nothing is refused at the door for arriving second, no instant orders two devices' rows, and every node holding the same rows reaches the same count. An agent signs its own rows, its tasks, its traces, its receipts, under its own key and its steward binding (CC 3.2), and a person MAY delegate goals or roles to it, that delegation being itself a row the person signs. The purpose of a node and its client is to let the person see the current state of the attestations they trust, through the policies they already hold (CC 4.4.3.8), and to put before them, as one queue, what waits on their hand. An **answer** records the state (`served`, `held`, `signed`, `deferred`, `changed`), the value, the digest of the item as it was shown, and the rows once they exist. Declining is a complete answer and never a failed vote. **What starts unanswered, and what starts on.** A choice whose answer sends the person's content or their agent's traces beyond their own circle starts unanswered and is never preselected: sending traces, and any grant of content to a peer. **Announcing starts on,** its cost on screen: for nearly everyone being reachable is the right and the easy answer, since an announced identity is shared as a short `ciris://` name rather than a node code handed by hand, takes part in public groups, and can join the geographic communities near it where the person also chooses to share a rough location; what announcing sends is the binding and its delegations to the roots the person trusts, never content, and content keeps the smallest cohort scope by default (CC 1.13.3.4); its two costs when declined are those of CC 5.4.6. A default is otherwise admissible only for a choice that narrows or keeps things as they are, and every default's cost is on screen. **Relay offering is the stated exception:** it carries other people's sealed rows and discloses the node's route to a referred stranger, nothing of the person's content or identity, and it is on by default for servers and home nodes under the device policy of CC 5.4.6, because the mesh does not seat its peers otherwise; it is a choice like the others, its cost on screen, and it may be turned off. An unanswered choice that the node does not need in order to run never blocks the person: it is `deferred`, nothing leaves on its account, since nothing leaves without a signed row, and it waits in the queue until answered. An option that cannot be undone is never a default and is confirmed before it is signed; an option that narrows takes one step. Nothing is answered on anyone's behalf: an item whose writes leave the node (a grant, a binding, a consent) answered before a claim is held and not applied, since no owner key exists to sign it, and lapses if the claim never comes; an item that writes nothing outward (a setting of the node itself, shaping among them) applies at once, because applying it leaks nothing and waiting would, and the owner's signed row supersedes the local record after the claim. A client renders an item with no node, from the data alone. **The epistemic web client (normative).** A client of this document renders two served shapes and nothing else: the **view** (`manifests/view.cddl`), one object's rows and the atoms the node folded from them through the person's own policies, each atom with its provenance, the rows it read and the rule it ran, so that what is true now is always shown beside why; and the **queue**, the items of this paragraph. The record and the

state are one thing seen twice: the record is the rows, the state is the node's fold of them, and the outcome of every choice is a row appearing in a view. **Every string the client shows is served by the node as a text reference,** an id with parameters resolved for the locale the client asked for; the client holds no copy of its own, and a string the node cannot localize is served in its source language and marked so, never invented. **The way there is the same contract:** while a node's bundle lacks a locale it serves the reference with its text in the source language and the language marked, and a consumer MAY resolve that id from a bundle it still holds, preferring the node's text whenever the node serves it in the requested locale; the ids are one namespace shared by every consumer, so the strings a consumer still holds and the ids a node serves can be compared, and what one has and the other lacks is the measured distance between the surfaces built by hand and the surfaces rendered from the view and the queue, closed one card at a time. **The client holds the keys of its deployment and no policy:** the person's own user key and the node's key, and the agent's key where an agent runs, two or three hybrid key sets by deployment, each signing only its own rows (CC 3.2); a view is read through the person's trust set (CC 4.4.3.8) and the client decides nothing the node did not. **How the view makes the queue (normative).** The queue is the door run forwards. For an object the person can see, the node holds the object's family, its lifecycle state, and the person's standing toward it, the roles the held rows give them: author, subject, member, moderator, founder, steward, holder, witness. A family's lifecycle is the transitions of its row in the family model, each a verb with the state it leaves and reaches, the roles that may sign it, its governance row and whether it widens; the queue is exactly the transitions enabled for this person in this state, one item each, and an item's writes are the row that transition signs. The admission door is the same predicate, so the queue never offers what the door would refuse and the door never admits what the queue would not have offered; a stranger to an object sees no item on it; an item that moved the object's state is gone from the next view; a widening item is confirmed before it is signed and a narrowing one takes one step. A node derives the queue from the same held rows and the same policy it admits by, never from a separate list. **An item never crosses between nodes:** a node builds it for its own client from the rows it holds, the policy table and the family model, and only the rows an answer signs travel; a person's other devices learn the answer by the row, so each node keeps its own queue, an item clears when its row arrives, and no queue is synchronised. The queue never blocks: the transport carries signed rows and cosignatures and reports that a decision row landed, and the host decides from the rows which items present themselves.